

Shipyard

Cursor Project Handoff Pack

Open-source CI/CD + artifact/package/OCI platform

This PDF consolidates the Shipyard architecture, current design contract, UI/UX research, design-system research, implementation-agent instructions, and historical design drafts into one document for use in Cursor or another coding agent.

Reading precedence

When two sections conflict, use this order:

1. **DESIGN.md** - canonical frontend/design implementation contract.
2. **Shipyard Developer Architecture Specification** - canonical backend, deployment, storage, runner, registry, pipeline, cluster, and implementation architecture.
3. **Shipyard UI/UX Design Specification** - canonical application layout and interaction guidance.
4. **Research-Based Design System Specification and Research Workbook** - supporting evidence and rationale for the current design system.
5. **Agent Prompts** - working instructions for coding agents and design exploration.
6. **Historical / superseded design material** - reference only; do not override the current documents above.

The project must preserve support for:

Standalone / single node
Distributed / multiple runner machines
Cluster / HA control plane

The core product lineage is:

source -> pipeline -> job -> artifact/package/image -> release -> deployment

Current canonical material

Shipyard Developer Architecture Specification

System architecture and implementation guidance covering:

- Go control plane

- PostgreSQL
- filesystem and S3-compatible storage
- CI/CD pipeline engine
- distributed runners
- BuildKit
- OCI/Docker registry
- artifact/package repositories
- releases and deployments
- authentication, RBAC, secrets, and audit logging
- standalone, distributed, and cluster/HA operation
- scheduling, leases, fencing, failure behavior, observability, and testing
- phased implementation plan

DESIGN.md

Normative frontend contract intended to be checked into or provided alongside the repository. It defines the current visual identity and the rules coding agents should follow while implementing UI.

Current design direction includes:

- IBM Plex Sans + IBM Plex Mono
- Tabler React icons
- Simple Icons for vendor marks
- React Aria Components as accessible low-level behavior
- TanStack Query/Table/Virtual
- React Flow for pipeline DAGs
- Monaco for YAML/code
- purpose-built virtualized log viewing
- graphite surfaces, Shipyard blue, restrained dock orange
- 4px spacing grid
- compact 6px/8px geometry
- medium-high information density
- explicit anti-generic / anti-AI-dashboard guardrails

Shipyard UI/UX Design Specification

Product-wide UI patterns for dashboard, projects, pipelines, job logs, artifacts, OCI registry, packages, releases, deployments, runners, cluster health, responsive behavior, accessibility, and quality gates.

Research-Based Design System Specification

Research synthesis based on major open-source developer/admin platforms and design systems, including GitLab/Pajamas, Grafana/Saga, Nextcloud, PatternFly, Harbor, Gitea, Argo CD, and Portainer.

Research-Based Design System Workbook

Tabular research matrix, qualitative community signals, principles, tokens, typography, geometry/density, frontend stack, icon map, screen blueprints, accessibility requirements, anti-generic guardrails, and source URLs.

Shipyard Agent Prompts

Contains:

- coding-agent / Cursor master implementation prompt
 - Claude Design UI concept exploration prompt
-

Historical material - important warning

The final sections of this PDF contain earlier Shipyard design drafts created before the research-based redesign.

They are included because this is an archival “all docs” handoff, but they are **superseded**.

Do not let these older sections override `DESIGN.md`, the current UI/UX specification, or the research-based design-system material.

The most important superseded decisions include the earlier Inter/Lucide/shadcn-oriented direction. The current direction instead favors IBM Plex, Tabler icons, React Aria-style low-level accessible behavior, Shipyard-owned styling, denser layouts, and stronger anti-generic visual rules.

Historical files included:

- Shipyard Design System Spec - earlier draft
 - Shipyard Design Tokens - earlier workbook
 - `DESIGN.mmd` - earlier repo design contract
-

Suggested Cursor usage

Add this PDF to the project context, then tell Cursor:

Read the Shipyard handoff PDF before making architectural or frontend decisions. Follow the document precedence in the handoff manifest. Treat `DESIGN.md` and the current architecture/UI specifications as normative. Historical sections are reference only. Inspect the repository before writing code, reuse existing implementations, and implement the next coherent vertical slice instead of generating broad scaffolding.

For frontend work, additionally require the agent to compare its implementation against the anti-generic UI guardrails before calling a page complete.

SHIPYARD

Developer Architecture & Implementation Specification

CI/CD + Artifact Hosting + Package & OCI Registry

STANDALONE

Single node

DISTRIBUTED

Multiple machines

CLUSTER / HA

Multiple control-plane nodes

Draft v0.1 • 9 August 2026

Implementation-oriented system design for an independently developed open-source platform inspired by public CI/CD and artifact-management concepts.

Document purpose

This document defines how to build Shipyard: an open-source platform that combines pipeline orchestration, distributed build execution, artifact hosting, package repositories, an OCI/Docker-compatible registry, release provenance, deployments, secrets, access control, and observability in one product. The architecture intentionally supports a one-machine installation first, then adds remote workers, and finally active-active clustered control planes without replacing the core server or changing the public API.

Compatibility goal — Aim for functional parity with the useful public concepts of Jenkins and artifact repository managers, but implement Shipyard independently. Do not copy proprietary implementation code, UI assets, trademarks, or undocumented behavior. Protocol compatibility such as OCI should follow the governing public specification.

Table of contents

1. Product principles and scope
2. Deployment modes: standalone, distributed, cluster/HA
3. Recommended technology stack
4. Logical architecture
5. Server/control-plane components
6. Pipeline engine and state model
7. Runner architecture and executors
8. Artifact, package, and OCI registry architecture
9. Storage and data model
10. Clustering, coordination, and failure recovery
11. API and protocol design
12. Authentication, authorization, secrets, and security
13. Observability and operations
14. Packaging and installation
15. Repository layout and engineering standards
16. Testing strategy
17. Implementation phases and acceptance criteria
18. OSS licensing and community model
19. Web UI/UX and design system
- Appendix A. Example shipyard.yml
- Appendix B. Example standalone Compose layout
- Appendix C. Source references

Status and design conventions

- MUST / MUST NOT indicate requirements that should be treated as release gates.
- SHOULD indicates a strong default that can be changed with a documented reason.
- MAY indicates an optional capability or later phase.
- References such as [S4] point to Appendix C. Sources are used to justify standards and external technology behavior; Shipyard-specific choices remain design decisions.

1. Product principles and scope

1.1 Core product statement

Shipyards is a software delivery control plane. Source code enters through a Git integration or manual trigger; pipeline jobs execute on local or remote runners; outputs become content-addressed artifacts; packages and OCI images are published into first-class repositories; releases bind artifacts to commits and build provenance; deployments reference immutable release outputs. A user should be able to trace commit -> pipeline -> job -> artifact -> release -> deployment from one UI and API.

1.2 Product principles

Principle	Meaning
One product, one model	CI/CD and artifact management must share identities, projects, permissions, build provenance, retention rules, and audit history.
Single-node first	A small team must be able to run the complete usable product on one Linux machine.
Scale without edition changes	Remote runners and clustered control planes use the same server code, schema, configuration model, API, and OSS license.
Standards over lock-in	Use OCI Distribution/Image specs for container registry behavior [S4][S5], OIDC for external identity [S13], and OpenTelemetry for telemetry [S12].
Durable pipelines	Pipeline state is persisted so controller restarts do not erase completed work. Jenkins documents resumable pipeline flows as an important behavior [S1].
Artifacts are first-class	Every produced artifact should retain digest, size, producer job, source commit, build, repository, retention state, and release/deployment relationships.
Secure defaults	Runners are untrusted compute relative to the control plane. Secrets are scoped and short-lived; build containers do not receive unrestricted control-plane credentials.
Operable by humans	Backups, upgrades, migrations, drain operations, cluster health, and disaster recovery are product features, not wiki-only procedures.

1.3 Initial functional scope

- Organizations/workspaces, projects, users, service accounts, teams, RBAC and audit logs.
- GitHub/GitLab/Forgejo/Gitea-compatible webhook and repository integrations through adapters; generic Git URL support.
- Pipeline-as-code using shipyard.yml, DAG jobs, dependencies, retries, timeouts, variables, matrices, manual approvals, schedules and concurrency groups.
- Runner registration, labels/capabilities, pools, health, draining and job leases.
- Shell/process executor, Docker/container executor, BuildKit image builds, and Kubernetes Job executor.

- Generic artifact repositories plus OCI/Docker registry; npm and Maven are the recommended first package-specific protocols after generic + OCI.
- Local, remote-cache, and virtual repository concepts based on public repository-manager patterns [S3].
- Artifact retention, immutable releases, provenance links, checksums, download statistics, replication hooks, and garbage collection.
- Docker Compose installation for standalone, package/binary installation for advanced users, and Helm for Kubernetes.
- Prometheus-compatible metrics, structured logs, traces via OpenTelemetry, health endpoints, and admin diagnostics.

1.4 Explicit non-goals for v1

- Implement every package ecosystem before the storage and repository abstractions are stable.
- Invent a custom container image format or Docker-compatible protocol. Shipyard should implement OCI standards instead [S4][S5].
- Build a custom container build engine. Delegate Dockerfile/OCI image builds to BuildKit [S6].
- Require Kubernetes for normal installation.
- Introduce a large microservice fleet for the first release. Start with a modular server process plus independent runners.
- Run arbitrary build workload inside an HA control-plane process in production.

2. Deployment modes: standalone, distributed, cluster/HA

Deployment mode is a foundational requirement, not an enterprise-only extension. The same Shipyard server binary must be able to participate in all modes. The main differences are how many control-plane processes exist, where runners execute, and whether stateful dependencies are local or shared.

2.1 Mode A — Standalone / single node

Target — A complete useful Shipyard installation on one machine, suitable for a developer, homelab, small company, or evaluation.

Component	Standalone placement	Notes
Shipyard server	Local process/container	API, web UI, scheduler, registry endpoints, artifact metadata, auth, background jobs.
PostgreSQL	Local container/service	Use PostgreSQL even on one node so the schema and transaction model are identical to clustered deployments.
Artifact blob store	Local filesystem backend by default	Content-addressed layout. Allow S3-compatible storage as an option from day one.
Runner	Local shipyard-runner	May use shell/process, Docker, and BuildKit executors.
BuildKit	Local daemon/container	Used for OCI/Docker image builds and cache operations [S6].

Component	Standalone placement	Notes
Reverse proxy/TLS	Optional	Shipyards can expose HTTP directly; production installs should normally terminate TLS at Caddy, nginx, Traefik, or an ingress.

The official quick-start should be Docker Compose because Compose is designed to define and run a multi-container application from one YAML configuration [S7]. Persistent database and local data should use named volumes; Docker documents volumes as its preferred persistence mechanism for container-generated data [S8].

```
docker compose up -d

# Services in the reference stack:
# shipyard-server
# postgres
# shipyard-runner
# buildkitd
```

2.2 Mode B — Distributed / multiple machines

Target — One authoritative Shipyard control plane, but build capacity is distributed across many physical machines, VMs, or Kubernetes clusters.

This is intentionally distinct from HA clustering. It solves compute scale and platform diversity without requiring multiple control-plane replicas. Jenkins similarly separates a controller that schedules work from agents that execute it, with agent labels/capabilities for workload selection [S2]. Shipyard should use the same general distributed-execution concept while implementing its own protocol and lifecycle.

Machine	Typical processes	Failure effect
Control node	shipyard-server + PostgreSQL + optional local/S3 blob store	If it fails, new scheduling/API operations stop until recovered; already-running runners should preserve logs and final result upload retries.
Linux worker 01	shipyard-runner + Docker/BuildKit	Only jobs assigned to this worker are affected.
Linux worker 02	shipyard-runner	Adds parallel capacity.
Windows worker	shipyard-runner	Runs Windows-native jobs; do not assume BuildKit Linux parity on Windows [S6].
macOS worker	shipyard-runner	Used for macOS/iOS signing/build workflows.

```
shipyard runner register \
  --server https://shipyard.example.com \
  --token <one-time-registration-token> \
  --labels linux,amd64,docker,high-memory
```

Runners maintain outbound authenticated connections to the server whenever possible. This makes workers usable behind NAT and reduces inbound firewall requirements. The server assigns jobs using labels/capabilities, capacity, pool policy, affinity, and current load.

2.3 Mode C — Cluster / high availability

Target — Multiple active Shipyard control-plane nodes on separate machines or pods, behind a load balancer, with no single Shipyard application node required for normal service availability.

Cluster mode is active-active for HTTP API, web UI, OCI registry reads/writes, runner coordination endpoints, and most stateless operations. Singleton or ownership-sensitive background functions use database-backed election/leases. All control-plane replicas share PostgreSQL and the same durable artifact/object storage.

Layer	Cluster requirement	Recommended v1 design
Load balancing	Any healthy server can receive API/registry traffic	L4/L7 load balancer; no sticky session required after auth/session design is stateless.
Database	Shared transactional source of truth	External PostgreSQL. Production HA PostgreSQL is deployment infrastructure, not reimplemented by Shipyard.
Blob storage	Shared durable namespace	S3-compatible object storage strongly recommended. Local filesystem is not valid for multi-control-plane HA unless backed by a proven shared filesystem.
Control plane	2+ supported; 3 recommended for operational redundancy	All replicas active. Health and readiness gates prevent traffic before migrations/config are valid.
Singleton work	Exactly one active owner for selected tasks	PostgreSQL advisory lock + explicit lease/fencing record. PostgreSQL provides application-defined advisory locks that are automatically cleaned up at session end [S9].
Events	DB outbox initially; optional broker later	PostgreSQL transactional outbox for correctness. NATS JetStream may be added for high-volume durable event fan-out [S15].
Runners	Connect through LB or any control-plane node	Jobs are leased from shared DB state, so runner ownership can move when a control-plane node fails.

2.4 Required upgrade path between modes

1. Install Standalone with PostgreSQL and filesystem or S3 blob storage.
2. Add remote runners. The installation is now Distributed; no control-plane migration is required.
3. Move blob storage from filesystem to S3-compatible storage if not already using it. Provide a built-in online/copy-verify migration command.
4. Move PostgreSQL to a production/HA PostgreSQL deployment if required.

5. Start additional shipyard-server replicas with the same cluster ID, encryption key material, DB DSN, and object-store configuration.
6. Put all server replicas behind a load balancer and enable cluster mode checks.
7. Drain the original standalone server only if desired; it can remain as one cluster member.

2.5 Invariants across every deployment mode

- One PostgreSQL schema and migration history.
- One API surface and authentication model.
- One runner protocol.
- One blob-storage interface with filesystem and S3 implementations.
- One scheduler state machine; cluster mode changes ownership/coordination, not semantics.
- No “enterprise-only” database tables or alternative proprietary coordinator required for HA.
- Every asynchronous operation must be idempotent or fenced so it is safe under retry/failover.

3. Recommended technology stack

Area	Technology	Reason
Backend	Go	Strong concurrency/networking ecosystem, simple static deployment, excellent fit for runners/registry services; BuildKit exposes Go integration points [S6].
Frontend	React + TypeScript + Vite	Fast SPA development; type-safe API client generation; suitable for live pipeline/log interfaces.
UI foundation	Tailwind CSS + shadcn/ui + TanStack Query	Composable UI, consistent design system, server-state caching.
Primary DB	PostgreSQL	Transactions, JSONB, locking, row-level security options, advisory locks and mature operations [S9][S10].
Blob storage	Filesystem + S3-compatible interface	Simple standalone path plus scalable shared storage for clusters.
Image builder	BuildKit	Reusable build graph, modern cache semantics and registry-exportable cache [S6].
Container registry	Native OCI Distribution API implementation	Interoperability with Docker/OCI clients; OCI Distribution standardizes push/pull/discovery/content management [S4].
Runner protocol	ConnectRPC/gRPC over HTTP/2 or WebSocket fallback	Streaming logs, typed RPC, heartbeats and bidirectional control.
Public API	REST + OpenAPI	Easy automation and language-neutral SDK generation.
Async events	PostgreSQL outbox first; NATS JetStream optional	Avoid mandatory broker for standalone; add durable fan-out when scale requires it [S15].

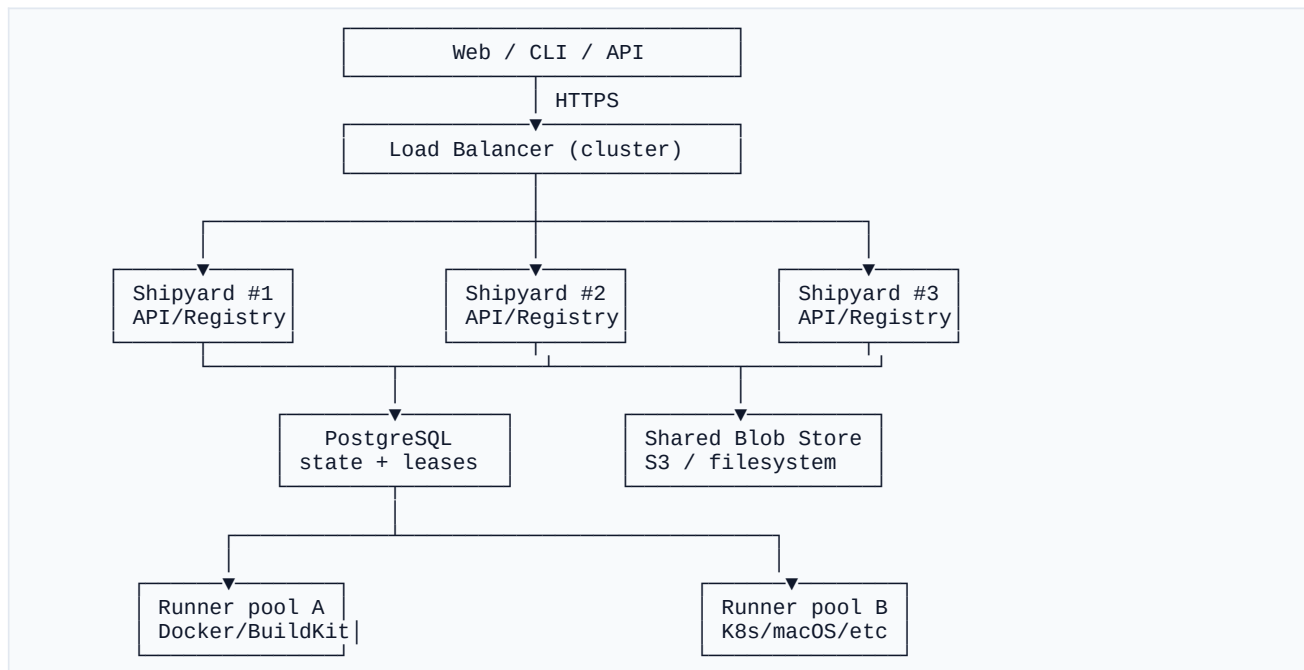
Area	Technology	Reason
Observability	OpenTelemetry + Prometheus endpoint	Vendor-neutral traces, metrics and logs [S12].
External auth	OIDC	Standard interoperable identity layer on OAuth 2.0 [S13].
Packaging	Docker/Compose + standalone binaries + Helm	Covers single host, VM/bare-metal and Kubernetes deployments.

3.1 Why a modular monolith first

The first production server should be one Go codebase with strict internal package boundaries rather than many networked microservices. The scheduler, registry metadata API, artifact metadata, auth, web API, release service, webhook processor, and administrative background jobs can share transactions safely. Runners remain separate processes because build workloads must be isolated from the control plane. Modules can later be separated behind interfaces if profiling shows a real scaling need.

Rule — Do not create a network hop merely to make an architecture diagram look distributed. Split a module only when it needs an independent security boundary, failure domain, scaling profile, or release lifecycle.

4. Logical architecture



4.1 Trust boundaries

Boundary	Trust assumption	Controls
Browser/CLI -> server	Untrusted client input	TLS, auth, CSRF protections for browser flows, request validation, rate limiting, audit.
Runner -> server	Runner may execute attacker-controlled repo code	Narrow runner token scope, signed job leases, no direct DB access, per-job credentials, output limits.
Build container -> runner	Build is hostile by default	Container/process isolation, workspace ownership, restricted mounts, optional rootless mode, no server master secrets.
Server -> object store	Trusted service but network can fail	Digest verification, conditional writes where available, retry/idempotency, server-side encryption options.
Server -> Git provider	External system	Least-privilege tokens/apps, webhook signature validation, retry queue.

5. Server/control-plane components

Module	Responsibilities
HTTP/API gateway	Routing, request IDs, auth middleware, rate limits, OpenAPI handlers, registry routes.
Identity service	Users, service accounts, sessions/tokens, OIDC providers, SSH/public keys if needed.
Organization/project service	Tenancy, membership, teams, permissions, quotas and project settings.
SCM integration service	Repository connections, webhooks, branch/PR discovery, commit status callbacks.
Pipeline compiler	Parse/validate shipyard.yml, expand matrices, produce immutable DAG execution plan.
Scheduler	Find runnable jobs, enforce dependencies/concurrency, match runner capabilities, grant leases.
Runner coordinator	Registration, heartbeat, capacity, log streams, cancellation, result collection.
Artifact service	Blob upload/download, digest verification, metadata, retention, provenance.
Repository service	Local/remote/virtual repositories, package routing, upstream cache policy [S3].

Module	Responsibilities
OCI registry service	Implements OCI Distribution routes and authentication scopes [S4].
Release service	Immutable release manifests, artifact sets, approvals, promotion and deployment links.
Deployment service	Environment records, rollout hooks/adapters, status, approvals, rollback references.
Secret service	Encrypted secret values, versions, scopes, runner envelope delivery, masking metadata.
Audit service	Append-only security/administrative event record with actor, target, request and outcome.
Maintenance service	GC, retention, webhook retries, expired leases/tokens, usage aggregation, cleanup.

5.1 Server process model

```
cmd/shipyard-server
├── api/http
├── api/registry
├── auth
├── scm
├── pipelines/compiler
├── pipelines/scheduler
├── runners/coordinator
├── artifacts
├── repositories
├── releases
├── deployments
├── secrets
├── audit
├── maintenance
└── platform/{db, blobstore, events, telemetry}
```

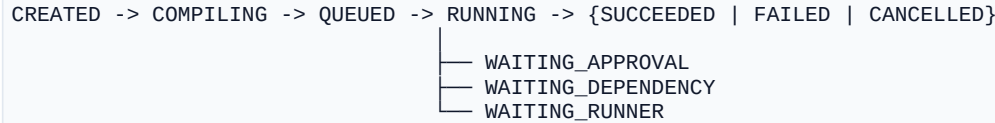
5.2 Background worker rules

- Every job type has a stable unique work key and idempotency key.
- Use short database transactions. Never hold a SQL transaction open while waiting for Git, object storage, a runner, or human approval.
- Cluster-singleton loops must hold a coordination lease, not rely on “only one process normally runs this.”
- Long-running activities persist checkpoints/state transitions before external side effects.
- All externally visible side effects should have retry-safe semantics or a recorded delivery token.

6. Pipeline engine and state model

Shipyard should preserve the strongest idea from Jenkins Pipeline as Code: the pipeline definition lives in the source repository and is versioned with the code [S1]. Shipyard should not use Groovy. The v1 format should be declarative YAML compiled into an internal immutable DAG and state machine.

6.1 Pipeline lifecycle



The pipeline record stores the triggering commit and an immutable copy/digest of the compiled plan. Re-reading a changed shipyard.yml during an already-running pipeline is forbidden.

6.2 Job lifecycle



6.3 DAG compiler responsibilities

- Parse YAML and reject unknown keys in strict mode.
- Resolve includes/reusable actions only from pinned versions or immutable digests when policy requires.
- Expand matrix jobs before scheduling and assign stable node IDs.
- Validate dependency cycles and missing dependencies.
- Validate executor requirements, secrets references, environment names, timeouts and resource limits.
- Produce a canonical JSON execution plan, hash it, and persist it with the pipeline.

6.4 Scheduler query model

The scheduler should use the database as the source of truth. Runnable jobs are selected in small batches using row locks such as `SELECT ... FOR UPDATE SKIP LOCKED`, then transitioned transactionally to an offered/leaseable state. A runner lease includes a monotonically increasing attempt number and fencing token. Results from an older attempt are rejected after a lease is superseded.

```

Pseudo-flow:
1. BEGIN
2. SELECT eligible jobs ... FOR UPDATE SKIP LOCKED LIMIT N
3. Match against healthy runner capacity
4. INSERT/UPDATE job_attempt with lease_id + fencing_token + expiry
5. COMMIT
6. Notify chosen runner
7. Runner ACKs lease; otherwise lease expires and job returns to queue
  
```

6.5 Required pipeline features

Feature	v1 behavior
Triggers	push, pull_request/merge_request, tag, manual, schedule, API/webhook.
Dependencies	needs: [jobA, jobB]; DAG execution.
Conditions	Expression language over trigger, git, prior results, inputs and variables.
Retries	Per-step or per-job retry count with exponential backoff and retry reason.

Feature	v1 behavior
Timeouts	Pipeline, job and step timeout.
Manual approval	Durable WAITING_APPROVAL state with RBAC and audit; Jenkins documents human input/gates as a pipeline use case [S1].
Concurrency groups	Serialize deployments or named critical resources; optional cancel-in-progress.
Matrices	Static expansion of OS/JDK/version combinations.
Artifacts	Upload/download action integrated with provenance.
Cache	Named cache backed by blob store with scoped keys; separate from immutable artifacts.
Environments	staging/production objects with approvals, secrets, protection rules and deployment history.
Restart/resume	Completed steps/jobs remain persisted across server restart; running leases reconcile after restart.

7. Runner architecture and executors

Runners provide execution capacity, analogous at a high level to distributed Jenkins agents [S2], but Shipyard should use an independently designed protocol. Runners never connect directly to PostgreSQL or the object-store master credentials. They obtain scoped, short-lived job capabilities from the control plane.

7.1 Runner identity and registration

1. Admin creates a one-time registration token scoped to organization/project/pool.
2. Runner generates a local keypair and exchanges the one-time token for a durable runner identity/certificate or signed token.
3. Server records runner ID, pool, labels, architecture, OS, executor capabilities, version and last heartbeat.
4. Runner token can request work and upload only for leases assigned to that runner; it cannot administer projects.
5. Runner supports drain mode: finish existing leases but accept no new work.

7.2 Runner selection

```

eligible(job, runner) =
  runner.online
  AND runner.pool in allowed_pools
  AND job.required_labels ⊆ runner.labels
  AND runner.available_slots > 0
  AND policy_allows(job.project, runner)
  AND executor_supports(job.executor)

```

7.3 Executor types

Executor	Use	Isolation/notes
process/shell	Fast trusted/self-hosted builds	Native host process. Highest host risk; document clearly.
docker	General Linux CI	Create per-job containers/networks/volumes; configurable privileged mode should default off.
buildkit	OCI image build	Runner talks to BuildKit; cache can be exported/imported through registry [S6].
kubernetes	Ephemeral cluster CI	Create a Kubernetes Job per Shipyard job. Kubernetes Jobs represent one-off tasks that run to completion [S11].
ssh/remote	Later	Adapter for controlled legacy machines; not required for first release.
windows-process	Windows builds	Native runner implementation; container building has separate BuildKit limitations [S6].
macos-process	Apple platform builds	Native runner; code signing secrets need dedicated protected-runner policy.

7.4 Workspace layout

```

/var/lib/shipyard-runner/
  work/<job-attempt-id>/
    repo/
    tmp/
    outputs/
    metadata.json
    cache/
    state/runner.json

```

Workspaces must be uniquely keyed by job attempt, not just project name. Cleanup occurs after upload/finalization. A diagnostic retention option may retain failed workspaces on explicitly trusted runners.

7.5 Log streaming

- Runner chunks stdout/stderr with sequence numbers and timestamps.
- Control plane persists chunks to an append-oriented blob/log backend or compressed object segments; DB stores index/metadata, not every line.
- Web clients stream through SSE/WebSocket from the server; clients can resume from last sequence number.
- Known secret values and generated credential patterns are masked before storage and again before display.
- Enforce maximum line size, total log limits and binary-output handling.

8. Artifact, package, and OCI registry architecture

8.1 Repository model

Shipyards should model artifact repositories using three v1 repository types: local, remote, and virtual. This pattern is publicly documented by JFrog: local repositories hold internally generated artifacts, remote repositories cache upstream sources, and virtual repositories aggregate local/remote repositories behind one resolution endpoint [S3]. Shipyards should implement the concept independently.

Type	Shipyards behavior
Local	Authoritative writable repository. Accepts internally published packages/artifacts.
Remote	Read-through proxy/cache for an upstream repository. Optional negative cache, checksum validation, allow/deny paths and offline mode.
Virtual	Ordered aggregate of local and remote repositories. Usually read endpoint; publish target must be explicitly configured if supported.

8.2 Content-addressed blob store

Artifact bytes are stored by digest while repository paths/tags are metadata references. The OCI Distribution spec defines blobs as digest-addressable binary content [S4], and Shipyards should reuse that same content-addressed approach across generic artifacts and package storage where feasible.

```
blobs/sha256/ab/cd/abcdef...
```

Database reference:

```
repository_path -> blob_digest -> size/media_type/storage_key
```

- Compute SHA-256 while streaming upload; reject declared digest mismatch.
- Write upload to a temporary/staging key, then atomically/conditionally publish final content key.
- Deduplicate identical blobs across repositories when tenancy/encryption policy permits.
- Never delete a blob just because one repository reference is removed. Garbage collection checks all references and grace periods.
- For standalone filesystem storage, use the same blob-store interface as S3 so higher layers do not branch on deployment mode.

8.3 OCI / Docker-compatible registry

The registry must implement the OCI Distribution Specification rather than a custom Shipyards image API. OCI Distribution standardizes content distribution and is based on the Docker Registry HTTP API V2 [S4]. OCI Image defines interoperable manifests, indexes, layers and configuration [S5].

```
Required route family (conceptual):
GET /v2/
GET /v2/<name>/manifests/<reference>
PUT /v2/<name>/manifests/<reference>
HEAD /v2/<name>/blobs/<digest>
GET /v2/<name>/blobs/<digest>
POST /v2/<name>/blobs/uploads/
PATCH/PUT upload session routes
GET /v2/<name>/tags/list
...plus required OCI errors/headers and referrers behavior per supported spec version
```

8.4 Package protocol roadmap

Phase	Protocols	Goal
P0	Generic files + OCI/Docker	Prove blob model, auth scopes, retention and registry scale.
P1	Maven + npm	Cover common Java/JVM and JavaScript ecosystems.
P2	PyPI + NuGet + Helm	Broaden developer workloads.
P3	Cargo + Go modules + RubyGems	Long-tail expansion through protocol adapters.

Each package implementation must be an adapter over common repository/blob/provenance primitives. Do not create separate storage silos per package type.

8.5 Artifact provenance

Every build-produced artifact record should include producer pipeline ID, job attempt ID, repository commit SHA, branch/tag, runner identity, creation time, digest, size, media type, package coordinates if known, and release/deployment references. Jenkins documentation highlights artifact fingerprinting for tracing an artifact through downstream use [S1]; Shipyard should make stronger provenance part of the core schema rather than an optional plugin.

8.6 Remote cache behavior

- Cache key is upstream URL/coordinates + repository ID, not only local path.
- Persist upstream ETag/Last-Modified where protocol permits.
- Negative-cache missing artifacts for a short configurable TTL.
- Remote checksum/digest mismatch is a hard failure and security event.
- Offline mode serves only cached content.
- Admin can evict a cached path or repository namespace without affecting local repositories.

9. Storage and data model

9.1 PostgreSQL responsibilities

- Authoritative metadata and workflow state.
- Organizations, users, projects, permissions, tokens and audit indexes.
- Pipelines, jobs, attempts, leases, approvals and schedules.
- Repositories, package metadata, tags/paths, blob references and retention policies.
- Releases, environments and deployment records.
- Cluster membership/heartbeats, leader leases, migration/version state.
- Outbox events for reliable asynchronous publication.

9.2 Blob store responsibilities

- Artifact/package bytes.
- OCI blobs/manifests when not stored inline as small metadata.
- Pipeline logs and test-report payloads.
- Large generated SBOM/provenance documents.

- Cache archives and BuildKit-related exported cache objects where applicable.

9.3 Suggested core tables

Table group	Representative tables
Identity	users, identities, sessions, api_tokens, service_accounts, organizations, memberships, teams, roles, role_bindings
SCM	scm_connections, repositories_scm, webhook_deliveries, commits, refs
Pipeline	pipeline_definitions, pipeline_runs, jobs, job_attempts, job_dependencies, approvals, schedules, concurrency_locks
Runners	runner_pools, runners, runner_labels, runner_sessions, runner_leases
Artifacts	blob_objects, artifact_records, artifact_references, artifact_downloads, retention_policies
Repositories	repositories, remote_configs, virtual_members, package_versions, package_properties
Releases	releases, release_items, environments, deployments
Security	secrets, secret_versions, secret_bindings, audit_events
Cluster	cluster_nodes, distributed_leases, schema_migrations, outbox_events

9.4 Tenant isolation

Application authorization remains mandatory. PostgreSQL Row-Level Security may be used as a defense-in-depth layer for selected tenant-owned tables; PostgreSQL supports policies that control which rows roles can see or modify [S10]. Do not rely on RLS alone because many server operations are cross-tenant administrative/system tasks.

9.5 Migrations

- Every migration has a monotonically increasing version and checksum.
- Cluster startup checks database compatibility before readiness.
- Only one node executes schema migration at a time using a global advisory lock.
- Prefer expand/contract migrations compatible with rolling upgrades: add nullable/new fields first, deploy code, backfill, then remove old fields in a later release.
- Never require all nodes to stop for routine minor-version upgrades unless a release is explicitly marked non-rolling.

10. Clustering, coordination, and failure recovery

10.1 Cluster membership

Each shipyard-server replica receives a stable `cluster_id` shared by the installation and a unique `node_id` generated/persisted per replica. Nodes heartbeat into `cluster_nodes` with `version`, `started_at`, `last_seen_at`,

advertised endpoint, build revision and capability flags. A node is considered unavailable only after a timeout; node health is advisory while durable leases determine ownership.

10.2 Leader election and singleton responsibilities

PostgreSQL advisory locks are suitable for coarse singleton ownership because PostgreSQL exposes application-defined advisory locks and session-level locks are released when the session ends [S9]. Shipyard should combine that property with explicit lease records and fencing tokens rather than relying on advisory locks as the complete distributed correctness model.

```
Example singleton roles:
- retention/gc coordinator
- schedule tick coordinator
- repository remote-cache maintenance coordinator
- schema migration lock
- usage aggregation coordinator

Acquire pattern:
1. Open dedicated DB session.
2. pg_try_advisory_lock(hash64(cluster_id, role)).
3. If acquired, increment fencing_token in distributed_leases.
4. Renew lease heartbeat periodically.
5. Side-effect handlers verify fencing_token before committing ownership-sensitive results.
6. On DB session loss, advisory lock releases; another node can acquire.
```

10.3 Why fencing tokens matter

A former leader can be paused by a network or runtime stall and later resume after a new leader is elected. A monotonically increasing fencing token makes stale ownership observable: storage/database operations that depend on exclusive ownership carry the current token, and results from an older token are rejected. This avoids “two leaders both think they own it” bugs even when lock loss detection is delayed.

10.4 Job leases and runner failover

1. Scheduler creates job_attempt N with lease ID, runner ID, expiry and fencing token.
2. Runner ACKs and starts work. Heartbeats renew the job lease.
3. If the control-plane node handling the runner connection dies, the runner reconnects to another healthy node and presents its lease ID.
4. The new node validates the shared DB lease and resumes log/control traffic without changing the attempt.
5. If the runner itself disappears past expiry, the attempt becomes LOST. Retry policy decides whether a new attempt N+1 is scheduled.
6. Late output from attempt N is rejected if attempt N+1 has already fenced it out.

10.5 Failure matrix

Failure	Standalone	Distributed	Cluster/HA
Runner dies	Job lost/retry	Only that worker capacity affected	Only that worker capacity affected
Shipyard server dies	Control plane unavailable until restart	Control plane unavailable; remote runner jobs reconcile after return	Load balancer routes to other nodes; leases reconnect/reconcile
One cluster node dies	N/A	N/A	Singleton locks release after DB session loss; other node assumes work

Failure	Standalone	Distributed	Cluster/HA
PostgreSQL unavailable	Service degraded/unavailable	Service unavailable for authoritative state changes	All control nodes degraded; DB HA is therefore a critical dependency
Object store unavailable	Artifact operations fail/retry	Artifact operations fail/retry	Metadata/API may remain partially available; artifact/registry traffic degrades
Network partition runner <-> server	Runner buffers/retries logs; job eventually lease-expires	Same	Runner can reconnect through LB; lease fencing prevents stale completion

10.6 Eventing

The v1 correctness path should use a transactional outbox table: business-state mutation and event creation occur in the same PostgreSQL transaction. A dispatcher publishes outbox rows to in-process subscribers/webhooks. This avoids requiring a message broker in Standalone. If deployments need very high fan-out or cross-service separation later, NATS JetStream provides durable streams/consumers and replicated key-value/object abstractions [S15].

11. API and protocol design

11.1 Public REST API

```

/api/v1/auth/...
/api/v1/orgs/...
/api/v1/projects/...
/api/v1/pipelines/...
/api/v1/runs/...
/api/v1/jobs/...
/api/v1/runners/...
/api/v1/artifacts/...
/api/v1/repositories/...
/api/v1/releases/...
/api/v1/environments/...
/api/v1/deployments/...
/api/v1/admin/...

/v2/... # OCI Distribution API; not wrapped in /api/v1

```

11.2 API requirements

- OpenAPI 3.x specification generated/validated in CI.
- Opaque stable UUID/ULID identifiers; human slugs are mutable aliases.
- Cursor pagination for large collections.
- Idempotency-Key support for mutating automation endpoints where duplicate submission is plausible.
- ETag/If-Match for selected optimistic concurrency operations.
- Structured error envelope with code, message, details, request_id.
- Every API request gets a correlation/request ID included in logs and audit events.
- No API behavior depends on browser-only session state.

11.3 Runner protocol

```

Runner -> Server:
  Register / Authenticate

```

```

Heartbeat(capacity, labels, version)
AckLease(lease_id)
StreamLogs(seq, bytes)
ReportStepState(...)
RequestScopedCredential(resource)
CompleteAttempt(result, outputs)

Server -> Runner:
  LeaseJob(compiled_job, lease_id, fencing_token)
  CancelJob(reason)
  RotateCredential(...)
  Drain(...)

```

Use explicit protocol version negotiation and server-advertised minimum runner version. Rolling upgrades must support at least one previous minor runner protocol when practical.

12. Authentication, authorization, secrets, and security

12.1 Authentication

- Local username/password for standalone/bootstrap, using a modern password hash such as Argon2id.
- OIDC for production external identity; OpenID Connect defines an interoperable identity layer on OAuth 2.0 [S13].
- API tokens with scopes, expiration, last-used metadata and revocation.
- Service accounts for automation.
- Runner identities separate from human/API identities.

12.2 RBAC model

```

Scopes:
installation -> organization -> project -> repository/environment

Example verbs:
read, create, update, delete, administer, execute, publish, deploy, approve, manage_secrets

```

Permissions should be resolved into a normalized policy check such as `authorize(actor, action, resource)`. Avoid scattering role-name comparisons across handlers.

12.3 Secrets

- Encrypt secret values at rest with envelope encryption. A master key is supplied by file/env/KMS integration; DB stores encrypted data keys and ciphertext.
- Secrets are scoped to organization/project/environment and optionally protected runner pools.
- Runner receives only secrets required by the active job and only after the lease is validated.
- Prefer short-lived provider credentials through OIDC/workload federation when integrations support it; static credentials remain a fallback.
- Never return secret plaintext through normal GET APIs after creation; provide rotate/replace operations.
- Audit secret metadata access and materialization events without logging values.
- Masking is defense in depth, not a guarantee: users who can run arbitrary code with a secret can intentionally exfiltrate it. Permission design must reflect that reality.

12.4 Runner hardening

- Control-plane nodes should default to zero local build executors in cluster/production deployments; keep hostile build workload off the application servers.
- Docker privileged mode disabled by default and optionally forbidden by organization policy.
- Docker socket mounting into build containers disabled by default; prefer isolated BuildKit endpoint.

- Per-job workspace permissions; cleanup with symlink/path traversal defenses.
- Network egress policies configurable for protected runners.
- Registration tokens single-use and short-lived.
- Protected tags/environments can require protected runner pools.

12.5 OCI registry authorization

Registry authentication should issue short-lived bearer tokens with repository actions such as pull/push rather than sharing a broad Shipyard API token with Docker clients. Token claims map to Shipyard repository permissions. The /v2/ API implementation must remain standards-compatible [S4].

12.6 Supply-chain features roadmap

- SBOM attachment and retention.
- Artifact signatures/attestations stored as OCI artifacts/referrers where appropriate.
- Policy gates: require signed artifacts, approved branches, successful scans or protected environment approval.
- Provenance export containing source commit, builder/runner identity, pipeline plan digest and artifact digests.
- Vulnerability scanning should be an adapter/integration first; do not block core registry delivery on building a scanner from scratch.

13. Observability and operations

OpenTelemetry is a vendor-neutral framework for generating, collecting and exporting traces, metrics and logs [S12]. Instrument Shipyard with OTEL from the beginning so cluster debugging does not depend on proprietary tracing.

13.1 Metrics

```
shipyard_http_requests_total
shipyard_http_request_duration_seconds
shipyard_pipeline_runs_total{result=...}
shipyard_job_queue_depth{pool=...}
shipyard_job_wait_seconds
shipyard_runner_online
shipyard_runner_slots_available
shipyard_registry_requests_total{operation=...}
shipyard_blob_bytes_stored
shipyard_blob_gc_candidates
shipyard_cluster_node_healthy
shipyard_outbox_pending
```

13.2 Health endpoints

Endpoint	Meaning
/health/live	Process is alive. Must not depend on external systems enough to create restart loops.
/health/ready	Node may receive traffic. Checks DB compatibility/connectivity and critical configuration.
/health/cluster	Authenticated admin detail: node membership, lease ownership, schema version, storage status.

Endpoint	Meaning
/metrics	Prometheus exposition; configurable auth/network restriction.

13.3 Audit events

- Login/logout/token creation and revocation.
- Role and membership changes.
- Secret create/rotate/delete/materialization metadata.
- Pipeline manual triggers/cancellations/approvals.
- Repository publish/delete/retention-policy changes.
- Release promotion and production deployment approval.
- Runner registration/drain/revocation.
- Cluster/admin configuration changes.

13.4 Backup and restore

- PostgreSQL backup is required for metadata/workflow state.
- Blob store backup/versioning strategy is required for artifacts if the object store itself does not provide durability guarantees.
- Backup documentation must specify ordering and consistency assumptions.
- Restore command verifies DB schema, encryption key availability and blob-store access before starting workers.
- Provide an integrity scanner that enumerates metadata references and verifies missing/orphaned blobs.

14. Packaging and installation

14.1 Official distribution artifacts

Artifact	Audience
shipyard-server Linux binaries	Bare-metal/VM operators.
shipyard-runner Linux/macOS/Windows binaries	Worker machines.
OCI images for server/runner	Docker/Kubernetes operators.
docker-compose.yml	Official standalone quick-start [S7].
Helm chart	Kubernetes control-plane and optional runner deployment.
CLI binary	Developers and automation.

14.2 Standalone reference Compose

```

services:
  shipyard:
    image: ghcr.io/<org>/shipyard:<version>
    depends_on: [postgres]
    volumes:
      - shipyard_data:/var/lib/shipyard

```

```

postgres:
  image: postgres:<supported-major>
  volumes:
    - postgres_data:/var/lib/postgresql/data

runner:
  image: ghcr.io/<org>/shipyard-runner:<version>
  # runner identity persisted in a volume

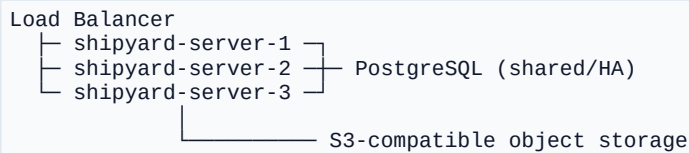
buildkit:
  image: moby/buildkit:<pinned-version>

volumes:
  shipyard_data:
  postgres_data:

```

Pin supported dependency versions in release manifests; do not use floating latest tags in the published production Compose file.

14.3 Cluster reference deployment



Remote runner pools connect to the load-balanced Shipyard endpoint.

14.4 Configuration layering

```

Defaults
< configuration file (shipyard.toml/yaml)
< environment variables
< explicit command-line flags

```

Sensitive values should be injectable via file paths or secret-manager integration, not only plain environment variables. Every configuration value must be documented with default, allowed values, restart/reload requirement, and whether it must match across cluster nodes.

15. Repository layout and engineering standards

15.1 Suggested monorepo

```

shipyard/
cmd/
  shipyard-server/
  shipyard-runner/
  shipyard-cli/
internal/
  api/
  auth/
  scm/
  pipelines/
  runners/
  artifacts/
  repositories/
  registry/
  releases/
  deployments/
  secrets/
  audit/
  cluster/
  platform/
pkg/

```

```

runnerproto/
pipeline-schema/
web/
migrations/
deploy/
compose/
helm/
docs/
architecture/
adr/
api/
tests/
integration/
e2e/
LICENSE
CONTRIBUTING.md
SECURITY.md
CODE_OF_CONDUCT.md

```

15.2 Module dependency rules

- Domain modules depend on interfaces in platform packages, not concrete global singletons.
- HTTP handlers do not contain business transactions; they call application services.
- Database repositories do not call external HTTP services.
- Runner protocol types live in a versioned package independent of the server implementation.
- Artifact/package adapters depend on the common repository/blob abstractions.
- Cluster coordination is infrastructure; domain state machines must remain testable with an in-memory/fake lease provider.

15.3 ADRs

Use Architecture Decision Records for choices that future contributors may otherwise reverse accidentally. Initial ADRs should cover: PostgreSQL everywhere, modular-monolith server, filesystem/S3 blob abstraction, BuildKit delegation, OCI compliance, runner lease/fencing model, outbox-first events, OIDC strategy, and Apache-2.0 licensing recommendation.

16. Testing strategy

16.1 Test layers

Layer	Examples
Unit	Pipeline expression parser, DAG validation, repository path rules, RBAC policy evaluation, digest helpers.
Property/fuzz	YAML/parser fuzzing, OCI route/path normalization, archive extraction, expression evaluation.
Database integration	Migrations, transactional scheduler claims, lease expiry, deadlock/retry paths, RLS policy tests.
Protocol conformance	OCI Distribution/Image compatibility suites where available; Docker/OCI client push/pull behavior.
Runner integration	Process/Docker executor lifecycle, cancellation, log resume, reconnect and fencing.

Layer	Examples
Cluster chaos	Kill server node during scheduling/upload, sever runner connection, restart DB connection, stale leader simulation.
End-to-end	Git push -> webhook -> pipeline -> artifact -> release -> OCI/package pull.
Upgrade	Rolling N-1 -> N server nodes; runner N-1 compatibility; migration expand/contract tests.

16.2 Critical correctness tests

- Two scheduler replicas must never successfully finalize the same job attempt.
- A stale runner attempt cannot overwrite a newer attempt result.
- A stale singleton leader cannot apply ownership-sensitive results after fencing token changes.
- Interrupted blob upload either resumes safely or leaves only reclaimable staging data.
- Artifact digest mismatch is rejected before metadata publication.
- Registry tag update is atomic from client perspective.
- Completed pipeline steps remain completed after server restart.
- Cluster rolling upgrade does not require sticky sessions.

17. Implementation phases and acceptance criteria

17.1 Phase 0 — Foundation

- Go workspace, frontend, code generation, migrations, configuration, logging, OTel bootstrap, test harness.
- PostgreSQL connection/migration library and blob-store interface with filesystem implementation.
- Organization/project/user primitives and local authentication.
- OpenAPI skeleton and request/audit middleware.

Exit gate — One server starts, migrates PostgreSQL, serves authenticated API/UI, writes/read blobs, exposes health/metrics.

17.2 Phase 1 — Standalone pipelines

- shipyard.yml parser/compiler and DAG state tables.
- Local runner registration and process executor.
- Scheduler, job lease, cancellation, log streaming.
- Git checkout action and manual/webhook pipeline trigger.
- Generic artifact upload/download linked to build provenance.

Exit gate — On one machine: push/manual trigger -> pipeline -> local runner -> artifact -> UI/API history, with restart recovery.

17.3 Phase 2 — Docker/BuildKit + OCI registry

- Docker executor.
- BuildKit integration and build cache controls [S6].

- OCI Distribution endpoints [S4] and OCI Image compatibility [S5].
- Registry auth tokens and repository permissions.
- CLI commands for login, run, artifact and registry operations.

Exit gate — docker/OCI client can authenticate, push and pull an image built by a Shipyard pipeline.

17.4 Phase 3 — Distributed runners

- Outbound remote runner connection protocol.
- Pools, labels/capabilities, capacity and drain.
- Reconnect/resume for live attempts.
- Windows/macOS runner binaries or process-executor support as platform maturity allows.

Exit gate — One control node dispatches concurrent jobs to at least three separate worker machines based on labels.

17.5 Phase 4 — Repository manager

- Local, remote and virtual repository engine [S3].
- Remote cache and retention/GC.
- Maven and npm adapters.
- Artifact search/properties and repository browsing.

Exit gate — A build can resolve dependencies through a Shipyard virtual repository, populate remote cache, and publish a package locally.

17.6 Phase 5 — Releases and environments

- Immutable release manifest containing artifact digests/provenance.
- Environments, deployment history, approvals and concurrency groups.
- Promotion workflow between repositories/environments.
- Rollback points based on prior immutable release.

17.7 Phase 6 — Cluster/HA

- Cluster node registration/health.
- Active-active API/registry nodes.
- PostgreSQL advisory-lock + distributed lease/fencing framework [S9].
- Runner reconnect through load balancer.
- S3 backend promoted to required shared-store option for HA.
- Rolling upgrade and node drain.

Exit gate — With three Shipyard server nodes, killing any one during API traffic and pipeline scheduling does not make the service unavailable and does not duplicate finalized jobs.

17.8 Phase 7 — Security and supply-chain hardening

- OIDC [S13], service accounts, fine-grained RBAC, protected environments/runners.

- Envelope-encrypted secrets and optional external KMS/Vault integration.
- SBOM/provenance/signature attachments.
- Policy gates and scanner integrations.
- Backup/restore validation and integrity scanner.

17.9 Phase 8 — Scale optimizations

- Performance profiling before service extraction.
- NATS JetStream optional event transport if DB-outbox dispatch becomes a proven bottleneck [S15].
- Artifact edge/cache nodes and replication.
- Additional package protocol adapters.
- Advanced autoscaled runner providers.

17.10 Definition of “v1 usable”

- ☐ Standalone Docker Compose installation documented and upgradeable.
- ☐ One control plane + multiple remote runners supported.
- ☐ OCI push/pull and BuildKit builds work.
- ☐ Generic artifacts + at least Maven and npm repositories.
- ☐ Pipeline DAG, approvals, retries, schedules, secrets and environments.
- ☐ Artifact provenance and releases.
- ☐ RBAC, audit, OIDC and API tokens.
- ☐ Backup/restore guide, metrics, health and logs.
- ☐ Cluster mode may ship as v1.x if not ready for initial 1.0, but the data model and runner protocol must already be cluster-safe.

18. OSS licensing and community model

Recommendation: Apache License 2.0 for the core repository, subject to legal review. The Apache Software Foundation identifies Apache-2.0 as an OSI-approved open-source license and publishes its terms for use, reproduction and distribution [S14]. The goal is to keep the server, runner, CLI, UI, protocol schemas and core registry/package functionality open rather than using an “open core” architecture where HA is structurally proprietary.

18.1 Suggested OSS boundary

- Open: server, runners, UI, CLI, SDKs, schemas, database migrations, Docker/Helm packaging, cluster coordination, OCI registry, package adapters.
- Possible commercial business: hosted Shipyard Cloud, managed support, enterprise SLAs, migration services, training, managed runner capacity.
- Avoid deliberately crippling self-hosted cluster/HA solely to create a paid feature wall if the project promise is truly OSS-first.

18.2 Community files

- LICENSE
- NOTICE if required by dependencies/distribution
- CONTRIBUTING.md
- SECURITY.md with private vulnerability reporting path
- CODE_OF_CONDUCT.md
- GOVERNANCE.md once maintainership expands
- Architecture docs and ADRs

- Public roadmap/issues with good-first-issue labels

19. Web UI/UX and design system

Shipyard MUST ship with a polished web interface that is treated as part of the product architecture, not as a thin administrative shell. The UI should make complex CI/CD, registry, artifact, runner, deployment, and cluster state understandable at a glance while still exposing the detail experienced operators need. The visual goal is a modern developer-platform experience: calm, fast, precise, keyboard-friendly, information-dense without being cluttered, and equally usable in light and dark themes.

19.1 Product experience principles

- Status must be obvious. Running, successful, failed, queued, degraded, offline, and blocked states need consistent icons, labels, and semantic colors; color alone MUST NOT be the only indicator.
- Progressive disclosure. Default screens should answer the common question quickly, while logs, metadata, digests, manifests, environment variables, and internal identifiers remain one click away.
- Fast navigation. Provide global search, command palette, breadcrumbs, recent projects, keyboard shortcuts, and stable deep links for builds, jobs, artifacts, packages, images, runners, releases, deployments, and cluster nodes.
- Live without being noisy. Pipeline logs, queue position, runner state, deployments, and cluster health should update in real time using SSE or WebSockets, while avoiding full-page refreshes and distracting animation.
- Developer-first copy. Prefer exact terminology such as digest, tag, job, runner, lease, repository, environment, and deployment. Error messages should explain what failed, where, and the next useful action.
- No enterprise-era visual clutter. Avoid oversized cards, excessive gradients, unnecessary modal dialogs, and dashboard widgets that do not help a user make a decision.

19.2 Frontend technology recommendation

Area	Recommendation	Reason
Application	React + TypeScript + Vite	Keeps the client modular and fast while matching the previously selected frontend stack.
Component foundation	shadcn/ui + Radix-based accessible primitives	Provides attractive open-code components that Shipyard can own and adapt into a dedicated design system rather than accepting a generic vendor theme. [S16]
Styling	Tailwind CSS + CSS design tokens	Use tokens for colors, spacing, radius, typography, borders, status states, elevations, and density so themes remain consistent.
Server state	TanStack Query or an equivalent query/cache layer	Centralize API caching, invalidation, retries, optimistic updates, and background refresh behavior.

Pipeline graph	React Flow / @xyflow/react	Useful for interactive DAG visualization with custom nodes, pan/zoom, selection, minimap and workflow-style layouts. [S17]
Logs / terminal	xterm.js	Purpose-built browser terminal frontend suitable for ANSI build output, terminal-style logs, resizing and streaming experiences. [S18]
Pipeline editor	Monaco Editor	Use for shipyard.yml editing, schema validation, diagnostics, completion, diff views and source-like editing. Monaco is the editor technology used by VS Code. [S19]
Charts	Lightweight chart library with accessible fallbacks	Use charts only for time-series or distribution data where a chart communicates more than a number/table.

19.3 Information architecture

The primary navigation should remain stable across installations. Features that are unavailable in Standalone mode should be hidden or clearly marked rather than reshuffling the entire product.

Navigation	Primary content
Home	Global activity, failing pipelines, queued work, recent releases, storage/registry summary, and actionable system warnings.
Projects	Repositories/projects, pipeline definitions, branch activity, environments, secrets, integrations and project settings.
Pipelines	Runs, DAG view, stages/jobs, live logs, retries, artifacts, timing, cache use and provenance.
Artifacts	Generic artifacts, package repositories, metadata, checksums/digests, versions, provenance, retention and downloads.
Registry	OCI repositories, tags, manifests, platforms, layers, image size, pull commands, signatures/SBOMs when enabled.
Releases	Versioned release bundles showing exactly which build outputs and commits were promoted.
Deployments	Environment history, current version, approval state, deployment status and rollback entry points.
Runners	Online/offline/busy runners, labels, capabilities, current job, utilization, executor type and recent failures.

Cluster	Available in distributed/cluster deployments: server nodes, leaders/leases, database/storage health, queue health and topology.
Administration	Organizations, users, RBAC, auth providers, audit logs, storage, retention, webhooks, API tokens and system settings.

19.4 Visual design system

- Default visual language: neutral surfaces, subtle borders, restrained shadows, compact spacing, high-quality typography, and one clear Shipyard brand accent rather than a rainbow of unrelated colors.
- Support both dark and light themes from the first public release. Theme tokens MUST be semantic (for example surface, panel, border, text-muted, status-success) rather than hard-coded per component.
- Use an 8px-oriented spacing system with a compact density option for large tables and operations screens.
- Prefer 8–12px corner radii for panels and controls. Avoid making every element a floating card; tables, split panes, side panels and grouped sections should carry most operational screens.
- Typography should distinguish page title, section title, body, metadata, code, numeric metrics, and log output. Digests, commit SHAs, paths, commands and identifiers should use a readable monospace font.
- Animations should communicate state transitions only: expanding details, live job progression, toast entry/exit and graph focus. Respect prefers-reduced-motion.

19.5 Core screen requirements

Screen	Required experience
Project overview	Immediate view of latest pipeline state, default branch, latest release, active deployments, repository usage, recent commits/webhooks and project health.
Pipeline run	Header with commit/branch/trigger/duration; interactive DAG; stage/job timeline; sticky status; retry/cancel controls; artifacts and provenance; live log panel.
Job detail	Split layout with searchable ANSI log viewer, step navigator, timings, runner identity, environment metadata, cache events and produced artifacts.
Artifact detail	Filename/package coordinates, size, digest/checksum, creation time, producing pipeline/job, commit, download/copy command, retention policy and deployment/release references.
OCI image detail	Repository/tag/digest, manifest/platform matrix, layers, size, created-by build, pull command, provenance/SBOM/signature status and related tags.
Runner detail	Availability, labels, architecture/OS, executor capabilities, current job, heartbeat, recent jobs, version and upgrade compatibility.
Cluster overview	Topology and health summary first; then node table, leader/lease ownership, scheduler health, queue

	depth, storage/database checks and version skew warnings.
Settings	Clear hierarchy with save-state feedback, destructive-action separation, validation before submission, and audit visibility for security-sensitive changes.

19.6 Pipeline visualization and logs

- The DAG is a visualization of the canonical pipeline plan, not a separate source of truth. Clicking a node opens the matching job detail without changing job state.
- Job nodes should show name, state, duration, runner label/executor and retry count. Collapsed matrix jobs may be grouped, with expansion on demand.
- For very large pipelines, support fit-to-view, search/focus, minimap, filtering by state/stage and a linear list/timeline fallback.
- Logs MUST stream incrementally and support ANSI colors, timestamps, step boundaries, search, copy, download, auto-follow, pause-follow, and jump-to-error. The UI should reconnect and resume from a server cursor/offset rather than requesting the whole log again.
- Secrets must remain server-side masked; the frontend must never receive a raw secret merely to decide whether to redact it.

19.7 Standalone, distributed, and cluster UI behavior

Mode	UI behavior
Standalone	Show the same product navigation but present the local server and local runner as a simple installation. Hide HA-only controls. System health should explain database/blob/backend status without cluster jargon.
Distributed	Add runner fleet views, machine/label filtering, queue placement information and connectivity health. The control plane remains represented as one logical Shipyard service.
Cluster / HA	Expose cluster topology, per-node readiness, active lease owners, version compatibility, replication/shared dependency status, node drain/maintenance state and failover history. A normal project user should not need to understand these screens to use CI/CD.

19.8 Responsive design and accessibility

- Desktop is the primary operational target, but pages MUST remain readable on tablets and phones for status checks, approvals, cancellation, artifact metadata and incident response.
- Complex graph editing and Monaco-based pipeline editing may intentionally provide a reduced mobile experience; do not pretend a tiny viewport is equivalent to a workstation.
- Interactive controls require visible keyboard focus, descriptive accessible names, logical tab order, adequate contrast, and non-color state indicators.

- Tables should support keyboard navigation where practical, column visibility, sorting/filtering and horizontal overflow without breaking the page.
- Every live-updating region should avoid stealing focus or repeatedly announcing noisy status changes to assistive technology.

19.9 Frontend performance and reliability requirements

- Route-level code splitting SHOULD keep administrative/cluster/editor code out of the initial project dashboard bundle when it is not needed.
- Virtualize very large log views and artifact/package tables. Do not keep an unbounded build log as thousands of individual DOM nodes.
- Use optimistic UI only for reversible/low-risk operations. Destructive and security-sensitive operations should wait for authoritative server confirmation.
- Every primary route needs explicit loading, empty, partial-error, permission-denied and disconnected states; blank screens and generic “something went wrong” pages are not acceptable.
- Use an application-level reconnect indicator when live event transport is degraded, while normal cached/read-only navigation remains functional.
- Frontend telemetry should capture route latency, failed API calls, live-stream reconnects and client exceptions without collecting pipeline secrets or log payloads by default.

19.10 UI quality gates

A feature is not considered complete merely because its API exists. User-facing features MUST meet the following release gates:

- Designed empty, loading, success, warning, failure, permission-denied and disconnected states.
- Keyboard and screen-reader smoke tests for the critical interaction path.
- Light and dark theme review with no hard-coded unreadable status colors.
- Responsive review at desktop and narrow widths.
- No layout shift or full-page refresh required for live pipeline/runner state.
- Error messages include a useful next action and preserve relevant technical identifiers for debugging.
- Visual regression coverage for core components and high-value screens.
- End-to-end tests for pipeline start/cancel/retry, artifact navigation, runner status, authentication and cluster-health workflows.

Appendix A — Example shipyard.yml

```
pipeline:
  name: Build & Release

on:
  push:
    branches: [main]
  pull_request: {}

concurrency:
  group: deploy-${{ git.branch }}
  cancel_in_progress: false

jobs:
  test:
    runs_on: [linux, amd64]
    container:
      image: eclipse-temurin:21
    steps:
      - uses: shipyard/checkout@v1
      - name: Test
        run: ./gradlew test
```

```

- name: Upload tests
  uses: shipyard/artifacts@v1
  with:
    path: build/test-results/**
    name: test-results

build:
  needs: [test]
  runs_on: [linux, docker]
  steps:
    - uses: shipyard/checkout@v1
    - name: Build JAR
      run: ./gradlew clean build
    - uses: shipyard/artifacts@v1
      with:
        path: build/libs/*.jar
        repository: releases

image:
  needs: [test]
  runs_on: [linux, buildkit]
  steps:
    - uses: shipyard/checkout@v1
    - uses: shipyard/build-image@v1
      with:
        context: .
        push: true
        tags:
          - registry.example.com/acme/app:${{ git.sha }}

deploy:
  needs: [build, image]
  environment: production
  approval:
    required: true
  steps:
    - run: ./deploy.sh ${{ git.sha }}

```

Appendix B — Example standalone configuration shape

```

# shipyard.yaml
server:
  external_url: https://shipyard.example.com
  listen: 0.0.0.0:8080

database:
  dsn_file: /run/secrets/postgres_dsn

storage:
  backend: filesystem
  filesystem:
    root: /var/lib/shipyard/blobs

cluster:
  mode: standalone
  cluster_id: shipyard-local

runners:
  registration_enabled: true

registry:
  enabled: true

telemetry:
  metrics: true
  otel_endpoint: ""

auth:
  local: true
  oidc: []

```

Appendix B.1 — Cluster configuration shape

```
storage:
  backend: s3
  s3:
    endpoint: https://s3.example.com
    bucket: shipyard-artifacts
    region: us-east-1

cluster:
  mode: ha
  cluster_id: production-01
  node_id_file: /var/lib/shipyard/node-id

# Every node uses the same external PostgreSQL DSN and object store.
# node_id remains unique per replica.
```

Appendix C — Source references

Sources below were reviewed on 9 August 2026. They are primarily standards bodies or official project/vendor documentation. References establish external protocol/technology behavior and comparable public concepts; Shipyard-specific implementation decisions in this document are recommendations, not claims made by those sources.

[S1] Jenkins — Pipeline as Code. Jenkins documentation. Pipeline definitions stored/versioned in source; complex stages, approvals, resumable flows and artifact fingerprinting concepts. <https://www.jenkins.io/doc/book/pipeline/pipeline-as-code/>

[S2] Jenkins — Using Jenkins agents. Jenkins documentation. Distributed controller/agent architecture, labels, executors and workload placement concepts. <https://www.jenkins.io/doc/book/using/using-agents/>

[S3] JFrog Artifactory — Repository Management. Official documentation of local, remote, virtual and federated repository concepts. <https://docs.jfrog.com/artifactory/docs/repository-management>

[S4] Open Container Initiative — Distribution Specification. Standard API for content distribution, including registry/repository/blob concepts; based on Docker Registry HTTP API V2. <https://github.com/opencontainers/distribution-spec/blob/main/spec.md>

[S5] Open Container Initiative — Image Format Specification. OCI image manifest/index/layer/configuration model for interoperable container images. <https://github.com/opencontainers/image-spec/blob/main/spec.md>

[S6] Docker — BuildKit documentation. BuildKit build graph/cache behavior, registry-exportable cache and Dockerfile frontend behavior. <https://docs.docker.com/build/buildkit/>

[S7] Docker — Compose documentation. Compose defines and runs multi-container applications from a YAML configuration. <https://docs.docker.com/compose/>

[S8] Docker — Volumes documentation. Official guidance on persistent Docker volumes and their lifecycle/use cases. <https://docs.docker.com/engine/storage/volumes/>

[S9] PostgreSQL — Explicit Locking / Advisory Locks. Application-defined advisory locks, session/transaction behavior and automatic cleanup. <https://www.postgresql.org/docs/current/explicit-locking.html>

[S10] PostgreSQL — Row Security Policies. Database row-level policies that control visible/modifiable rows. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>

[S11] Kubernetes — Jobs. Kubernetes Jobs as one-off tasks that run to completion with retry/completion semantics. <https://kubernetes.io/docs/concepts/workloads/controllers/job/>

[S12] OpenTelemetry — Documentation. Vendor-neutral telemetry framework for traces, metrics and logs. <https://opentelemetry.io/docs/>

[S13] OpenID Connect Core 1.0. Interoperable identity layer on OAuth 2.0. https://openid.net/specs/openid-connect-core-1_0.html

[S14] **Apache License, Version 2.0.** Apache Software Foundation license text and identification as Apache-2.0/OSI-approved. <https://www.apache.org/licenses/LICENSE-2.0>

[S15] **NATS — JetStream documentation.** Durable streams/consumers and related replicated persistence abstractions; optional future Shipyard event transport. <https://docs.nats.io/concepts/jetstream>

[S19] **Monaco Editor — Official documentation.** Browser code editor that powers VS Code, with editor, diff, completion/diagnostic and language APIs useful for shipyard.yml editing. <https://microsoft.github.io/monaco-editor/>

[S18] **xterm.js — Documentation.** Browser terminal front-end component and API suitable for terminal-style streamed build/log experiences. <https://xtermjs.org/docs/>

[S17] **React Flow — Official documentation.** Open-source React library for node-based UIs with custom nodes, selection, panning, zooming and related diagram controls. <https://reactflow.dev/>

[S16] **shadcn/ui — Introduction.** Official documentation describing its open-code, composable component approach and accessible, customizable design-system foundation. <https://ui.shadcn.com/docs>

Appendix D — First implementation checklist

- ☐ Create repository and Apache-2.0 license proposal/decision ADR.
- ☐ Bootstrap Go server, React UI and PostgreSQL migrations.
- ☐ Define Shipyard design tokens, light/dark themes, typography, status semantics and reusable application shell.
- ☐ Build UI primitives and high-value screens before feature proliferation: project overview, pipeline run, job/log viewer, artifact detail and runner list.
- ☐ Add React Flow pipeline DAG, xterm.js log viewer and Monaco shipyard.yml editor behind stable internal abstractions.
- ☐ Add accessibility, visual-regression and responsive UI checks to CI so visual quality is enforced continuously.
- ☐ Define identifiers, tenancy schema, audit event envelope and configuration schema.
- ☐ Implement BlobStore interface with filesystem backend and digest-verifying streaming API.
- ☐ Implement local auth, project RBAC and API tokens.
- ☐ Define shipyard.yml JSON Schema and compile YAML to canonical execution plan.
- ☐ Create runner protocol v1 and local runner process executor.
- ☐ Implement scheduler leasing/fencing from the beginning — even Standalone uses it.
- ☐ Add log streaming/resume and job cancellation.
- ☐ Add artifact upload/download/provenance.
- ☐ Integrate BuildKit and then implement OCI Distribution API.
- ☐ Add remote runners and label-based placement.
- ☐ Add S3 blob backend and storage migration command.
- ☐ Add local/remote/virtual repository engine, then Maven/npm adapters.
- ☐ Add releases, environments, approvals and deployments.
- ☐ Add active-active server replicas, cluster leases and failover chaos tests.
- ☐ Add OIDC, hardened secrets, supply-chain metadata and production backup/restore tooling.

Shipyard UI/UX Design System & Implementation Contract

Status: Normative

Research snapshot: 2026-08-09

Applies to: Shipyard Web UI, reusable @shipyard/ui components, Storybook, generated screenshots, and AI-assisted frontend work.

This file is not inspiration. It is the implementation contract. Human contributors and AI coding agents **MUST** follow it unless a maintainer explicitly approves an exception.

0. Non-negotiable rules for humans and AI agents

1. **Do not make Shipyard look like a generated SaaS dashboard.** Do not default to giant rounded cards, purple/blue gradients, glass panels, excessive pills, large empty whitespace, glowing CTAs, sparkle icons, or a generic shaden demo appearance.
2. **Shipyard is an engineering workspace.** Optimize for recognition of state, diagnosis, comparison, and safe action.
3. **Use Shipyard-owned components.** Product code imports reusable UI from @shipyard/ui. Do not import a pre-styled component library directly into feature screens.
4. **Accessible behavior comes from React Aria Components.** Styling and visual identity remain Shipyard-owned. [R14]
5. **Use semantic design tokens.** Do not hard-code colors, radii, shadows, or spacing in feature components unless the design-system package does not yet provide a required token.
6. **Tables are first-class.** Do not replace useful tabular data with a grid of cards. Use TanStack Table for complex data tables. [R15]
7. **Graphs are optional views, not the only truth.** Pipeline DAG and cluster topology screens **MUST** also provide list/table views. Argo CD's multiple detail views are a useful precedent. [R12]
8. **Dark, light and system theme modes are required.** Do not implement dark mode through one-off selectors; semantic tokens drive themes. [R2][R9]
9. **Keyboard support is required.** Every primary workflow must be operable without a mouse.
10. **Shipyard uses IBM Plex Sans and IBM Plex Mono.** Do not substitute Inter unless the font cannot load. IBM Plex is open source and designed for UI use. [R17]
11. **Product icons use Tabler Icons.** Brand/vendor marks use Simple Icons or official vendor SVGs. Do not mix Lucide, Heroicons, Font Awesome and Tabler on the same surface. [R16]

12. **Ordinary CI logs are not a fake terminal.** Use a purpose-built virtualized log viewer. Use xterm.js only for a genuinely interactive exec/shell session.
 13. **One obvious primary action per action group.** Secondary actions should be visually quiet.
 14. **Status is never communicated by color alone.** Pair semantic color with icon and text.
 15. **No feature is UI-complete without loading, empty, error, permission-denied, disconnected and partial-data states.**
-

1. Design objective

Shipyards combines CI/CD, artifact/package hosting, OCI registry functions, runners and cluster operations. Its UI should feel closer to a high-quality developer workstation than a marketing website.

The visual language is **industrial precision with subtle harbor identity**:

- graphite and neutral surfaces;
- Shipyards blue for selection and primary actions;
- restrained dock orange for rare release/attention accents;
- compact geometry;
- explicit borders and separators;
- dense information with strong hierarchy;
- stable navigation and predictable action locations;
- technical typography and tabular numerals;
- no decorative nautical gimmicks.

The product should still be friendly to new operators: density is not an excuse for clutter. The research shows that mature FOSS products do not converge on one fashionable framework; instead, projects such as GitLab and Grafana maintain their own design systems, Nextcloud standardizes layout patterns, and PatternFly treats dense enterprise data workflows as a design problem. [R1][R4][R7][R8]

2. Research summary and what Shipyards should borrow

GitLab / Pajamas

Borrow: - design tokens as the shared contract between design and code; - versioned component/pattern libraries; - dark-mode review as a normal part of design QA; - clear separation between foundations, components, patterns, icons and visualization. [R1][R2]

Do not copy: - GitLab-specific brand styling; - every page's complexity or navigation density.

Grafana / Saga

Borrow: - treat the design system as a maintained software product; - document components with limitations and intended use; - bake accessibility into components rather than fixing it after features ship; - expose technical data without trying to turn every screen into a marketing layout. [R4][R5]

Do not copy: - the recognizable Grafana dashboard visual skin.

Nextcloud

Borrow: - predictable left navigation; - content + sidebar/detail-inspector layouts; - list + content patterns; - theme-aware colors and restrained use of the primary brand color. [R6][R7]

Do not copy: - consumer-file-manager spacing where Shipyard needs a denser operator view.

PatternFly

Borrow: - 14px default technical body scale; - tabular numerals for changing numeric values; - strong table patterns; - semantic light/dark tokens. [R8][R9]

Do not copy: - Red Hat's visual identity or the full PatternFly component suite.

Harbor

Harbor's current portal identifies itself as an Angular + Clarity UI. The important lesson is not the framework; it is that artifact/registry administration benefits from straightforward project/repository/tag hierarchy. [R10]

Gitea

Gitea's maintainers have discussed moving away from a heavy styled dependency toward flexible low-level/headless UI pieces. That supports Shipyard's decision to own its visual layer while using lower-level accessibility primitives. [R11]

Argo CD

Argo CD allows operators to choose among tree, pod, network and list application-detail views. Shipyard should follow the same principle: one visualization should not be forced on every operator or every scale. [R12]

Portainer

Portainer's current repository shows an incremental move toward modern React tooling while still carrying older framework code. Shipyard should avoid that migration burden by defining a stable UI package and feature boundaries early. [R13]

Community preference signals

Public developer discussions are **qualitative, not a survey**. They do consistently surface three useful signals:

- power users often value higher information density, provided hierarchy and readability remain clear; [Q1][Q2]
- keyboard-driven access reduces interaction overhead for technical workflows;
- mobile tables work better when the UI prioritizes the user’s task, preserves identity columns, and uses expansion/column selection instead of blindly squeezing desktop tables. [Q3]

Shipyards therefore defaults to **medium-high desktop density**, with Compact and Comfortable table density options.

3. Frontend architecture

Required stack

Concern	Choice	Rule
Framework	React + TypeScript + Vite	Keep feature code framework-simple.
Accessible behavior	react-aria-components	Wrap inside @shipyards/ui ; never accept its example styling as Shipyards styling. [R14]
Styling	CSS Modules + CSS custom properties	No CSS-in-JS requirement and no stock component-library theme.
Server state	@tanstack/react-query	Async/server state only.
Data tables	@tanstack/react-table	Headless logic; Shipyards owns HTML, density and styling. [R15]
Virtualization	@tanstack/react-virtual	Large tables, logs, artifact lists, runner fleets.
DAG/topology	@xyflow/react	Custom Shipyards nodes; list/table alternative required.
YAML/code	monaco-editor	shipyards.yml , diffs, policy/config editors.

Concern	Choice	Rule
Log viewer	custom + virtualization + ANSI parser	Search, timestamps, wrap, pause/follow, copy selection.
Interactive shell	@xterm/xterm	Only real shell/exec sessions.
Operational charts	uPlot	Small fast time-series; no generic chart-card factory.
Icons	@tabler/icons-react	Only general product icon family. [R16]
Brand icons	Simple Icons / official SVG	Vendors only.
Component docs	Storybook	Every reusable state documented.
E2E	Playwright	Critical workflows + keyboard paths.
Accessibility testing	axe-core + manual keyboard/SR checks	Automated checks are a floor, not the finish line.

Explicitly not canonical

- shadcn/ui as the visual system;
- Material UI, Ant Design, Chakra, Mantine or another pre-styled suite as the visual owner;
- Bootstrap-style page composition;
- Tailwind component recipes copied directly from templates.

A contributor MAY use a utility framework only inside @shipyard/ui if maintainers adopt it later. Feature screens must still consume semantic Shipyard components/tokens.

Package boundaries

```
apps/
  web/
    src/
      app/
      features/
      routes/
      shell/
packages/
  ui/
    src/
      components/
```

```
patterns/  
tokens/  
icons/  
typography/  
styles/  
pipeline-graph/  
log-viewer/  
config-editor/  
api-client/  
domain/
```

Feature code must not duplicate a component already owned by `packages/ui`.

4. Design-token system

Token source of truth

Use a generated token pipeline:

```
packages/ui/src/tokens/  
  source.json      # canonical values + semantic mapping  
  generated.css    # CSS custom properties  
  generated.ts     # typed TS token names/values  
  README.md       # token intent
```

A small repository script may generate CSS/TS from `source.json`. Do not maintain duplicate hand-edited values.

Naming

Use semantic names:

```
--sy-color-bg-canvas  
--sy-color-bg-surface  
--sy-color-text-primary  
--sy-color-text-muted  
--sy-color-border-default  
--sy-color-action-primary  
--sy-color-status-success  
--sy-space-4  
--sy-radius-panel  
--sy-control-height-default
```

Do not write feature CSS using palette-only names such as `--blue-500` when the meaning is actually `action-primary`.

5. Color system

Dark theme

Token	Value	Usage
bg.canvas	#0B0D10	app background
bg.surface	#111419	navigation, main panes
bg.subtle	#15191F	nested/alternate surface
bg.raised	#191E25	popovers/dialogs only
border.default	#2A3038	separators
border.strong	#3A424D	strong boundary
text.primary	#E7EAF0	primary text
text.muted	#9AA4B2	secondary metadata
text.faint	#6F7A87	low-priority metadata
action.primary	#6E97FF	selection/primary action
action.primaryHover	#89AAFF	hover
accent.dock	#E2A45B	rare Shipyard accent
status.success	#45B97C	success/healthy
status.warning	#D8A13B	warning/degraded
status.danger	#E26666	fail/destructive
status.info	#69A7E8	info
focus.ring	#9AB8FF	keyboard focus

Light theme

Token	Value	Usage
bg.canvas	#F5F7F9	app background
bg.surface	FFFFFF	main pane/navigation
bg.subtle	#F8FAFC	nested/alternate surface
border.default	#D8DEE6	separators
border.strong	#B7C0CC	strong boundary
text.primary	#1F2328	primary text
text.muted	#59636E	secondary text
text.faint	#7A8591	low priority
action.primary	#315EFB	primary action
action.primaryHover	#244CD8	hover
accent.dock	#B86519	rare accent, not default body text
status.success	#177A49	success/healthy
status.warning	#A05E00	warning/degraded
status.danger	#B92C2C	fail/destructive
status.info	#256FAF	info
focus.ring	#1F5EFF	focus

Color rules

- No gradients in normal product chrome.
 - Never use status red/green as a decorative accent.
 - Primary blue means **selection or primary action**, not “anything important.”
 - Dock orange is limited to rare release/attention moments and brand details.
 - In tables, prefer a subtle tinted row/background + icon + label instead of fully saturated status fills.
 - Never use colored body text when the contrast is insufficient; use primary text plus a colored icon/border instead.
-

6. Typography

Fonts

- **UI:** IBM Plex Sans
- **Code/technical:** IBM Plex Mono
- **Fallback:** system sans / system monospace

IBM Plex provides Sans and Mono families under an open font license and is designed to work well in user interfaces. [R17]

Scale

Token	Size / line	Weight	Use
Page title	24 / 32px	600	one per page
Section	18 / 26px	600	major section
Panel title	15 / 22px	600	panel/table heading
Body	14 / 20px	400	default
Body strong	14 / 20px	500	names/important values
Small	13 / 18px	400	dense tables/metadata
Micro	12 / 16px	500	badges/timestamps
Code	13 / 20px	400	logs/YAML/commands
Code small	12 / 18px	400	digests/coordinates

PatternFly’s use of 14px default body text and tabular numerals is a useful reference for dense technical interfaces. [R8]

Rules

- Sentence case.
- Avoid all caps except short protocol/architecture labels (OCI, AMD64, ARM64).

- Use `font-variant-numeric: tabular-nums` for build numbers, duration, storage, percentages, queue counts and changing metrics.
- Do not use monospace as decoration; use it for machine-readable content.
- Page headings are left aligned.
- Avoid huge 32-48px headings inside the authenticated product UI.

7. Geometry, spacing and density

4px base grid

0, 4, 8, 12, 16, 20, 24, 32, 40, 48, 64

Radius

- controls: 6px
- panels/popovers: 8px
- large dialogs: max 10-12px
- pills: only when the semantic shape is a pill (tag/status/chip)

Control heights

- compact desktop: 28px
- default desktop: 32px
- prominent form action: 36px
- touch-oriented effective target: aim near 44px where practical while meeting WCAG minimums. [R18]

Table row density

- Compact: 30px
- Comfortable: 36px (default)

Users may choose Compact or Comfortable globally. Do not invent a third density per page.

Elevation

Borders before shadows. Shadows are reserved for overlays, floating menus and dialogs. Normal cards/panels use a 1px border and spacing.

8. Iconography

Canonical product icons

Use direct imports from `@tabler/icons-react`.

Meaning	Icon
Dashboard	IconLayoutDashboard
Projects	IconFolderCode
Pipelines	IconGitBranch
Job/run	IconPlayerPlay
Artifacts	IconPackage
Registry	IconContainer
Releases	IconRocket
Deployments	IconCloudUpload
Runners	IconServer2
Cluster	IconTopologyStar3
Logs	IconTerminal2
Settings	IconSettings
Secrets	IconKey
Security	IconShieldCheck
Success	IconCircleCheck
Failure	IconCircleX
Running	IconLoader2
Queued	IconClock
Warning	IconAlertTriangle
Retry	IconRefresh
Stop	IconSquare
Copy	IconCopy
Download	IconDownload
Upload	IconUpload
Search	IconSearch
Command palette	IconCommand

Tabler provides React SVG icons under an MIT license. [R16]

Rules

- navigation: 18px
- normal control: 16px
- stroke: library default unless design-system wrapper changes it globally
- icon-only buttons MUST have an accessible label and tooltip
- do not use emoji as product icons
- do not use sparkle/wand/brain icons for generic “smart” functions
- vendor brands must not be recolored in ways that violate their brand rules

9. Application shell

Desktop shell

	Topbar: project/context global search status account
Sidebar	Page header: title context primary action
Overview	Toolbar / filters / tabs
Projects	
Pipelines	Main work area
Artifacts	
Registry	[optional inspector]
Releases	
Deploy	
Runners	
Cluster	

Dimensions: - topbar: 48px - sidebar expanded: 224px - sidebar collapsed: 56px
- desktop page gutter: 20-24px

Navigation order

1. Overview
2. Projects
3. Pipelines
4. Artifacts
5. Registry
6. Releases
7. Deployments
8. Runners
9. Cluster
10. Settings

Do not reorder navigation based on usage analytics or permissions. Hide unavailable sections when appropriate, but preserve relative order.

Command palette

Cmd/Ctrl+K opens global navigation/action search.

Initial commands: - open project - open pipeline/run - open artifact by digest/name - open runner - open environment - trigger pipeline (when permitted) - create release (when permitted) - theme switch

Dangerous actions do not execute immediately from command search; they open the relevant confirmation flow.

10. Component contract

Button

- variants: primary, secondary, ghost, destructive, link, icon-only
- sizes: 28 / 32 / 36px
- radius: 6px
- icons: 16px
- one high-emphasis primary action per local action group
- destructive buttons are not visually primary until the user enters a destructive confirmation context

Input

- default height: 32px
- persistent visible label
- helper/error below field
- code/path/token inputs may use IBM Plex Mono
- do not use floating labels

Select / ComboBox

Use React Aria behavior. [R14]

- typeahead required
- searchable when options > ~8 or when options are remote
- clear selected/value state
- multi-select only where the domain truly supports multiple choices

Badge / chip

Allowed for: - status - environment - role - tag - runner label

Not allowed for every metadata value. Plain text is often better.

Dialog

- typical max width: 560px
- use for create/edit/confirm
- destructive dialog names the target and states the impact
- typing the object name is reserved for high-impact irreversible operations, not every delete

Inspector drawer

Use a right-side inspector for selected pipeline nodes, artifacts, runners, deployments and cluster members when keeping list/DAG context is useful.

Toast

- transient confirmation and low-risk information
 - a critical failure must also remain visible in the relevant page/state
 - never make a toast the only record of a failed operation
-

11. Tables and lists

Tables are a core Shipyard interaction surface.

Use a table when

- multiple objects share the same properties;
- users compare state across rows;
- sorting/filtering/bulk action matters;
- identity and numeric/technical fields need scanability.

Standard table anatomy

[Search]	[Status v]	[Owner v]	[Columns]	[Density]	[Action]
Name	Status	Duration	Commit	Started	...
api-build #1842	Failed	04:31	a82c10f	2m ago	...
worker #1841	Success	03:12	118faca	9m ago	...

Required capabilities for complex tables

- semantic `<table>` markup whenever practical;
- stable identity/name column;
- sortable headers;
- filters displayed above the table, not hidden in a mystery icon;
- column chooser;
- density selector;
- sticky header for long lists;
- optional sticky identity column;
- row selection only if a bulk action exists;
- right-align numeric values;
- tabular numerals;
- virtualization for very large row sets;
- preserve query/filter state in the URL when useful.

TanStack Table is headless and allows Shipyard to keep full control over markup and styling. [R15]

Mobile table strategy

Do not blindly transform every row into a huge card.

This is consistent with qualitative feedback from developers discussing mobile data tables. [Q3]

Pipeline run desktop layout

Pipeline graph rules

- Argo CD's support for multiple operational views is an important precedent.
[R12]

Every job row/node shows: - job name - state - elapsed/final duration - runner or executor - retry count when >0 - optional artifact indicator

Failure details prioritize: 1. failed step; 2. concise failure reason; 3. relevant log location; 4. retry action if safe; 5. runner/executor context.

13. Log viewer

Ordinary CI logs

Build a dedicated log viewer, not a terminal emulator skin.

Features: - streaming append without destroying text selection; - Follow toggle; - Pause stream; - search; - regex search optional later; - timestamp toggle; - line numbers optional; - wrap toggle; - ANSI colors with Shipyard-safe palette mapping; - copy selected; - download full log; - jump to first error/warning when structured markers exist; - virtualized rendering; - preserve scroll position when the user scrolls away from the bottom.

Interactive shell

Only when a user explicitly starts a live shell/exec session should Shipyard use xterm.js. Separate this visually and semantically from immutable CI logs.

14. Artifact and package UX

Artifact browser

Default view is a structured list/table, not a tile gallery.

Columns: - name/path - type - size - checksum/digest - producing pipeline - version/tag - created - retention state

Selecting an artifact opens an inspector with: - canonical identifier; - SHA-256; - MIME/package type; - source commit; - build/run; - runner/executor; - SBOM/provenance links when available; - download/copy command; - retention and delete policy.

The primary interaction model is: **identify -> verify -> copy/use -> trace provenance**.

Package page

Show package-manager-native instructions as copyable commands.

Example:

Maven

`com.example:shipyard-agent:1.4.2`

Repository
`https://shipyard.example.com/maven/releases`

[Copy Gradle] [Copy Maven]

Do not hide install coordinates behind a secondary modal.

15. OCI registry UX

Repository page: - repository name - pull command - tag count - storage usage
- latest push - policy/security summary

Tag/manifest table: - tag - digest - media type - platforms - size - pushed -
signer/provenance status

Manifest inspector: - digest first; - copyable pull-by-digest command; - architec-
ture/OS variants; - layer list; - config metadata; - referers/attestations/SBOM;
- source pipeline and commit.

Avoid Docker-like decorative whale/container imagery in normal operations. Do-
main identity should come from terminology and information structure.

16. Releases and deployments

Release page

A release connects:

`source commit -> pipeline -> artifacts/images -> release -> environments`

Page sections: - release identity/version - source commit/tag - artifacts/images
included - provenance/security summary - changelog/notes - deployment time-
line

Deployment page

Always display the environment context near the action area:

PRODUCTION
us-east-1 / payments
currently: 1.8.2
candidate: 1.9.0

Production actions must not be visually indistinguishable from dev/staging ac-
tions.

Rollback: - specify target release/digest; - show what changes; - require confir-
mation; - show resulting deployment event in the timeline.

17. Runners and cluster operations

Runner fleet

Use a table with: - name - online/drainning/offline - executor - labels - version - architecture - active/capacity - last heartbeat - current job

Bulk actions: - drain - enable/disable scheduling - assign/remove labels

Cluster page

Health comes before topology.

Top section: - control-plane quorum/availability - scheduler leader/lease state - PostgreSQL health - object storage health - queue/message layer health when enabled - connected runners

Then: - member table - dependency health - topology view

Topology is secondary; operators must be able to understand health from text/table state alone.

18. Responsive design

Desktop first, not desktop only

Shipyards is a technical operator product. Full creation/configuration workflows target desktop/tablet widths. Mobile must still support monitoring, approvals, incident checks and common safe actions.

Breakpoints are content-driven, not device-brand-driven.

At narrow widths: - sidebar becomes a drawer; - inspector becomes a full-width detail route/sheet; - table preserves identity + status + one key metric; - lower-priority metadata moves into row expansion; - graphs default to list view when the graph cannot remain readable; - logs keep search/follow/wrap but collapse secondary controls.

Do not create a completely different mobile information architecture.

19. Motion

Motion is functional only.

Allowed: - 100-160ms hover/focus/selection transition; - 160-220ms menu/dialog/drawer entrance; - small progress indication; - graph edge activity while currently running.

Not allowed: - ambient gradients; - glowing borders; - infinite decorative pulses; - springy page transitions; - animated counters that delay reading; - parallax.

Respect **prefers-reduced-motion** and remove nonessential movement.

20. Accessibility

Target: **WCAG 2.2 AA**.

Required: - semantic headings/landmarks; - every form control labeled; - visible focus ring; - no keyboard traps; - meaningful focus return after dialogs; - status icon + text + color; - sufficient contrast in both themes; - reduced-motion support; - graph alternative; - accessible table headers/captions where appropriate; - no critical information only in hover/tooltip; - live regions used sparingly.

React Aria provides style-free accessible interaction primitives, focus management and internationalization support, making it appropriate for Shipyard's owned design system. [R14] Grafana's accessibility guidance similarly warns against indiscriminate live-region use. [R5]

Definition of done for a component

A reusable component is not done until Storybook includes: - default - hover/focus/pressed - disabled - loading if relevant - invalid/error if relevant - dark theme - light theme - long text - keyboard interaction - accessible name/label behavior

21. “Do not make it AI-looking” guardrails

This section is deliberately explicit because generic generated frontend code tends to converge on a recognizable set of visual shortcuts.

Never default to

- purple -> blue -> cyan gradients;
- glassmorphism;
- translucent floating nav;
- 16-24px radii on every component;
- “bento” dashboard made entirely from cards;
- a giant **Welcome back** hero in an operator console;
- four giant KPI cards as the entire dashboard;

- a pill around every status, label, branch, timestamp and value;
- oversized icons in colored circles;
- sparkles/brain/wand icons for automation;
- fake terminal chrome for plain logs;
- glowing primary buttons;
- centered marketing copy inside authenticated pages;
- excessive shadows;
- placeholder charts that exist only to fill space;
- random gradients on avatar/project initials;
- stock shaden demo spacing and component composition.

Use instead

- tables and lists for comparable objects;
- split panes for inspect-without-losing-context workflows;
- subtle section backgrounds;
- borders/separators;
- one strong primary action;
- compact metadata;
- text labels and domain icons;
- provenance paths;
- explicit operational state;
- meaningful empty states with a single next step.

Review question

Before merging a new screen, ask:

If the Shipyard logo and product name disappeared, would this screen still look obviously designed for CI/CD, artifacts, registries, runners or cluster operations — or would it look like a generic AI-generated admin dashboard?

If the answer is “generic dashboard,” revise it.

22. Dashboard blueprint

The dashboard should answer: 1. Is Shipyard healthy? 2. What recently failed or changed? 3. Is capacity constrained? 4. What action needs attention?

Recommended composition:

```
Shipyard
System healthy | 24 runners | 3 jobs queued | 68% artifact storage
-----
Needs attention
! Pipeline #1842 failed   production/api           4m ago    [Open]
```

```

! Runner linux-07 offline cluster-east          9m ago    [Open]
-----
Recent pipelines                                Recent releases
#1842 failed api      a82c10f 4m ago      1.9.0 production 31m ago
#1841 success web     118faca 9m ago      1.8.9 staging    2h ago
-----
Capacity / storage mini time series (only if it drives action)
Avoid making each line above a separate oversized rounded card.

```

23. AI coding-agent procedure

When an AI agent is asked to build or modify Shipyard UI, it **MUST** execute this sequence:

Step 1 - classify the screen

Choose one primary pattern: - table/list; - list + inspector; - graph + inspector; - settings form; - editor; - operational dashboard; - timeline/detail; - terminal/log viewer.

Do not invent a novel page pattern unless none fits.

Step 2 - identify the operator question

Write one sentence internally:

The user opens this screen to answer: ____.

Every major UI element must help answer that question or trigger the next safe action.

Step 3 - reuse components

Search @shipyard/ui and existing patterns before creating new UI.

Do not: - clone a button/input/table style inside a feature folder; - import a new visual component library to solve one screen; - add a second icon family.

Step 4 - implement all states

At minimum: - loading; - loaded; - empty; - error; - permission denied when applicable; - stale/disconnected when applicable.

Step 5 - implement accessibility

Verify: - keyboard path; - focus order; - visible focus; - labels; - screen-reader names; - status not color-only; - reduced motion.

Step 6 - test density and long content

Use fixture data with: - 100+ rows; - very long project names; - long artifact paths; - full SHA-256 digests; - several status types; - zero results; - partial data.

Step 7 - run the anti-generic review

Reject the implementation if it introduced: - gradients; - excessive rounded cards; - random pill metadata; - arbitrary shadows; - generic hero/marketing sections; - stock component-demo layout.

Step 8 - Storybook and tests

Add/modify: - Storybook story for reusable UI; - Playwright path for critical workflow; - accessibility check; - visual regression snapshot where maintained.

24. Example implementation conventions

Semantic token CSS

```
:root {
  --sy-color-bg-canvas: #f5f7f9;
  --sy-color-bg-surface: #ffffff;
  --sy-color-text-primary: #1f2328;
  --sy-color-text-muted: #59636e;
  --sy-color-border-default: #d8dee6;
  --sy-color-action-primary: #315efb;
  --sy-radius-control: 6px;
  --sy-radius-panel: 8px;
  --sy-control-height-default: 32px;
}

[data-theme="dark"] {
  --sy-color-bg-canvas: #0b0d10;
  --sy-color-bg-surface: #111419;
  --sy-color-text-primary: #e7eaf0;
  --sy-color-text-muted: #9aa4b2;
  --sy-color-border-default: #2a3038;
  --sy-color-action-primary: #6e97ff;
}
```

Component imports

```
import { Button, DataTable, StatusBadge } from '@shipyard/ui';
import { IconRefresh } from '@tabler/icons-react';
```

Preferred:

```
<Button variant="secondary" icon={<IconRefresh />}>  
  Retry failed  
</Button>
```

Do not feature-local-style a random native button to look “close enough.”

Status representation

```
<StatusBadge status="failed">Failed</StatusBadge>
```

The component is responsible for icon, semantic color and accessible text.

25. Design review checklist

Product fit

- ☐ The main operator question is obvious.
- ☐ Primary action is obvious but not oversized.
- ☐ Technical metadata is copyable where useful.
- ☐ Provenance is visible for artifacts/releases/deployments.
- ☐ Dense data is a table/list, not cards.
- ☐ Graph has a list/table alternative.

Visual system

- ☐ IBM Plex Sans / Mono only.
- ☐ Tabler product icons only.
- ☐ Semantic tokens only.
- ☐ 4px spacing system.
- ☐ 6px controls / 8px panels.
- ☐ No product gradients.
- ☐ Shadows only where elevation is real.

States

- ☐ loading
- ☐ empty
- ☐ error
- ☐ permission denied if applicable
- ☐ disconnected/stale if applicable
- ☐ long-content behavior

Accessibility

- ☐ keyboard complete
- ☐ visible focus
- ☐ labels/names

- ☐ no color-only state
- ☐ theme contrast
- ☐ reduced motion
- ☐ responsive/mobile monitoring path

Anti-generic

- ☐ no bento-card dashboard by default
- ☐ no giant hero copy
- ☐ no random pills
- ☐ no sparkle/AI visual cliché
- ☐ no stock shaden demo composition
- ☐ screen still looks domain-specific without Shipyard logo

26. Governance

DESIGN.md is normative.

Changes to any of the following require a design-system review: - font family; - icon family; - base colors; - spacing scale; - control heights; - radius system; - navigation structure; - table interaction model; - pipeline graph node anatomy; - core package stack (**react-aria-components**, TanStack Table, React Flow).

When code and this file disagree, either: 1. fix the code; or 2. update this file in the same pull request with a written rationale.

Do not silently drift.

27. Research references

- [R1] **GitLab Pajamas - Design**. Design tokens, components, patterns, dark-mode review, design-system workflow.
<https://design.gitlab.com/get-started/design/>
- [R2] **GitLab Pajamas - Design Tokens**. Semantic tokens and scalable theme/dark-mode practices.
<https://design.gitlab.com/product-foundations/design-tokens-using/>
- [R3] **GitLab Typography**. Readable sans/mono pairing and restrained typographic hierarchy.
<https://design.gitlab.com/brand-design/typography/>
- [R4] **Grafana Saga - Overview**. Open-source design system as a versioned product with foundations, components and patterns.
<https://grafana.com/developers/saga/about/overview/>
- [R5] **Grafana Saga - Accessibility**. Accessible component behavior and restrained live-region usage.

<https://grafana.com/developers/saga/foundations/accessibility/accessibility-styleguide/>

- **[R6] Nextcloud - Foundations.** Primary-color restraint, light/dark themes, typography and icon guidance.
https://docs.nextcloud.com/server/stable/developer_manual/design/foundations.html
- **[R7] Nextcloud - Layout.** Navigation/content/sidebar and list/content information hierarchies.
https://docs.nextcloud.com/server/stable/developer_manual/design/layout.html
- **[R8] PatternFly - Typography.** 14px body text, tabular numerals and technical hierarchy.
<https://www.patternfly.org/foundations-and-styles/typography/>
- **[R9] PatternFly - Dark Theme Handbook.** Semantic tokens and light/dark theme discipline.
<https://www.patternfly.org/developer-resources/dark-theme-handbook/>
- **[R10] Harbor UI package.** Current Harbor portal stack identifies Angular and Clarity.
<https://github.com/goharbor/harbor/blob/main/src/portal/package.json>
- **[R11] Gitea UI replacement proposal.** Preference for flexible, low-level/headless UI dependencies rather than a heavy styled framework.
<https://github.com/go-gitea/gitea/issues/29849>
- **[R12] Argo CD UI customization.** Multiple application detail views including tree, pods, network and list.
<https://github.com/argoproj/argo-cd/blob/master/docs/operator-manual/ui-customization.md>
- **[R13] Portainer package.** Current hybrid frontend with React migration, Storybook and modern React data tooling.
<https://github.com/portainer/portainer/blob/develop/package.json>
- **[R14] React Aria.** Style-free accessible components, focus management, keyboard and internationalization.
<https://react-aria.adobe.com/>
- **[R15] TanStack Table.** Headless data-grid engine with full control over markup and styling.
<https://tanstack.com/table/latest>
- **[R16] Tabler Icons React.** MIT-licensed React SVG icon package.
<https://github.com/tabler/tabler-icons/blob/main/packages/icons-react/README.md>
- **[R17] IBM Plex.** Open-source Sans/Mono family designed to work in UI environments.
<https://github.com/IBM/plex/blob/master/README.md>
- **[R18] WCAG 2.2 - Target Size Minimum.** Minimum target-size accessibility guidance.
<https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>
- **[Q1] HN - Density userstyle discussion.** Qualitative power-user discussion: density is valuable when information hierarchy remains clear.
<https://news.ycombinator.com/item?id=32627675>

- [Q2] **HN - High-information-density UI discussion.** Qualitative discussion praising information-dense professional interfaces.
<https://news.ycombinator.com/item?id=43925732>
- [Q3] **HN - Mobile data tables.** Qualitative discussion emphasizing task-focused columns, sticky identity and expandable rows.
<https://news.ycombinator.com/item?id=45483080>

Research interpretation note

Official project documentation and repositories are used to identify patterns and current technology choices. Hacker News links are included only as qualitative preference signals from technical users; they are not representative usability research and should not override direct Shipyard user testing.

SHIPYARD

UI/UX Design System & Implementation Specification

Research-backed design contract for human contributors and AI coding agents

Research snapshot: August 9, 2026 | Status: Normative

Executive design decision

Shipyards is a power-user engineering workspace for CI/CD, artifacts, packages, OCI images, runners and cluster operations. Its interface must prioritize state recognition, diagnosis, comparison and safe action over marketing-style presentation.

The recommended visual direction is industrial precision with subtle harbor identity: graphite neutral surfaces, Shipyards blue for primary action/selection, restrained dock orange for rare accents, compact geometry, strong information hierarchy, stable navigation, tables and split panes, and minimal decorative motion.

The research does not support copying one trendy framework. GitLab and Grafana maintain full design systems; Nextcloud documents recurring navigation/content/sidebar layouts; PatternFly explicitly addresses dense enterprise tables and semantic themes; Harbor, Gitea, Argo CD and Portainer use very different technical stacks. The common lesson is to own the product language and choose lower-level primitives that support it. [R1][R4][R7][R8][R10][R11][R12][R13]

Non-negotiable	Shipyards rule
Visual identity	No generic AI/SaaS dashboard look: no glassmorphism, ambient gradients, giant rounded-card mosaics or decorative sparkles.
Component ownership	Feature code consumes @shipyards/ui. React Aria supplies behavior; Shipyards owns visuals.
Typography	IBM Plex Sans + IBM Plex Mono.
Icons	Tabler Icons for product UI; Simple Icons/official marks for vendors.
Density	Medium-high desktop density; tables/lists before card grids.
Accessibility	WCAG 2.2 AA target; keyboard complete; status never color-only.
Visualization	Pipeline/topology graphs always have a list/table alternative.
Themes	Light, dark and system modes are semantic-token driven.

1. Research method and findings

Research was performed against current official design-system documentation, developer manuals and project repositories on August 9, 2026. Public technical-community discussions are included only as qualitative preference signals; they are not treated as a statistically representative usability study.

Project / source	Observed pattern	Shipyards takeaway
GitLab / Pajamas	Design tokens, component library, patterns, iconography and dark-mode review are maintained together.	Treat design system as product infrastructure; version tokens/components/patterns together. [R1][R2]
Grafana / Saga	Open-source foundations, components and patterns with accessibility guidance.	Document not only how a component looks but when and why it is used. [R4][R5]
Nextcloud	Theme-aware foundations and recurring navigation/content/sidebar and list/content layouts.	Use predictable shell and inspector patterns rather than inventing page layouts per feature. [R6][R7]
PatternFly	14px technical body scale, tabular numerals, semantic dark tokens and enterprise table guidance.	Optimize dense information surfaces deliberately. [R8][R9]

Harbor	Current portal identifies Angular + Clarity; registry hierarchy remains straightforward.	Borrow registry information architecture, not the framework. [R10]
Gitea	Maintainers discuss replacing a heavy styled dependency with more flexible/headless pieces.	Prefer low-level behavior primitives and project-owned styling. [R11]
Argo CD	Application details can use tree, pods, network or list views.	Never force a graph as the only operational view. [R12]
Portainer	Current repo shows incremental modernization around React/Storybook and modern data tooling.	Establish UI package boundaries early to avoid long-lived hybrid debt. [R13]

Qualitative power-user signals

- Technical users often value higher information density for tools they use frequently, but density must preserve hierarchy and readable line lengths. [Q1][Q2]
- Mobile data tables should prioritize the task and identity columns, then use expansion/column selection rather than blindly squeezing the desktop layout. [Q3]
- Treat these as directional signals. Direct Shipyard usability testing overrides them.

2. Frontend architecture

Concern	Choice	Implementation rule
Framework	React + TypeScript + Vite	Keep feature code simple and modular.
Accessible behavior	react-aria-components	Wrap in @shipyard/ui; do not inherit example styling. [R14]
Styling	CSS Modules + semantic CSS custom properties	No feature-local raw palette values.
Server state	TanStack Query	Remote async state only.
Tables	TanStack Table	Headless logic; Shipyard owns DOM and style. [R15]
Virtualization	TanStack Virtual	Large tables/logs/artifact lists/runners.
DAG/topology	React Flow	Custom Shipyard node visuals; list alternative required.
Code/YAML	Monaco Editor	shipyard.yml, diffs and config editing.
Logs	Custom virtualized log viewer	Do not use fake terminal chrome.
Interactive shell	xterm.js	Only real exec/shell sessions.
Charts	uPlot	Small operational time series only.
Icons	@tabler/icons-react	Single general icon family. [R16]
Component docs	Storybook	Every reusable state.
E2E/a11y	Playwright + axe-core + manual keyboard checks	Critical paths and accessibility are release gates.

```
apps/
  web/src/{app, features, routes, shell}/
packages/
  ui/src/{components, patterns, tokens, icons, typography, styles}/
  pipeline-graph/
  log-viewer/
  config-editor/
```

3. Visual identity

Shipyards should feel engineered rather than decorated. “Harbor” identity comes from container/dock structure, provenance and logistics terminology, not waves, anchors, rope textures or cartoon containers.

Element	Decision
Base	Graphite neutral canvas and surfaces.
Primary	Shipyards blue: action + selection.
Secondary accent	Dock orange: rare release/attention/brand accent.
Geometry	6px controls; 8px panels; borders before shadows.
Information	Tables, lists, split panes, inspectors, timelines.
Motion	Functional and restrained.

4. Color tokens

Semantic token	Dark	Light	Use
bg.canvas	#0B0D10	#F5F7F9	Application canvas
bg.surface	#111419	#FFFFFF	Navigation/main panes
bg.subtle	#15191F	#F8FAFC	Nested/alternate surface
bg.raised	#191E25	#FFFFFF	Popover/dialog only
border.default	#2A3038	#D8DEE6	Separators
border.strong	#3A424D	#B7C0CC	Strong boundary
text.primary	#E7EAF0	#1F2328	Primary text
text.muted	#9AA4B2	#59636E	Secondary text
action.primary	#6E97FF	#315EFB	Primary action/selection
accent.dock	#E2A45B	#B86519	Rare Shipyards accent
status.success	#45B97C	#177A49	Healthy/success
status.warning	#D8A13B	#A05E00	Warning/degraded
status.danger	#E26666	#B92C2C	Failure/destructive
status.info	#69A7E8	#256FAF	Informational
focus.ring	#9AB8FF	#1F5EFF	Keyboard focus

- No gradients in normal product chrome.
- Status colors only communicate status, never decoration.
- Primary blue means selection or primary action, not general importance.
- Use primary text plus a colored icon/border when colored text would not meet contrast.

5. Typography

Use IBM Plex Sans for UI and IBM Plex Mono for machine-readable content. IBM Plex is open source and explicitly designed to work in UI environments. [R17]

Style	Size / line	Weight	Use
Page title	24 / 32px	600	One per page
Section	18 / 26px	600	Major section
Panel title	15 / 22px	600	Panel/table heading
Body	14 / 20px	400	Default UI
Body strong	14 / 20px	500	Names/important values
Small	13 / 18px	400	Dense tables/metadata
Micro	12 / 16px	500	Badges/timestamps
Code	13 / 20px	400	Logs/YAML/commands
Code small	12 / 18px	400	Digests/coordinates

- Sentence case; avoid all caps except short protocol labels.
- Use tabular numerals for duration, counts, percentages, storage and changing metrics. [R8]
- Do not use monospace decoratively.

6. Geometry, spacing and density

Rule	Value
Base spacing grid	4px
Control radius	6px
Panel radius	8px
Large dialog radius	10-12px max
Compact control	28px
Default control	32px
Prominent control	36px
Table row compact	30px
Table row comfortable	36px
Topbar	48px
Sidebar expanded	224px
Sidebar collapsed	56px
Desktop page gutter	20-24px

Touch-oriented effective targets should approach 44px where practical while still meeting WCAG 2.2 minimum target-size requirements. [R18]

Borders before shadows: normal panels do not float. Shadows are reserved for real elevation such as menus, popovers and dialogs.

7. Icon system

General UI icons use @tabler/icons-react. Tabler provides React SVG components and is MIT licensed. [R16]
Vendor marks use Simple Icons or official vendor assets.

Meaning	Icon
Dashboard	IconLayoutDashboard

Projects	IconFolderCode
Pipelines	IconGitBranch
Jobs	IconPlayerPlay
Artifacts	IconPackage
Registry	IconContainer
Releases	IconRocket
Deployments	IconCloudUpload
Runners	IconServer2
Cluster	IconTopologyStar3
Logs	IconTerminal2
Settings	IconSettings
Secrets	IconKey
Security	IconShieldCheck
Success	IconCircleCheck
Failure	IconCircleX
Running	IconLoader2
Queued	IconClock
Warning	IconAlertTriangle
Retry	IconRefresh
Stop	IconSquare
Copy	IconCopy
Download	IconDownload
Upload	IconUpload
Search	IconSearch
Command palette	IconCommand

- Navigation icons: 18px. Normal controls: 16px.
- No emoji product icons, no decorative sparkle/wand/brain icons.

8. Application shell and navigation

```

+-----+
| Topbar: context | search | status | account |
+-----+
| Sidebar | Page header: title | context | primary action |
|         +-----+
| Overview | Toolbar / filters / tabs |
| Projects | +-----+
| Pipelines | Main work area [optional inspector] |
| Artifacts | |
| Registry | |
| Releases | |
| Deploy | |
| Runners | |
| Cluster | |
+-----+

```

Navigation order: Overview, Projects, Pipelines, Artifacts, Registry, Releases, Deployments, Runners, Cluster, Settings. Preserve relative order even when permissions hide sections.

Cmd/Ctrl+K opens a command palette for navigation and safe actions. High-impact destructive operations must still enter their explicit confirmation flow.

9. Component rules

Component	Contract	Anti-generic rule
Button	28/32/36px; primary/secondary/ghost/destructive/link /icon-only; 16px icons.	No gradients, giant pills or glowing CTA.
Input	32px; persistent label; helper/error below.	No floating-label gimmick.
Select/ComboBox	React Aria behavior; typeahead; search for large/remote option sets.	No stock component-demo styling.
Table	30/36px rows; sort/filter/columns/density; virtualization for large sets.	Do not convert comparison data into a card wall.
Badge	Status/environment/role/tag/runner label only.	Do not pill every metadata value.
Dialog	Typical max 560px; target + impact in destructive flows.	No giant centered marketing modal.
Inspector	Right-side detail pane that preserves list/DAG context.	Do not navigate away for every detail.
Toast	Transient confirmation/info; critical errors also persist in page state.	Never toast-only incident reporting.

10. Tables and data lists

Tables are a primary Shipyard interaction surface. TanStack Table supplies headless data-grid logic while Shipyard retains complete control over markup and styling. [R15]

- Stable identity/name column.
- Sortable headers and visible filters.
- Column chooser and global Compact/Comfortable density.
- Sticky header for long lists; optional sticky identity column.
- Row selection only when a bulk action exists.
- Numeric values right aligned with tabular numerals.
- Preserve useful query/filter state in the URL.
- On mobile, preserve identity/status, move secondary fields into expansion, and use column choice before forcing horizontal scroll. [Q3]

11. Pipeline UX

```
Build #1842 FAILED    commit a82c10f    production
[Retry failed] [Cancel] [More]

[Graph] [List] [Timeline]
+-----+-----+-----+-----+
| checkout -> test -> build -> image | Inspector | |
|                                     | build    |
|                                     \-> security |         |
|                                     | Failed in 04:31 |         |
```

	runner linux-04	
+-----+-----+		
	Live log panel (resizable)	
+-----+-----+		

- Graph node state uses icon + label + semantic color.
- List view is mandatory and the user choice is remembered. Argo CD similarly supports multiple detail views. [R12]
- Selected node opens inspector without losing graph/list context.
- Failure summary prioritizes failed step, concise reason, log location, retry option and runner context.

12. Logs, artifacts and registry

CI log viewer

Ordinary CI logs use a purpose-built virtualized log viewer: follow/pause, search, timestamp toggle, wrap, ANSI mapping, selection/copy, download, structured error markers and stable scroll behavior. Use xterm.js only for real interactive shell sessions.

Artifacts and packages

Artifact pages optimize for identify -> verify -> copy/use -> trace provenance. Default to structured lists/tables with name/path, type, size, checksum, producing pipeline, version/tag, created time and retention. The inspector shows SHA-256, source commit, run, runner, SBOM/provenance and copy/download commands.

OCI registry

Repository and manifest views surface digest, tag, media type, platforms, size, pushed time, provenance, referrers and copyable pull commands. Keep digest/provenance first-class. Do not decorate ordinary registry screens with cartoon container imagery.

13. Releases, deployments, runners and cluster

Releases and deployments

Release information should visibly connect source commit -> pipeline -> artifacts/images -> release -> environments. Production context must remain explicit near deployment actions. Rollback shows target release/digest and resulting change before confirmation.

Runner fleet

Use a fleet table: name, online/draining/offline, executor, labels, version, architecture, active/capacity, last heartbeat and current job. Support bulk drain, scheduling enable/disable and label management.

Cluster

Health comes before topology. First show control-plane availability, leader/lease state, PostgreSQL, object storage, optional queue/message layer and connected runners. Then show member/dependency tables and an optional topology view. The topology is never the only health representation.

14. Responsive behavior, motion and accessibility

Desktop is the primary creation/configuration environment, but mobile must support monitoring, approvals, incident checks and common safe actions. Narrow layouts collapse the sidebar, convert inspector to a sheet/detail route, preserve table identity/status, and default unreadable graphs to list view.

Motion is functional only: 100-160ms state transitions and 160-220ms overlays are acceptable. No ambient gradients, glows, parallax, springy page transitions or infinite decorative pulse. Respect prefers-reduced-motion. Target WCAG 2.2 AA. React Aria is intentionally style-free and includes accessible semantics, keyboard behavior and focus management, which lets Shipyard own visual styling without rebuilding basic accessibility behavior. [R14]

- Every primary workflow keyboard operable.
- Visible focus and logical focus return.
- Status icon + label + semantic color.
- Graph/list alternative.
- Live regions used sparingly; Grafana makes the same recommendation in its accessibility guidance. [R5]

15. Anti-generic / anti-AI visual guardrails

Do not default to	Use instead
Purple/blue/cyan gradients	Neutral surfaces + one primary blue + restrained dock orange
Glassmorphism/translucent panels	Opaque surfaces, borders, real elevation only
16-24px radii everywhere	6px controls / 8px panels
Bento card dashboard	Health strip + task-relevant tables/activity
Pill for every value	Plain text/table cells; chips only for state/tag/category
Sparkle/wand/brain icons	Branch/package/container/server/terminal/topology icons
Inter + Lucide + stock shadcn appearance	IBM Plex + Tabler + Shipyard-owned React Aria components
Huge whitespace	Balanced density and full-width operator surfaces
Shadow on every panel	Borders and spacing
Animated/glowing CTA	Solid primary action
Fake terminal chrome for logs	Purpose-built log viewer

Review question: if the Shipyard logo and name disappeared, would the page still look obviously designed for CI/CD, artifacts, registries, runners or cluster operations? If it looks like a generic admin dashboard, revise it.

16. Dashboard blueprint

```
Shipyard
System healthy | 24 runners | 3 jobs queued | 68% artifact storage
-----
Needs attention
! Pipeline #1842 failed   production/api           4m ago   [Open]
! Runner linux-07 offline cluster-east           9m ago   [Open]
-----
Recent pipelines          Recent releases
#1842 failed api         a82c10f 4m ago   1.9.0 production 31m ago
#1841 success web        118faca 9m ago   1.8.9 staging    2h ago
-----
Capacity / storage mini time series only when it drives action
```

Avoid turning each line above into a separate oversized rounded card. The dashboard answers: Is Shipyard healthy? What recently failed or changed? Is capacity constrained? What needs attention?

17. AI coding-agent implementation contract

1. **Classify the screen:** Choose table/list, list+inspector, graph+inspector, settings form, editor, operational dashboard, timeline/detail or terminal/log viewer.
2. **State the operator question:** Internally write: “The user opens this screen to answer: ____.” Major UI elements must support that question.
3. **Reuse first:** Search @shipyard/ui and existing patterns. Do not add another visual library/icon family.
4. **Implement all states:** Loading, loaded, empty, error, permission denied when applicable, stale/disconnected when applicable.
5. **Implement accessibility:** Keyboard path, focus order, labels, accessible names, non-color status and reduced motion.
6. **Stress content:** Test 100+ rows, long names, long artifact paths, full SHA-256, all statuses, zero results and partial data.
7. **Anti-generic review:** Reject gradients, card walls, random pills, arbitrary shadows, hero/marketing sections and stock demo compositions.
8. **Document and test:** Storybook for reusable UI; Playwright for critical workflow; accessibility and visual regression where maintained.

18. Example implementation conventions

```
:root {
  --sy-color-bg-canvas: #f5f7f9;
  --sy-color-bg-surface: #ffffff;
  --sy-color-text-primary: #1f2328;
  --sy-color-text-muted: #59636e;
  --sy-color-border-default: #d8dee6;
  --sy-color-action-primary: #315efb;
  --sy-radius-control: 6px;
  --sy-radius-panel: 8px;
  --sy-control-height-default: 32px;
}

[data-theme="dark"] {
  --sy-color-bg-canvas: #0b0d10;
  --sy-color-bg-surface: #111419;
  --sy-color-text-primary: #e7eaf0;
  --sy-color-text-muted: #9aa4b2;
  --sy-color-border-default: #2a3038;
  --sy-color-action-primary: #6e97ff;
}

import { Button, DataTable, StatusBadge } from '@shipyard/ui';
import { IconRefresh } from '@tabler/icons-react';
```

Feature code should use the owned component rather than styling a random native element “close enough.”

19. Definition of done and review checklist

- The main operator question is obvious.
- Dense comparable data uses a table/list.

- Primary action is obvious but not oversized.
- Graph has list/table alternative.
- IBM Plex Sans/Mono only.
- Tabler product icons only.
- Semantic tokens only; 4px spacing; 6px controls / 8px panels.
- No gradients in normal product chrome; shadows only for real elevation.
- Loading, empty, error, permission, stale/disconnected and long-content states are handled as applicable.
- Keyboard complete, visible focus, labeled controls, non-color-only status, theme contrast and reduced motion.
- Responsive monitoring path works.
- No bento-card default, giant hero, random pills, sparkle cliché or stock component-demo composition.
- Screen remains domain-specific even without Shipyard branding.

20. Governance

DESIGN.md is normative. Changes to font family, icon family, base colors, spacing scale, control heights, radius system, navigation structure, table interaction model, pipeline graph node anatomy or core UI stack require design-system review.

When code and DESIGN.md disagree, either fix the code or update DESIGN.md in the same pull request with rationale. Do not silently drift.

21. Research references

[R1] **GitLab Pajamas - Design.** Design tokens, components, patterns, dark-mode review, design-system workflow. <https://design.gitlab.com/get-started/design/>

[R2] **GitLab Pajamas - Design Tokens.** Semantic tokens and scalable theme/dark-mode practices. <https://design.gitlab.com/product-foundations/design-tokens-using/>

[R3] **GitLab Typography.** Readable sans/mono pairing and restrained typographic hierarchy. <https://design.gitlab.com/brand-design/typography/>

[R4] **Grafana Saga - Overview.** Open-source design system as a versioned product with foundations, components and patterns. <https://grafana.com/developers/saga/about/overview/>

[R5] **Grafana Saga - Accessibility.** Accessible component behavior and restrained live-region usage. <https://grafana.com/developers/saga/foundations/accessibility/accessibility-styleguide/>

[R6] **Nextcloud - Foundations.** Primary-color restraint, light/dark themes, typography and icon guidance. https://docs.nextcloud.com/server/stable/developer_manual/design/foundations.html

[R7] **Nextcloud - Layout.** Navigation/content/sidebar and list/content information hierarchies. https://docs.nextcloud.com/server/stable/developer_manual/design/layout.html

[R8] **PatternFly - Typography.** 14px body text, tabular numerals and technical hierarchy. <https://www.patternfly.org/foundations-and-styles/typography/>

[R9] **PatternFly - Dark Theme Handbook.** Semantic tokens and light/dark theme discipline. <https://www.patternfly.org/developer-resources/dark-theme-handbook/>

[R10] **Harbor UI package.** Current Harbor portal stack identifies Angular and Clarity. <https://github.com/goharbor/harbor/blob/main/src/portal/package.json>

[R11] **Gitea UI replacement proposal.** Preference for flexible, low-level/headless UI dependencies rather than a heavy styled framework. <https://github.com/go-gitea/gitea/issues/29849>

[R12] **Argo CD UI customization.** Multiple application detail views including tree, pods, network and list. <https://github.com/argoproj/argo-cd/blob/master/docs/operator-manual/ui-customization.md>

[R13] **Portainer package.** Current hybrid frontend with React migration, Storybook and modern React data tooling. <https://github.com/portainer/portainer/blob/develop/package.json>

[R14] **React Aria.** Style-free accessible components, focus management, keyboard and internationalization. <https://react-aria.adobe.com/>

[R15] **TanStack Table.** Headless data-grid engine with full control over markup and styling. <https://tanstack.com/table/latest>

[R16] **Tabler Icons React.** MIT-licensed React SVG icon package. <https://github.com/tabler/tabler-icons/blob/main/packages/icons-react/README.md>

[R17] **IBM Plex.** Open-source Sans/Mono family designed to work in UI environments. <https://github.com/IBM/plex/blob/master/README.md>

[R18] **WCAG 2.2 - Target Size Minimum.** Minimum target-size accessibility guidance. <https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>

[Q1] **HN - Density userstyle discussion.** Qualitative power-user discussion: density is valuable when information hierarchy remains clear. <https://news.ycombinator.com/item?id=32627675>

[Q2] **HN - High-information-density UI discussion.** Qualitative discussion praising information-dense professional interfaces. <https://news.ycombinator.com/item?id=43925732>

[Q3] **HN - Mobile data tables.** Qualitative discussion emphasizing task-focused columns, sticky identity and expandable rows. <https://news.ycombinator.com/item?id=45483080>

Interpretation note: official docs/repos identify current patterns and technology choices. Hacker News links are included only as qualitative signals from technical users; they are not representative usability research and do not override direct Shipyard user testing.

Research-Based Design System & UI Specification

For the open-source CI/CD, artifact, package, OCI registry, release, runner, and cluster platform

Version 2.0 | Research refreshed August 9, 2026

Core direction

Shipyards should look like a mature engineering tool that has been refined by operators over years: compact, precise, calm, fast, and domain-specific. It must not look like a generic AI-generated SaaS dashboard or a reskinned component-library demo.

Primary design influences: GitLab Pajamas, Grafana Saga, Nextcloud design guidelines, PatternFly, Harbor, Gitea, Argo CD, and Portainer. Community discussions are used only as qualitative signals, not statistical evidence.

Contents

1. Executive design position
2. Research method and evidence quality
3. What mature FOSS interfaces are doing
4. Community preference signals
5. Shipyard visual identity
6. Typography
7. Color system and themes
8. Geometry, density, spacing and layout
9. Iconography
10. Frontend design stack
11. Navigation and information architecture
12. Core component specifications
13. Domain screen specifications
14. Pipeline DAG and live execution UX
15. Logs, terminals and code
16. Artifact/package/OCI UX
17. Runner and cluster UX
18. Accessibility and input modalities
19. Responsive behavior
20. Motion and feedback
21. Anti-AI-look rules
22. Design engineering workflow and testing
23. Definition of Done
24. Research comparison matrix
25. Sources and references

1. Executive design position

Shipyard is not a marketing dashboard. It is an operator-facing engineering workspace. Its default screens must optimize for recognizing state, finding an object, understanding why something failed, and taking the next safe action.

Design thesis

Borrow the discipline of mature FOSS products, not their skins. Use a Shipyard-owned design system with headless/behavioral primitives underneath. Brand through typography, geometry, information architecture, and restrained accent use - not through gradients, glass, giant radii, or decorative illustrations.

Rule	Decision
Primary visual character	Industrial, technical, calm, compact, reliable. Subtle container/dock drafting cues are allowed; nautical decoration is not.
Density	Medium-high information density on desktop, with user-selectable Compact / Comfortable table density.
Surfaces	Mostly flat surfaces and borders. Elevation is rare and functional.
Color	Neutral graphite foundation + one Shipyard blue primary + restrained dock-orange accent. Status colors are semantic, not decorative.
Typography	IBM Plex Sans for UI; IBM Plex Mono for logs, code, hashes and coordinates.
Icons	Tabler Icons React for product UI; Simple Icons only for third-party brands.
Components	Owned Shipyard components built on React Aria Components; never expose raw library components directly to feature code.
Tables	First-class interaction surface using TanStack Table + virtualization. Avoid converting every data set into cards.
Animations	Short functional transitions only. No floating blobs, shimmer everywhere, parallax, or decorative spring motion.
Theme	Light, dark, and system. Both themes are designed, not automatically inverted.

2. Research method and evidence quality

The spec uses three evidence layers: official design systems/documentation from mature open-source projects; current project repositories/package manifests for implementation evidence; and community discussions for qualitative preference signals. Community discussion is explicitly treated as anecdotal.

Evidence class	Examples	How it is used
A - Official product/design docs	GitLab Pajamas, Grafana Saga, Nextcloud, PatternFly, Harbor docs, W3C	Foundation rules, accessibility, navigation, tables, theme strategy.
B - Current source manifests	Grafana, Harbor, Gitea, Argo CD, Portainer repositories	What large FOSS projects are actually shipping or migrating toward.
C - Community signals	Hacker News discussions on UI density, keyboard use, mobile tables	Helps tune density and interaction priorities; never treated as consensus.

3. What mature FOSS interfaces are doing

Project / system	Observed direction	Shipyard takeaway
GitLab / Pajamas	Full product design system with tokens, components, patterns, data visualization, dark-mode checking, and custom open-source UI typefaces.	Treat the design system as product infrastructure. Provide tokens, components, patterns, Figma/Penpot assets, and documentation together.
Grafana / Saga	Open-source design system with React components and repeatable patterns. Grafana also ships high-density data surfaces and specialized grids/charts.	Build reusable page templates for lists, detail views, metrics and configuration; keep operational data dense.

Nextcloud	Reusable Vue components, stable left navigation, 4px spacing multiples, minimum touch targets, system-minded themes, limited primary-color usage.	Use predictable navigation and 4px spacing. Reserve brand color for action and selection, not backgrounds everywhere.
PatternFly	Enterprise FOSS system with tokenized dark theme, 14px standard body text, data-heavy tables with filtering/sorting/toolbars, responsive table guidance.	Treat tables as primary workflows and support density modes, sticky headers, column controls, bulk actions and responsive fallbacks.
Harbor	Current Harbor UI identifies itself as Angular + Clarity and supports light/dark customization.	Registry pages should be straightforward and utilitarian. Borrow domain organization, not Clarity visual styling.
Gitea	Current repo includes Tailwind/Vue and maintainers have discussed replacing Fomantic with lower-level/headless pieces such as Floating UI.	Prefer low-level behavior primitives over a styled mega-library so Shipyard owns its look.
Argo CD	React UI uses Monaco/xterm/virtualized rendering; resource visualization is central and users can choose different application views.	Complex graphs should have alternate list views; visualization choice can be persistent per user/project.
Portainer	Current package includes React, TanStack Query/Table, Tailwind and Storybook while legacy Angular remains during migration.	A gradual design-system migration is realistic; make shared components/framework boundaries explicit from day one.

4. Community preference signals

Qualitative developer discussions repeatedly show a tension: users of technical tools value dense, stable interfaces, but excessive density without accessible targets and hierarchy becomes tiring. Shipyard should solve that tension rather than choosing either extreme.

Signal	Interpretation for Shipyard
Dense tables can be appreciated when they remain readable.	Default to useful information density, not empty whitespace. Use row hierarchy, tabular numerals, alignment, and optional density controls.
Users dislike interfaces where controls move, disappear, or do not look clickable.	Preserve stable layouts, obvious affordances, visible focus, and predictable action placement.
Keyboard-centric users value discoverable shortcuts and complete keyboard navigation.	Support Cmd/Ctrl+K, list/table navigation, slash/search focus, and show shortcuts in menus/tooltips without requiring memorization.
On mobile, blindly preserving every desktop table column performs poorly.	Prioritize the task: show essential fields first, allow expansion/column choice, and keep headers/identity visible.

Important

These are qualitative signals from public discussions, not a statistically representative usability survey. They are used to validate, not dictate, design choices.

5. Shipyard visual identity

5.1 Personality

Shipyard should feel like a control room, source browser, package warehouse, and deployment console built by one product team. The maritime metaphor is structural: containers, docks, manifests, routes, staging, loading, releases. Avoid anchors, waves, ropes, cartoon ships, port photography, or novelty nautical copy inside the product UI.

5.2 Brand mark guidance

Element	Rule
Logo concept	Abstract stack of containers / dock rails / S monogram. Must work at 16px favicon scale.
Wordmark	Sentence/title case Shipyard. Avoid extra-wide futuristic lettering.

Product chrome	No repeating logo watermarks. Logo only in global nav, auth screen, about/settings, and exported reports.
Illustration	Rare. Prefer schematic line art or product diagrams. Empty states should usually use icon + concise instruction instead of illustration.

6. Typography

The previous Inter-first direction is intentionally replaced. Inter is excellent, but it is also extremely common in modern generated SaaS interfaces. IBM Plex gives Shipyard a more technical, editorial identity while remaining open source and UI-appropriate.

Token	Family	Weight	Size / line-height	Use
type.pageTitle	IBM Plex Sans	600	24 / 32px	One page title.
type.section	IBM Plex Sans	600	18 / 26px	Major section heading.
type.panel	IBM Plex Sans	600	15 / 22px	Panel/card/table title.
type.body	IBM Plex Sans	400	14 / 20px	Default application text.
type.bodyStrong	IBM Plex Sans	500	14 / 20px	Emphasis, names, active values.
type.small	IBM Plex Sans	400	13 / 18px	Dense metadata and tables.
type.micro	IBM Plex Sans	500	12 / 16px	Badges, timestamps, compact labels.
type.code	IBM Plex Mono	400	13 / 20px	Logs, commands, snippets, YAML.
type.codeSmall	IBM Plex Mono	400	12 / 18px	Digests, hashes, image coordinates.

Use font-variant-numeric: tabular-nums for durations, build numbers, storage figures, throughput, percentages and other rapidly changing aligned metrics. Avoid all-caps labels except extremely short protocol/acronym badges such as OCI, AMD64, ARM64, SHA256.

7. Color system and themes

Color is restrained. Shipyard should not use neon cyan/purple gradients in global chrome. Dark surfaces are graphite, not tinted navy. Brand blue is used for selection and the primary action. Dock orange is a secondary brand signal used sparingly for release/staging emphasis, not as a second primary button color.

Semantic token	Dark	Light	Usage
canvas	#0B0D10	#F5F7F9	Application background
surface	#111419	#FFFFFF	Primary panes and sidebar
surface.subtle	#15191F	#F8FAFC	Nested rows / alternate surface
surface.raised	#191E25	#FFFFFF	Popover/dialog only
border	#2A3038	#D8DEE6	Default structural border
border.strong	#3A424D	#B7C0CC	Strong boundary
text	#E7EAF0	#1F2328	Primary text
text.muted	#9AA4B2	#59636E	Secondary text
text.faint	#6F7A87	#7A8591	Low-priority metadata
primary	#6E97FF	#315EFB	Primary action/selection
primary.hover	#89AAFF	#244CD8	Primary hover
accent.dock	#E2A45B	#B86519	Rare brand accent / release cue
success	#45B97C	#1F8A55	Healthy/success
warning	#D8A13B	#A96800	Warning/degraded
danger	#E26666	#C83737	Failure/destructive
info	#69A7E8	#256FAF	Informational
focus	#9AB8FF	#1F5EFF	Keyboard focus ring

Never encode status by color alone. Status surfaces use icon + label + color. No status color is used as decoration. Charts use a separate categorical palette and do not reuse success/danger unless the data itself represents success/failure.

8. Geometry, density, spacing and layout

Token/rule	Value	Notes
Spacing grid	4px base	All component spacing is a multiple of 4px.
Corner radius - control	6px	Buttons, inputs, selects, compact controls.
Corner radius - panel	8px	Cards, panes, popovers. Avoid 16-24px SaaS bubbles.
Corner radius - pill	999px	Only tags/status chips/avatar indicators.
Desktop control height	32px default / 28px compact	Touch mode expands hit target to >=44px where applicable.
Top bar	48px	Minimal; global context/search/account only.
Sidebar	224px expanded / 56px collapsed	Persistent on desktop.
Page gutter	20-24px desktop	Operational pages can use full width.
Readable form width	640-760px	Settings and text-heavy forms.
Table row	36px comfortable / 30px compact	User preference saved per account.
Panel border	1px	Borders define structure before shadows.
Shadow	Rare	Only dialogs/popovers/dropdowns; never every card.

8.1 Card policy

Cards are not the default layout primitive. Use them when a group has a distinct scope or action model. Do not wrap every table, metric, filter, and paragraph in separate rounded cards. On operational screens, prefer sections, split panes, toolbars, and bordered groups.

9. Iconography

Use one general icon family: @tabler/icons-react. Tabler is MIT-licensed, large, consistent, tree-shakeable, and less visually tied to the default shadcn/Lucide aesthetic. Third-party brand marks use Simple Icons, ideally through a thin Shipyard wrapper rather than exposing vendor packages everywhere.

Area	Tabler icon	Meaning
Dashboard	IconLayoutDashboard	Global overview
Projects	IconFolderCode	Projects
Pipelines	IconGitBranch	Pipelines/workflows
Jobs	IconPlayerPlay	Run/job
Artifacts	IconPackage	Artifacts/packages
Registry	IconContainer	OCI images/registry
Releases	IconRocket	Release
Deployments	IconCloudUpload	Deployments
Runners	IconServer2	Runners
Cluster	IconTopologyStar3	Cluster/topology
Logs	IconTerminal2	Logs/terminal
Settings	IconSettings	Settings
Secrets	IconKey	Secrets/credentials
Security	IconShieldCheck	Security/policy
Success	IconCircleCheck	Succeeded/healthy
Failure	IconCircleX	Failed/offline
Running	IconLoader2	Running
Queued	IconClock	Queued
Warning	IconAlertTriangle	Warning/degraded
Retry	IconRefresh	Retry/rerun
Stop	IconSquare	Stop/cancel
Copy	IconCopy	Copy value
Download	IconDownload	Download
Upload	IconUpload	Upload
Search	IconSearch	Search
Command	IconCommand	Command palette

Icon defaults: 16px in controls, 18px in navigation, 20px in empty-state/section contexts. Stroke 1.75 unless the source icon requires otherwise. Important actions should usually include text; icon-only actions require a tooltip and accessible name.

10. Frontend design stack

Layer	Choice	Why
Framework	React + TypeScript	Matches the original Shipyard stack and is proven in large FOSS developer tools such as Grafana and the React portions of Portainer.
Build	Vite	Fast dev/build pipeline; keep UI framework-independent at component boundaries.
Component behavior	react-aria-components	Style-free accessible behavior, keyboard/focus/touch semantics; Shipyard owns CSS and visual API.
Styling	CSS Modules + global CSS custom properties	No default library aesthetic. Easy theme tokens, predictable source ownership, no runtime CSS-in-JS requirement.
Server state	@tanstack/react-query	Cache, async lifecycle, refetch and mutation state.
Data grids	@tanstack/react-table	Headless table logic with full markup/style ownership.
Virtualization	@tanstack/react-virtual	Large logs, tables, artifact lists, runner fleets.
DAG/topology	@xyflow/react	Pipeline graph and bounded topology views using Shipyard custom nodes.
Code/YAML	monaco-editor	shipyard.yml, config diff and YAML editing.
Log rendering	Virtualized custom log viewer + ANSI parser	Do not fake every log as a terminal. Preserve search, timestamps, selection, wrapping.
Interactive terminal	xterm.js only when shell/exec is actually interactive	Keep terminal dependency out of ordinary log views.
Time series	uPlot	Small, fast operational time-series charts; already used in the Grafana ecosystem.
Icons	@tabler/icons-react	Single general-purpose visual language.
Brand icons	Simple Icons	Docker/GitHub/GitLab/Kubernetes/etc. only.
Storybook	Storybook	Design-system development, visual states and documentation.
E2E/a11y	Playwright + axe-core	Keyboard, accessibility, visual regression and critical-path tests.

Explicitly not recommended
Do not ship shadcn/ui default themes/components as the product look. Do not import a styled enterprise suite such as MUI/Ant/Clarity/PatternFly wholesale. Their ideas are useful research, but Shipyard should own its visual system.

11. Navigation and information architecture

Level	Items / behavior
Global sidebar	Dashboard, Projects, Pipelines, Artifacts, Registry, Releases, Deployments, Runners, Cluster, Administration. Items may be permission-hidden but their order is stable.
Top bar	Breadcrumb/project context, global search/command palette, notifications, docs/help, account. No duplicate primary navigation.
Project local nav	Overview, Pipelines, Artifacts, Packages, Registry, Releases, Deployments, Settings.
Detail navigation	Use tabs only when sections are peer views of the same object. Use right-side detail drawer for quick inspection without losing list context.
Command palette	Search objects + navigate + common safe actions. Destructive actions are never one-key palette actions.

12. Core component specifications

Component	Specification
-----------	---------------

Button	28/32/36px visual heights. Primary, secondary, ghost, destructive, link. One high-emphasis primary action per action group. No gradient fills.
Input	32px desktop; persistent label; helper/error below; monospace variant for tokens/paths.
Select/combobox	React Aria behavior; search for >8 options; keyboard and typeahead mandatory.
Table	Sticky header option, sort, filter toolbar, saved column visibility, resize/pin where useful, row actions, bulk selection, density toggle.
Badge	Short state/metadata only. Do not turn every noun into a pill.
Tabs	Peer subviews only; not global navigation. Underline/rail style, not chunky segmented pills by default.
Dialog	Focused creation/edit/confirmation. Destructive confirmations include exact target and impact.
Drawer	Quick details/inspection while maintaining list or graph context.
Toast	Transient success/info. Critical errors must also appear at the failure source or persistent activity center.
Tooltip	Clarifies unlabeled icons/abbreviations; never stores required instructions.
Skeleton	Shape follows content; no permanent shimmer spectacle. Honor reduced motion.
Empty state	Short explanation + one clear next action. Usually no illustration.
Inline alert	Persistent contextual warning/error with action. Avoid giant banner unless scope is global.

13. Domain screen specifications

Screen	Primary layout	Must answer
Dashboard	Operational summary + recent activity, not a card mosaic	Is Shipyard healthy? What failed? What is running/queued? Is capacity/storage constrained?
Project overview	Header + recent pipeline/release activity + environments	What changed recently and what can I do next?
Pipeline list	Dense table with branch, commit, trigger, duration, status, actor	Which run matters and why?
Pipeline run	Compact DAG/list switch + job inspector + live log split pane	What is running, blocked, failed, or waiting?
Artifacts	Repository/path browser + metadata drawer	What artifact is this, where did it come from, is it trustworthy, how do I fetch it?
OCI image	Tags/manifests/platforms/layers/provenance + pull command	Which digest/tag/platform should I use?
Packages	Package versions + coordinates + install snippet + metadata	How do I consume this version safely?
Releases	Version timeline + included artifacts + source commit + deploy status	What exactly is in this release?
Deployments	Environment list/timeline + current version + rollback/approval	What is deployed where, by whom, and can it be safely changed?
Runners	Fleet table + capacity + labels + heartbeat + drain state	Do I have enough healthy capacity and where are jobs going?
Cluster	Health summary + control-plane membership + dependency health + topology	Is the control plane healthy, who is leader/holder, and what dependency is degraded?

14. Pipeline DAG and live execution UX

The graph is a representation, not the only interface. Every pipeline graph must have a list/table alternative and retain job selection across view switches.

DAG rule	Specification
Node shape	Compact rectangle, typically 168-220px wide and 44-60px tall. 6px radius. No floating bubble nodes.
State	Left 3px state rail + icon + state text. Do not fill the whole node bright green/red.

Content	Job name, duration/state, runner label if relevant. Secondary metadata appears on selection/hover.
Edges	Neutral thin lines; dependency edges only. No decorative animation. Running path may use a subtle directional treatment, reduced-motion safe.
Scale	Cluster/group stages when graphs exceed a useful density. Use virtualization/LOD strategies if necessary.
Interaction	Click selects and opens inspector; keyboard navigation between nodes; fit/reset controls; minimap only on large graphs.

15. Logs, terminals and code

15.1 Job log viewer

Requirement	Rule
Rendering	Virtualized lines with incremental streaming. Preserve selection and scroll anchor during updates.
Controls	Search, follow/unfollow, timestamps, wrap, ANSI toggle, raw download, copy selected, jump to error/warning where available.
Typography	IBM Plex Mono 12.5-13px, line-height 18-20px, tabular numerals.
Chrome	No fake macOS terminal dots, title bars, or decorative shell prompt when displaying ordinary CI output.
Color	Honor ANSI where safe, but cap contrast and do not let logs recolor the surrounding UI.

15.2 Monaco editor

Use Monaco for shipyard.yml, policy/config editing and diffs. Default to split diff for changes, preserve keyboard shortcuts, show schema diagnostics inline, and avoid surrounding Monaco with unnecessary card chrome.

16. Artifact, package and OCI UX

Pattern	Specification
Provenance first	Every artifact detail view exposes producing pipeline, job, commit, timestamp, actor, checksum/digest and retention state near the top.
Copy-first values	Digests, pull coordinates, package install commands and repository URLs have explicit copy actions.
Path browser	Use a filesystem-like list with breadcrumb and metadata columns rather than card tiles for artifact repositories.
OCI tags	Table tags with digest, created time, size, platform count, signer/scan state if available.
Platform manifests	Display OS/architecture/variant clearly; monospace for digest only, not every metadata value.
Dangerous actions	Delete/retention changes explain dependency/replication impact and require scoped confirmation.

17. Runner and cluster UX

17.1 Runners

Runner management defaults to a table, not a node-card grid. Columns: name, state, current jobs/capacity, labels, executor, version, last heartbeat, machine/zone, drain state. Bulk drain/enable and label filtering must work for large fleets.

17.2 Cluster

Cluster mode should be understandable without requiring a topology diagram. The default is a health summary and membership/dependency tables. A topology view is secondary and uses the same status language as tables.

Cluster status	UI behavior
Healthy	Quiet success indicator; no celebratory green panel.
Degraded	Persistent inline alert naming the degraded dependency/node and known effect.
Leader/lease change	Timeline event, not a toast-only event.
Partition suspected	Surface last-seen, lease/fencing status, and affected scheduling scope.
Storage/DB issue	Global scoped alert with impacted operations; disable actions that cannot safely complete.

18. Accessibility and input modalities

Target WCAG 2.2 AA. The W3C minimum target-size criterion is 24x24 CSS pixels with defined exceptions; Shipyard should exceed that on touch-oriented layouts and aim for ~44px effective touch targets while preserving compact desktop visuals.

Area	Requirement
Keyboard	Every workflow usable without mouse. Arrow-key navigation where patterns support it; logical Tab order; Escape closes transient overlays.
Focus	Visible 2px focus ring with offset. Sticky headers/toolbars must not obscure focused elements.
Color	Contrast verified in both themes. Status never color-only.
Screen readers	Semantic headings/landmarks; forms labeled; state changes announced selectively. Do not announce every streaming log line.
Motion	prefers-reduced-motion respected. No flashing or infinite decorative animation.
Touch	>=24px WCAG minimum; use 44px effective targets for primary touch controls and mobile navigation where practical.
Graph alternative	Pipeline and topology graphs always have semantic list/table equivalents.
Localization	Layouts tolerate longer strings; use logical CSS properties; no text baked into images.

19. Responsive behavior

Shipyard is desktop-first because CI/CD, registry and cluster workflows are inherently information-dense. Mobile is not ignored, but it is optimized for monitoring, approvals, restarts, artifact lookup and incident triage rather than full pipeline authoring.

Width	Behavior
>=1280px	Full sidebar, split panes, rich tables, DAG + inspector.
960-1279px	Collapsible sidebar; hide low-priority columns; drawers replace some split panes.
640-959px	Sidebar becomes overlay; tables expose essential columns + row expansion/column chooser.
<640px	Monitoring mode: priority fields, stacked attribute/value detail, bottom/sheet actions; complex editors prompt for larger viewport when necessary.

20. Motion and feedback

Interaction	Timing / rule
Hover/focus	80-120ms color/border transition.
Popover/dialog	120-160ms opacity/scale <= 1.01.
Drawer	160-200ms.
Table row updates	No layout jump; optionally 400ms subtle background fade.
Running indicator	Small rotating loader or progress icon; never full-card pulsing glow.
Skeleton	Subtle static/low-motion placeholder; reduced-motion disables shimmer.

21. Anti-AI-look rules

"AI-looking" is not a formal UX category, so these are product-specific guardrails based on avoiding overused generated-dashboard motifs and matching the more restrained patterns observed in mature engineering tools.

Do not	Use instead
Purple/blue/cyan gradient backgrounds in product chrome	Neutral graphite/white surfaces with one primary blue and restrained dock orange.
Glassmorphism, blur panels, translucent cards everywhere	Opaque surfaces, 1px borders, rare elevation.
16-24px rounded cards wrapping every section	6-8px radius and structural sections/split panes/tables.
Giant four-card KPI grid as every dashboard	Compact health summary + task-relevant tables/activity.
Pill badges for every value	Plain text or table cells; chips only for statuses/tags/categories.
Sparkle/wand/brain icons or decorative AI glyphs	Domain icons: branch, package, container, server, terminal, shield, topology.
Inter + Lucide + stock shadow styling	IBM Plex + Tabler + Shipyard-owned components.
Huge whitespace with tiny content islands	Balanced density; full-width operational surfaces.
Soft shadow on every card	Borders and spacing; shadow only for overlays.
Animated gradient buttons / glowing primary CTA	Solid primary button with clear label and stable position.
Fake terminal chrome for logs	Purpose-built log viewer.
Generic hero-copy tone inside application	Concrete engineering language: Run pipeline, Publish artifact, Drain runner, Promote release.

22. Design engineering workflow and testing

Artifact	Requirement
Design tokens	Versioned CSS custom properties + TypeScript token metadata. Raw hex values prohibited in feature components except approved visualization palettes.
Component package	packages/ui with Storybook, visual tests, a11y tests and examples. Feature code imports Shipyard components only.
Patterns	Document list page, detail page, settings page, object picker, destructive action, live-run, logs, empty/loading/error patterns.
Design file	Penpot recommended for OSS-friendly collaboration; Figma export may exist but code/tokens are source of truth.
Testing	Playwright critical paths, axe-core checks, keyboard-only smoke tests, visual regression for themes/density.
Performance	Virtualize long lists/logs; avoid graph rendering of thousands of DOM nodes without clustering/LOD.

23. UI Definition of Done

- Uses semantic design tokens; no unapproved raw colors/radii/spacing.
- Works in dark, light, and system theme.
- Keyboard path tested and focus remains visible.

- Loading, empty, success, degraded, failed, permission-denied and offline states are designed.
- Long names, hashes, versions and translated strings do not break layout.
- Critical actions and errors are understandable without relying on color.
- Large lists/logs meet performance budget and do not shift layout while updating.
- Component is represented in Storybook when reusable.
- No default third-party visual styling leaks into Shipyard UI.
- UI review checks anti-AI-look guardrails as well as usability.

24. Research comparison matrix

Project	Current UI evidence (2026 research)	Notable pattern to borrow	Do not copy blindly
GitLab	Pajamas design system; GitLab Sans/Mono; token-driven dark mode.	Design system as product infrastructure; type hierarchy; patterns.	GitLab-specific visual/brand language.
Grafana	Saga open-source design system; React; Emotion; data grid and uPlot dependencies.	Dense operational data, reusable templates/patterns, specialized visualization.	Grafana dashboard aesthetic as a skin.
Nextcloud	Vue components; 4px spacing; stable left navigation; 44px touch guidance; native system typography.	Predictable navigation, restrained primary color, consistent reusable layout.	Consumer file-app spacing where CI surfaces need more density.
PatternFly	Tokenized themes, dense table system, 14px standard body, responsive table patterns.	Enterprise table behavior and accessibility discipline.	Red Hat visual identity or full component suite.
Harbor	Angular 21 + Clarity 18 in current portal package.	Registry information architecture and utilitarian project/repository focus.	Clarity styling or Angular simply because Harbor uses it.
Gitea	Tailwind config, Vue in repo; maintainers discuss replacing Fomantic with headless/low-level UI pieces.	Low-level component philosophy and lightweight server-rendered sensibility.	Legacy Fomantic appearance.
Argo CD	React UI, Monaco, xterm, react-virtualized; multiple application views.	Graph/list duality, code/editor integration, operator-focused resource state.	Legacy React stack/version choices.
Portainer	Current develop package mixes React migration with TanStack Query/Table, Tailwind, Storybook.	Container admin UX patterns; table-first operations; incremental migration realism.	Legacy hybrid framework complexity.

25. Sources and references

R1. GitLab Pajamas Design System - Official design system overview. <https://design.gitlab.com/>

R2. GitLab - Type fundamentals - UI typography, scale, GitLab Sans/Mono, contrast notes. <https://design.gitlab.com/product-foundations/type-fundamentals/>

R3. GitLab - Typography - Typeface rationale and brand typography. <https://design.gitlab.com/brand-design/typography/>

R4. Grafana Saga - Overview - Open-source design system principles and React component model. <https://grafana.com/developers/saga/about/overview/>

R5. Grafana Saga - Button - Button hierarchy and accessible icon-only guidance. <https://grafana.com/developers/saga/components/buttons/button>

R6. Grafana Saga - Text - Heading hierarchy and text component guidance. <https://grafana.com/developers/saga/components/text>

R7. Grafana repository package.json - Current frontend dependency evidence including React/Emotion/data-grid/uPlot ecosystem. <https://github.com/grafana/grafana/blob/main/package.json>

R8. Nextcloud - Foundations - Theme colors, typography, icon and legibility guidance. https://docs.nextcloud.com/server/stable/developer_manual/design/foundations.html

R9. Nextcloud - Layout components - Navigation, reusable components, 4px spacing and touch target guidance. https://docs.nextcloud.com/server/stable/developer_manual/design/layoutcomponents.html

R10. PatternFly - Typography - Enterprise typography, 14px body, tabular numerals, line length.

<https://www.patternfly.org/foundations-and-styles/typography/>

R11. PatternFly - Table guidelines - Table toolbars, sorting, actions and responsive patterns.

<https://www.patternfly.org/components/table/design-guidelines/>

R12. PatternFly - Dark theme handbook - Semantic tokens and theme implementation. <https://www.patternfly.org/design-foundations/theming/dark-theme-handbook/>

R13. Harbor portal package.json - Current Angular/Clarity implementation evidence.

<https://github.com/goharbor/harbor/blob/main/src/portal/package.json>

R14. Harbor docs - customize look and feel - Theme/customization evidence. <https://goharbor.io/docs/2.15.0/build-customize-contribute/customize-look-feel/>

R15. Gitea UI replacement discussion - Maintainer discussion of replacing Fomantic with lower-level/headless UI approaches. <https://github.com/go-gitea/gitea/issues/29849>

R16. Argo CD UI package.json - React, Monaco, xterm and virtualization evidence.

<https://github.com/argoproj/argo-cd/blob/master/ui/package.json>

R17. Argo CD UI customization - Multiple application detail views and operator customization.

<https://github.com/argoproj/argo-cd/blob/master/docs/operator-manual/ui-customization.md>

R18. Portainer develop package.json - React migration, TanStack Query/Table, Tailwind, Storybook evidence.

<https://github.com/portainer/portainer/blob/develop/package.json>

R19. React Aria - Style-free accessible React component behavior and interaction support. <https://react-aria.adobe.com/>

R20. TanStack Table - Headless data-grid/table engine. <https://tanstack.com/table/latest>

R21. React Flow - Custom node-based UI library. <https://reactflow.dev/>

R22. uPlot - Small, fast time-series charting. <https://github.com/leeoniya/uPlot>

R23. Tabler Icons - MIT-licensed icon set and React package. <https://github.com/tabler/tabler-icons>

R24. Simple Icons - CC0 brand icon source. <https://simpleicons.org/>

R25. IBM Plex - Open Font License family designed for UI and technical use. <https://github.com/IBM/plex>

R26. W3C WCAG 2.2 - Target Size Minimum - 24x24 CSS pixel minimum target criterion and exceptions.

<https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>

R27. Hacker News - UI density discussion - Qualitative community signal on dense but readable technical UIs.

<https://news.ycombinator.com/item?id=41088013>

R28. Hacker News - mobile table discussion - Qualitative community discussion of priority columns, sticky identity and task-focused mobile layouts. <https://news.ycombinator.com/item?id=45483080>

R29. Hacker News - keyboard-driven desktop discussion - Qualitative signal on discoverable keyboard navigation/shortcuts. <https://news.ycombinator.com/item?id=42884099>

Appendix A. Recommended package boundary

```
packages/
  ui/
    src/
      tokens/
      primitives/      # Shipyard wrappers over React Aria
      components/      # Button, Input, Table, Drawer, Dialog...
      patterns/        # ListPage, DetailPage, SettingsPage, RunInspector...
      icons/           # curated Tabler mappings + brand wrappers
      charts/
      theme/
    storybook/
  web/
    src/
      routes/
      features/
        projects/
        pipelines/
        artifacts/
        registry/
        releases/
        deployments/
        runners/
        cluster/
```

Appendix B. CSS token sketch

```
:root {
  --sy-font-sans: "IBM Plex Sans", ui-sans-serif, system-ui, sans-serif;
  --sy-font-mono: "IBM Plex Mono", ui-monospace, SFMono-Regular, monospace;

  --sy-radius-control: 6px;
  --sy-radius-panel: 8px;
  --sy-space-1: 4px;
  --sy-space-2: 8px;
  --sy-space-3: 12px;
  --sy-space-4: 16px;

  --sy-color-canvas: #f5f7f9;
  --sy-color-surface: #ffffff;
  --sy-color-border: #d8dee6;
  --sy-color-text: #1f2328;
  --sy-color-primary: #315efb;
  --sy-color-accent-dock: #b86519;
}

[data-theme="dark"] {
  --sy-color-canvas: #0b0d10;
  --sy-color-surface: #111419;
  --sy-color-border: #2a3038;
  --sy-color-text: #e7eaf0;
  --sy-color-primary: #6e97ff;
  --sy-color-accent-dock: #e2a45b;
}
```

Shipyard UI Research Matrix

Current FOSS design-system and frontend evidence gathered August 9, 2026. Community discussions are qu

Project / system	FOSS role	Current UI / stack evidence
GitLab / Pajamas	Large DevOps platform	Pajamas product design system, token-driven components/patterns; GitLab Sans/Mono UI
Grafana / Saga	Observability platform	Open source saga design system, react components; Grafana repo currently uses
Nextcloud	Collaboration/cloud suite	React-based components, left navigation, 1px spacing guidance; dark/light; minimum touch
PatternFly	Open-source enterprise design system	Responsive light/dark themes, 16px standard size, robust table toolbar/sort/filter/responsive
Harbor	OCI registry	Harbor 2.15 portal identifies itself as Angular + Clarity; supports light/dark look customization.
Gitea	Git/CI/package platform	discussed replacing Fomantic with
Argo CD	GitOps deployment platform	React UI with Monaco, xterm, react-virtualized; supports multiple application detail views.
Portainer	Container/Kubernetes management	React-based UI, uses Ant Design, Redux, Redux-Query/Table, Tailwind and Storybook while legacy

Qualitative signals, not statistical surveys.

Observed design pattern	Borrow for Shipyard
Design system is treated as product infrastructure, not a style sheet. Dense operational data, reusable page patterns, specialized visualization. Stable navigation, reusable layouts, restrained primary color. Enterprise data workflows are first-class design problems. Registry admin UI is utilitarian and domain-organized. Long-lived FOSS products benefit from owning visual styling. Complex operational state needs graph/list duality and code-centric tools. Large FOSS apps can modernize incrementally around shared React/data primitives.	Tokens + patterns + components + design docs stay versioned together. High-density technical surfaces and dedicated UI package. 4px grid, predictable nav, accessible targets. Table behavior, semantic tokens, tabular numerals. Project/repository/tag information hierarchy. Headless/low-level component philosophy. Pipeline graph + list switch; integrated YAML/diff/log tooling. TanStack table/query patterns and Storybook discipline.



Do not copy blindly

GitLab-specific brand treatment and page complexity.

Grafana dashboard aesthetic as a

Shipyard skin.

Consumer file-app spacing where CI

pages need more density.

Full Red Hat visual identity/component suite.

Angular/Clarity just because Harbor uses them.

Legacy Fomantic visual appearance.

Legacy React versions/dependency choices.

Hybrid-framework complexity.

Primary source

<https://design.gitlab.com/>

<https://grafana.com/developers/saga/about/overview/>

https://docs.nextcloud.com/server/stable/developer_manual/design/layoutcomponents.html

<https://www.patternfly.org/components/table/design-guidelines/>

<https://github.com/goharbor/harbor/blob/main/src/portal/package.json>

<https://github.com/go-gitea/gitea/issues/29849>

<https://github.com/argoproj/argo-cd/blob/master/ui/package.json>

<https://github.com/portainer/portainer/blob/develop/package.json>

Qualitative UI Preference Signals

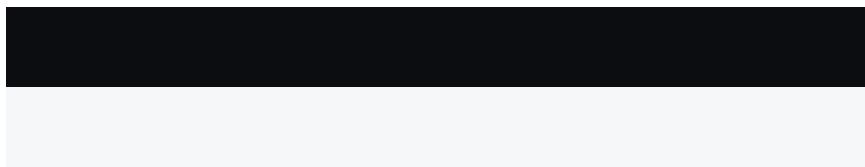
Public developer-community discussions used only to validate interaction priorities. Not statistical

Signal	Observed discussion
Readable density	Technical users in GitHub discussions often praise dense tables when hierarchy and readability remain strong; excessive
Stable, obvious controls	Community comments call out interfaces where buttons stop looking like buttons or layouts jump around.
Keyboard use	Keyboard-oriented users value discoverable shortcuts and predictable keyboard navigation.
Mobile tables	Users value essential columns, sticky identity, expansion/column choice rather than preserving

cally representative.

Shipyard decision

Medium-high desktop density; optional Compact / Comfortable table modes; no empty card mosaics.
Stable action placement, visible focus, obvious controls, no layout jumps during streaming updates.
Cmd/Ctrl+K, table/list navigation, shortcut hints in menus/tooltips, full keyboard workflows.
Mobile monitoring layout with essential fields, row expansion, column chooser and sticky identity.



Source

<https://news.ycombinator.com/item?id=41088013>

<https://news.ycombinator.com/item?id=41088013>

<https://news.ycombinator.com/item?id=42884099>

<https://news.ycombinator.com/item?id=45483080>

Shipyard Design Principles

Normative product principles derived from research and Shipyard's operator workflow.

Principle	Rule
Engineering workspace	Optimize for state recognition, diagnosis and safe action, not marketing presentation.
Own the visual system	Feature code consumes Shipyard components/tokens, not raw styled libraries.
Dense but calm	Use tables, lists, split panes and compact metadata with hierarchy.
Stable navigation	Global ordering and action placement should not change unpredictably.
Color is semantic	Brand color for selection/primary action; status colors only for status.
Borders before shadows	Structure with spacing and 1px borders; elevation only for overlays.
Graph is not the only view	Every pipeline/topology graph has a list/table equivalent.
Progressive disclosure	Details drawer/inspector opens from list or DAG without destroying context.
Accessibility is correctness	Keyboard, labels, contrast, focus and reduced motion are part of Done.

Why

Shipyards are used during builds, deploys and incidents.

Mature FOSS projects tend to maintain their own systems.

Technical users need more than a few card metrics per screen.

Reduces cognitive load and supports keyboard habits.

Prevents dashboard rainbow/noise.

Avoids generic SaaS/card aesthetic.

Scalability, accessibility and operator preference.

Supports fast incident triage.

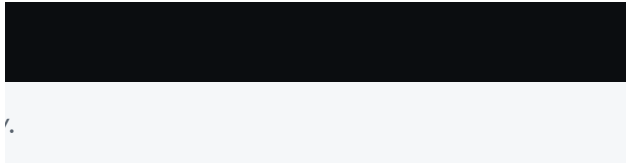
React Aria and WCAG 2.2 make this implementable.

Typography

IBM Plex replaces the previous Inter-first direction to give Shipyard a more technical, less generic SaaS identity

Token	Family	Weight	Size	Line height
type.pageTitle	IBM Plex Sans		600 24px	32px
type.section	IBM Plex Sans		600 18px	26px
type.panel	IBM Plex Sans		600 15px	22px
type.body	IBM Plex Sans		400 14px	20px
type.bodyStrong	IBM Plex Sans		500 14px	20px
type.small	IBM Plex Sans		400 13px	18px
type.micro	IBM Plex Sans		500 12px	16px
type.code	IBM Plex Mono		400 13px	20px
type.codeSmall	IBM Plex Mono		400 12px	18px

Use tabular numerals for durations, build numbers, storage figures, percentages and rapidly changing alignment protocol/architecture badges.



Usage

One page title

Major section heading

Panel/table/card title

Default UI text

Names and emphasized values

Dense tables and metadata

Badges/timestamps/compact labels

Logs, YAML, commands, snippets

Digests, hashes, coordinates

ed metrics. Avoid all caps except short

Color Tokens

Graphite foundation + Shipyard blue + restrained dock orange. No gradient

Token	Dark	Light
canvas	#0B0D10	#F5F7F9
surface	#111419	#FFFFFF
surface.subtle	#15191F	#F8FAFC
surface.raised	#191E25	#FFFFFF
border	#2A3038	#D8DEE6
border.strong	#3A424D	#B7C0CC
text	#E7EAF0	
text.muted	#9AA4B2	#59636E
text.faint	#6F7A87	#7A8591
primary	#6E97FF	#315EFB
primary.hover	#89AAFF	#244CD8
accent.dock	#E2A45B	#B86519
success	#45B97C	#1F8A55
warning	#D8A13B	#A96800
danger	#E26666	#C83737
info	#69A7E8	#256FAF
focus	#9AB8FF	#1F5EFF



is in *product chrome*.

Usage

- Application background
- Primary pane/sidebar
- Nested/alternate surface
- Popover/dialog only
- Default structural border
- Strong boundary
- Primary text
- Secondary text
- Low-priority metadata
- Primary action/selection
- Primary hover
- Rare brand/release cue
- Healthy/success
- Warning/degraded
- Failure/destructive
- Informational
- Keyboard focus ring

Geometry & Density

4px grid, compact operator UI, borders before shadows.

Token / rule	Value	Category
spacing.base	4px	Spacing
radius.control	6px	Radius
radius.panel	8px	Radius
radius.large	10-12px max	Radius
control.compact	28px	Control
control.default	32px	Control
control.prominent	36px	Control
touch.effective	~44px	Input
table.row.compact	30px	Table
table.row.comfortable	36px	Table
topbar	48px	Layout
sidebar.expanded	224px	Layout
sidebar.collapsed	56px	Layout
page.gutter	20-24px	Layout
form.content	640-760px	Layout
shadow	overlay only	Elevation



Notes

All spacing uses multiples of 4px

Buttons, inputs, compact controls

Cards/panes/popovers

Rare large dialog only

Dense desktop toolbar/table action

Default desktop control

Prominent primary form action

Touch-oriented layouts where practical

User-selectable compact density

Default comfortable density

Minimal global context/search/account

Stable desktop navigation

Collapsed navigation

Desktop operational page

Settings/text-heavy forms

Dialogs/popovers/dropdowns; not every card

Frontend Design Stack

Recommended implementation stack selected to maximize accessibility c

Layer	Package / choice
Framework	React + TypeScript + Vite
Accessible behavior	react-aria-components
Styling	CSS Modules + CSS custom properties
Server state	@tanstack/react-query
Data grids	@tanstack/react-table
Virtualization	@tanstack/react-virtual
Pipeline DAG	@xyflow/react
YAML/code	monaco-editor
Logs	Custom virtualized log viewer + ANSI parser
Interactive terminal	xterm.js
Operational charts	uPlot
Icons	@tabler/icons-react
Brand icons	Simple Icons
Component docs	Storybook
E2E/a11y	Playwright + axe-core

and Shipyard visual ownership.

Shipyard rule

Feature code should not depend on design-library internals.

Wrap in @shipyard/ui; style entirely with Shipyard CSS/tokens.

No stock component-library theme; semantic tokens only.

Async/server state only; keep local UI state local.

Headless; Shipyard owns markup, density and style.

Large tables, logs, artifacts and runner fleets.

Custom nodes; always pair with a list/table alternative.

shipyard.yml, diffs, policy/config editing.

Ordinary logs are not rendered as fake terminals.

Only for an actually interactive exec/shell session.

Small, fast time-series charts; no generic chart-card factory.

Only general product icon family.

Third-party logos only.

Every reusable component/state documented.

Critical-path, keyboard and accessibility tests.

Source

<https://react-aria.adobe.com/>

<https://tanstack.com/query/latest/docs/framework/react>

<https://tanstack.com/table/latest>

<https://tanstack.com/virtual/latest>

<https://reactflow.dev/>

<https://microsoft.github.io/monaco-editor/>

<https://xtermjs.org/docs/>

<https://github.com/leeoniya/uPlot>

<https://github.com/tabler/tabler-icons>

<https://simpleicons.org/>

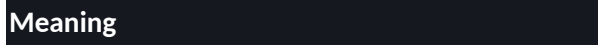
<https://storybook.js.org/>

<https://playwright.dev/>

Canonical Icon Map

General UI icons use @tabler/icons-react. Third-party vendor marks are separate.

Area	React icon	Default size
Dashboard	IconLayoutDashboard	18px
Projects	IconFolderCode	18px
Pipelines	IconGitBranch	18px
Jobs	IconPlayerPlay	16px
Artifacts	IconPackage	18px
Registry	IconContainer	18px
Releases	IconRocket	18px
Deployments	IconCloudUpload	18px
Runners	IconServer2	18px
Cluster	IconTopologyStar3	18px
Logs	IconTerminal2	16px
Settings	IconSettings	18px
Secrets	IconKey	16px
Security	IconShieldCheck	16px
Success	IconCircleCheck	16px
Failure	IconCircleX	16px
Running	IconLoader2	16px
Queued	IconClock	16px
Warning	IconAlertTriangle	16px
Retry	IconRefresh	16px
Stop	IconSquare	16px
Copy	IconCopy	16px
Download	IconDownload	16px
Upload	IconUpload	16px
Search	IconSearch	16px
Command palette	IconCommand	16px



Meaning

Global overview

Projects

Pipelines/workflows

Run/job

Artifacts/packages

OCI registry/images

Release

Deployment

Runner/machine

Cluster/topology

Logs/terminal

Settings

Secrets/credentials

Policy/security

Succeeded/healthy

Failed/offline

Running

Queued

Warning/degraded

Retry/rerun

Stop/cancel

Copy

Download

Upload

Search

Command palette

Component Rules

Reusable UI patterns should be owned by Shipyard and documented

Component	Visual / sizing
Button	28/32/36px; 6px radius
Input	32px default; persistent label
Select/Combobox	32px; React Aria behavior
Table	30/36px rows; sticky header optional
Badge	20-22px compact
Tabs	Rail/underline
Dialog	8-10px radius
Drawer	Right-side inspection
Toast	Compact transient
Skeleton	Content-shaped
Empty state	Icon + concise instruction

d in Storybook.

Behavior	Anti-generic rule
Primary, secondary, ghost, destructive, link; one high-emphasis action per group.	No gradients or giant pill buttons
Error/help below; monospace mode for paths/tokens.	No floating-label gimmick
Search >8 options; keyboard/typeahead mandatory.	No stock shadow select styling
Sort/filter/columns/bulk/density; virtualize large sets.	Do not replace useful tables with card grids
Status/tag/role/environment only.	Do not pill every metadata value
Peer subviews only.	No huge segmented-control navigation
Create/edit/confirm; destructive target + impact.	No giant centered marketing modal
Preserve list/DAG context.	Use for details, not every edit flow
Success/info; critical error persists elsewhere too.	No toast-only incident reporting
Reduced motion aware.	No shimmer everywhere
One clear next action.	Usually no illustration

Domain Screen Blueprint

Shipyard-specific page composition rules.

Screen	Default layout
Dashboard	Health summary + recent activity/tables
Project overview	Header + recent pipelines/releases/environments
Pipeline list	Dense table
Pipeline run	DAG/list + inspector + live log split
Artifacts	Path/repository list + metadata drawer
OCI image	Tags/manifests/platforms/provenance
Packages	Versions + coordinates + install snippet
Releases	Timeline + source + artifacts + environments
Deployments	Environment table/timeline + current version
Runners	Fleet table + capacity/labels/heartbeat
Cluster	Health + members/dependencies; topology secondary

Primary question	Avoid
Is Shipyard healthy and what needs attention?	Four giant KPI cards as the entire page
What changed and what can I do next?	Marketing-style project hero
Which run matters and why?	Card per run
What is running, blocked or failed?	Graph-only workflow
Where did this artifact come from and how do I fetch it?	Tile browser
Which digest/tag/platform should I use?	Decorative registry cards
How do I consume this safely?	Hide copyable commands
What exactly is in this release?	Release card gallery
What is deployed where and can it change safely?	Color-only status
Do I have enough healthy capacity?	Runner card wall
What is degraded and what is the effect?	Topology as the only truth

Anti-Generic / Anti-AI Guardrai

Product-specific rules to avoid overused generated S

Do not

Purple/blue/cyan gradients in product chrome

Glassmorphism / translucent panels

16-24px radii everywhere

Every section inside a rounded card

KPI card mosaic as default dashboard

Pill for every value

Sparkle/wand/brain icons

Inter + Lucide + stock shadow appearance

Huge whitespace

Shadow on every panel

Animated gradient CTA

Fake terminal chrome for logs

ls

aaS dashboard motifs.

Use instead

Neutral surfaces + one blue + restrained dock orange

Opaque surfaces, borders, rare elevation

6px controls / 8px panels

Sections, toolbars, split panes, tables

Compact health strip + task-relevant tables/activity

Plain text/table cell; chips only for state/tag/category

Branch/package/container/server/terminal/topology icons

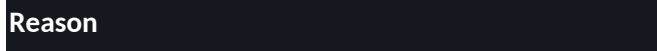
IBM Plex + Tabler + Shipyard-owned React Aria components

Balanced density and full-width operational surfaces

Borders and spacing

Solid primary action

Purpose-built log viewer



Reason

More mature engineering-tool visual language

Keeps state legible and avoids trend fingerprint

Less toy-like; more compact

Better density and hierarchy

Useful operational context

Reduces visual noise

Domain identity

More distinctive visual fingerprint

Supports scanning and real work

More restrained

Stable, accessible action hierarchy

Function over decoration

Accessibility Requirements

Target WCAG 2.2 AA. Accessibility is component correctness, not a later polish pass.

Area	Requirement
Keyboard	Every primary workflow operates without mouse.
Focus	Visible 2px focus ring and unobscured focused control.
Target size	Meet WCAG 2.2 minimum; use ~44px effective touch targets where practical.
Color	Status and action meaning never depends on color alone.
Screen reader	Semantic headings/landmarks; labeled controls; selective live announcements.
Motion	Respect prefers-reduced-motion.
Graphs	Pipeline/topology graph has list/table equivalent.
Localization	Layouts survive longer strings and RTL/logical properties.



Implementation note

React Aria primitives + explicit keyboard smoke tests

Sticky headers/toolbars must not cover focus

Desktop visual controls may remain compact with sufficient target/spacing

Icon + label + semantic color

Never announce every log line

No infinite decorative glow/pulse

Selection persists across view changes

No text baked into images

Research Sources

Official docs/repos first; community links explicitly

ID	Source
R1	GitLab Pajamas
R2	GitLab Type Fundamentals
R3	Grafana Saga
R4	Grafana package.json
R5	Nextcloud Foundations
R6	Nextcloud Layout Components
R7	PatternFly Typography
R8	PatternFly Table
R9	PatternFly Dark Theme
R10	Harbor portal package
R11	Gitea UI proposal
R12	Argo CD UI package
R13	Argo CD UI customization
R14	Portainer package
R15	React Aria
R16	TanStack Table
R17	React Flow
R18	uPlot
R19	Tabler Icons
R20	Simple Icons
R21	IBM Plex
R22	WCAG 2.2 target size
Q1	HN: UI density
Q2	HN: mobile tables
Q3	HN: keyboard-driven UI

labeled qualitative.

URL

<https://design.gitlab.com/>
<https://design.gitlab.com/product-foundations/type-fundamentals/>
<https://grafana.com/developers/saga/about/overview/>
<https://github.com/grafana/grafana/blob/main/package.json>
https://docs.nextcloud.com/server/stable/developer_manual/design/foundations.html
https://docs.nextcloud.com/server/stable/developer_manual/design/layoutcomponents.html
<https://www.patternfly.org/foundations-and-styles/typography/>
<https://www.patternfly.org/components/table/design-guidelines/>
<https://www.patternfly.org/design-foundations/theming/dark-theme-handbook/>
<https://github.com/goharbor/harbor/blob/main/src/portal/package.json>
<https://github.com/go-gitea/gitea/issues/29849>
<https://github.com/argoproj/argo-cd/blob/master/ui/package.json>
<https://github.com/argoproj/argo-cd/blob/master/docs/operator-manual/ui-customization.md>
<https://github.com/portainer/portainer/blob/develop/package.json>
<https://react-aria.adobe.com/>
<https://tanstack.com/table/latest>
<https://reactflow.dev/>
<https://github.com/leeoniya/uPlot>
<https://github.com/tabler/tabler-icons>
<https://simpleicons.org/>
<https://github.com/IBM/plex>
<https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>
<https://news.ycombinator.com/item?id=41088013>
<https://news.ycombinator.com/item?id=45483080>
<https://news.ycombinator.com/item?id=42884099>

Used for

Design-system structure

Typography/tokens/dark-mode evidence

Open-source design-system principles

Current React/Emotion/grid/chart stack evidence

Theme/type/icon guidance

Navigation/4px/touch guidance

14px body/tabular numerals/hierarchy

Dense table interaction

Semantic theming

Angular 21 + Clarity 18 evidence

Headless/low-level component direction

React/Monaco/xterm/virtualization evidence

Graph/list view choice

React/TanStack/Storybook migration evidence

Style-free accessible primitives

Headless data-grid engine

Node/DAG visualization

Fast time-series chart

Primary icon family

Brand icons

Open-source UI/mono font family

24x24 CSS px minimum criterion

Qualitative density/stability signal

Qualitative responsive-table signal

Qualitative keyboard signal

Shipyards Agent Prompts

This appendix contains the two working prompts prepared for Shipyards development and design exploration.

Prompt A - Cursor / Coding Agent Master Implementation Prompt

You are the lead software engineer responsible for implementing **Shipyards**, an open-source, self-hosted software delivery platform combining Jenkins-style CI/CD automation with JFrog Artifactory-style artifact, package, and OCI management.

Shipyards is one cohesive developer platform centered on this traceability chain:

source -> pipeline -> job -> build -> artifact/package/image -> release -> deployment

Every important object should be traceable through this chain.

Read first

Before writing code, inspect the repository and read all project-local instructions and Shipyards specifications. In particular, treat the following as authoritative when present:

- AGENTS.md
- CLAUDE.md
- DESIGN.md
- Shipyards architecture specification
- Shipyards UI/UX design specification
- repository README and build files

Existing repository code outranks assumptions. More specific project-local instructions outrank this general prompt.

Do not spend the entire session producing another plan. Inspect the current implementation state, identify the next coherent vertical slice, and begin implementing it unless there is a genuine blocker.

Product scope

Shipyards is intended to provide:

- pipeline-as-code CI/CD
- distributed runners
- shell and Docker execution
- BuildKit integration

- OCI/Docker registry
- generic artifact hosting
- package repositories
- local, remote, and virtual repositories
- releases
- deployment environments
- secrets and credentials
- webhooks
- users, organizations, and projects
- RBAC
- audit logging
- build logs
- artifact provenance and checksums
- build caching
- standalone deployment
- distributed multi-machine deployment
- clustered / HA control-plane deployment
- Docker Compose installation
- Kubernetes deployment
- a polished operator/developer UI

Shipyards are open-source first. Do not artificially cripple core self-hosted functionality.

Deployment modes

Shipyards must preserve compatibility with three deployment modes from the beginning.

Standalone

One machine may run:

```
shipyards-server
PostgreSQL
object storage
BuildKit
shipyards-runner
```

A simple self-hosted installation should be possible through Docker Compose. Standalone must use the same logical architecture as larger deployments. Do not create an incompatible “lite” implementation.

Distributed

Distributed means one Shipyards control plane plus multiple runner machines.

Runners should use authenticated outbound connections and expose capabilities such as:

```
linux
windows
macos
amd64
arm64
docker
kubernetes
gpu
high-memory
```

Support runner labels, concurrency, draining, maintenance state, heartbeats, registration, revocation, and job leases.

Cluster / HA

Cluster mode means multiple Shipyard control-plane instances behind a load balancer, using shared durable PostgreSQL and object storage.

Every implementation involving queues, scheduling, migrations, background jobs, leases, registry state, or periodic work must be evaluated for cluster safety.

Account for:

- scheduler coordination
- leases and fencing
- duplicate-job prevention
- leader election where required
- node failure
- runner reconnects
- API failover
- rolling upgrades
- migration coordination
- shared registry consistency

Do not call Shipyard HA simply because multiple HTTP servers can start.

Backend

Primary backend language: **Go**.

Start as a modular monolith rather than premature microservices.

Logical domains may include:

```
auth
organizations
projects
```

- git
- pipelines
- scheduler
- jobs
- runners
- artifacts
- registry
- packages
- releases
- deployments
- secrets
- webhooks
- audit
- cluster
- storage
- database

Keep boundaries clear without introducing meaningless forwarding layers.

Data and storage

Use PostgreSQL for transactional metadata and relational state.

Do not store large artifact bodies directly in PostgreSQL.

Use filesystem storage for simple standalone installs and an S3-compatible abstraction for production/distributed installs. The object-storage layer should be compatible with providers such as AWS S3, MinIO, Cloudflare R2, Ceph RGW, and Backblaze B2 S3.

Use cryptographic digests, especially SHA-256, as blob identity where appropriate.

Metadata belongs in PostgreSQL. Payload bytes belong in object storage.

OCI / Docker registry

Implement standards-compatible OCI Distribution behavior. Do not invent a proprietary image protocol.

Core concepts include:

- repositories
- blobs
- manifests
- tags
- digests
- uploads
- referrers

Normal Docker/OCI tools should eventually be able to log in, push, and pull.

Connect registry content to Shipyard provenance: pipeline run, job, commit, runner, release, and deployment.

Build system

Use BuildKit for container image builds. Do not implement a Docker builder from scratch.

Support Dockerfile builds, OCI output, pushes, caching, remote cache, multi-platform builds, build secrets, arguments, and remote BuildKit workers.

Pipelines

Canonical pipeline file: `shipyard.yml`.

Prefer a declarative pipeline model. Do not clone Jenkins Groovy.

Core concepts:

- trigger
- pipeline run
- job
- dependency
- step
- executor
- artifact
- environment
- secret
- output

Job dependencies form a DAG.

Define explicit validated state machines for pipeline runs and jobs. Never scatter state-transition rules through unrelated handlers.

Example job states:

- pending
- queued
- assigned
- running
- succeeded
- failed
- canceled
- timed_out
- lost
- skipped

Scheduler and leases

Runner assignment must remain correct with multiple schedulers and multiple control-plane nodes.

Use leases and fencing rather than a naive “select first queued job” pattern.

Protect against:

- duplicate dispatch
- control-plane restart
- stale workers
- runner disconnect
- lease expiry
- network partitions
- delayed completions
- cancellation racing completion

A stale execution attempt must not overwrite a newer attempt.

Runners

Primary runner binary: **shipyard-runner**, implemented in Go.

Runners should initiate connections to the control plane.

Runner responsibilities include:

- authentication
- capability registration
- heartbeat
- receiving leased work
- executor setup
- live log streaming
- artifact upload
- step/job state reporting
- lease renewal
- cancellation
- workspace cleanup

Initial executors: shell and Docker. Kubernetes, Windows, and macOS can follow.

Artifacts and packages

Artifacts are first-class entities with provenance.

An artifact should be traceable to organization, project, pipeline, run, job, commit, runner, checksum, release, and deployment when those relationships exist.

Repository concepts:

local
remote
virtual

Prioritize generic artifacts, OCI/Docker, Maven, and npm before expanding to many package protocols.

Releases and deployments

A release is an immutable relationship between source, artifacts/images, and deployment history - not just a string label.

Deployment records should track environment, release, initiator, timestamp, state, source commit, artifacts/images, approval state, and rollback relationships.

Security

Default toward least privilege.

Implement proper validation, parameterized SQL, safe path handling, secure cookies where applicable, CSRF protection where applicable, secure password hashing, scoped credentials, hashed persistent tokens, expiring registration tokens, digest verification, and repository-scoped registry authorization.

Never expose secrets in logs, API responses, ordinary frontend state, or audit payloads.

Webhook signatures must be verified.

Treat artifact names, paths, archives, registry uploads, manifests, tags, headers, runner reports, and webhook payloads as hostile input.

API and CLI

Expose a versioned HTTP API, for example `/api/v1`.

Use stable error shapes and do not leak raw Go errors.

Generate/document OpenAPI.

A future **shipyard** CLI should consume the public API where practical.

Frontend

Frontend stack:

React
TypeScript
Vite
react-aria-components

@tanstack/react-query
@tanstack/react-table
@tanstack/react-virtual
@xyflow/react
monaco-editor
@tabler/icons-react
Simple Icons
Storybook
Playwright
axe-core

Use uPlot for small operational time-series charts where appropriate. Use xterm.js only for actual interactive terminal sessions, not as fake chrome around normal build logs.

UI contract

Read and follow DESIGN.md exactly.

The intended direction is:

industrial precision
graphite surfaces
Shipyard blue
restrained dock orange
dense but calm
technical
operator-focused

Typography:

IBM Plex Sans
IBM Plex Mono

General product icon family:

@tabler/icons-react

Vendor marks use Simple Icons.

Do not introduce a second general-purpose icon family.

Anti-generic UI rules

Actively reject:

- purple-blue gradients
- cyan glow everywhere
- glassmorphism
- translucent dashboard cards
- giant rounded cards

- excessive pills
- shadows around every panel
- animated gradient buttons
- giant empty spaces
- KPI-card mosaics as the default dashboard
- card-per-row layouts where a table belongs
- decorative blobs
- generic AI-assistant aesthetics
- giant authenticated-page hero sections
- fake terminal decoration

Prefer tables, lists, split panes, inspectors, toolbars, timelines, and structured metadata.

Use a strict 4px spacing grid. Use restrained 6px control radii and 8px panel radii. Use borders and spacing before shadows.

Reuse before writing

Before creating a new component, helper, service, manager, hook, type, DTO, storage wrapper, or UI primitive, search the repository for an existing implementation.

Search by behavior, not just name.

If an existing implementation provides roughly 80% of the requirement, extend it instead of creating a near-duplicate.

Do not fork UI components for minor visual variations.

Simplicity

Use the lightest architecture that correctly solves the current milestone.

Do not introduce Kafka, RabbitMQ, NATS, Redis Streams, microservices, or plugin frameworks simply because Shipyard may be large someday.

Start with PostgreSQL-backed coordination and add specialized infrastructure only when a demonstrated requirement needs it.

Code comments

Do not add explanatory code comments by default. Prefer clear names, explicit state, and small functions. A one-line warning is acceptable only for a real invisible production footgun that cannot reasonably be expressed in code.

Working behavior

For each session:

1. inspect the repository;
2. read applicable architecture and design contracts;
3. determine what already exists;
4. identify the current implementation phase;
5. implement the next coherent vertical slice;
6. run relevant tests and builds;
7. inspect the complete diff as a hostile reviewer;
8. remove duplication and unnecessary abstraction;
9. report what changed and exactly what was verified.

Do not stop at directories, empty interfaces, placeholder handlers, blank pages, or TODO functions.

Mocks belong in tests. Do not present hardcoded frontend demo data as completed backend functionality.

Suggested implementation order

Phase 0 - foundation

Repository structure, Go server, frontend workspace, configuration, structured logging, PostgreSQL, migrations, storage abstraction, health/readiness, Docker development environment, and design tokens.

Phase 1 - identity and projects

Users, authentication, organizations, project model, membership, API tokens, initial RBAC, audit foundations, and real frontend project navigation.

Phase 2 - pipelines

shipyard.yml, parsing/validation, triggers, runs, jobs, steps, DAG, state machines, queue, pipeline list, pipeline detail, and DAG/list views.

Phase 3 - runners

Runner registration/authentication, capabilities, heartbeat, leasing, shell execution, logs, cancellation, cleanup, and a real remote-runner end-to-end flow.

Phase 4 - artifacts

Metadata, blob storage, SHA-256, upload/download, provenance, retention foundations, and pipeline -> job -> artifact traceability.

Phase 5 - BuildKit and OCI

Docker executor, BuildKit, registry blobs/manifests/tags, auth, image push/pull, and image provenance.

Phase 6 - releases and deployments

Release model, artifact/image relationships, environments, approvals, deployment history, and source -> build -> artifact -> release -> environment traceability.

Phase 7 - distributed mode

Multiple runners, capability matching, concurrency, draining, runner loss, re-connects, and lease expiry.

Phase 8 - cluster / HA

Multiple control-plane nodes, scheduling safety, migration locking, fencing, node failover, rolling restarts, shared PostgreSQL, and shared object storage.

Phase 9 - package protocols

Generic, Maven, npm, and additional protocols after the core repository model is stable.

Definition of done

Before calling work complete, ask:

- Does this work in standalone mode?
- Does it accidentally assume one control-plane server?
- Is shared state stored in the correct place?
- Are transitions and concurrency rules validated?
- Can duplicate execution occur?
- Are digests and paths validated?
- Does the frontend follow **DESIGN.md**?
- Are loading, empty, error, and permission states handled?
- Is keyboard navigation correct?
- Did this introduce a duplicate helper/component?
- Is there unnecessary abstraction?
- Did unrelated code change?
- Were relevant tests/builds actually run?

Final report format:

Implemented

- ...

Architecture

- ...

Verified

- ...

Remaining / next logical slice

- ...

The most important product rule is that Shipyard must answer, from one system:

Where did this artifact come from?

Which commit produced it?

Which pipeline and job built it?

Which runner executed it?

Which release contains it?

Where is that release deployed?

Can I verify its digest?

What failed, and why?

Build the platform around that traceability.

Prompt B - Claude Design / UI Concept Exploration

Create a high-fidelity concept exploration for **Shipyard**, an open-source platform combining CI/CD pipelines, distributed runners, artifact hosting, package repositories, OCI registry, releases, deployments, and cluster administration.

Before designing anything, read the provided `DESIGN.md` and Shipyard UI/UX design specification. They define the visual system and override generic defaults.

The purpose is not to invent a new design language. The purpose is to demonstrate the existing Shipyard design system on realistic product screens.

Product feeling

The interface should feel:

industrial

precise

technical

dense

calm

professional

open-source

operator-focused

The Shipyard identity should be expressed through container/dock/assembly structure and terminology, not cheesy nautical decoration.

Do not fill screens with boats, waves, anchors, ropes, ocean artwork, or novelty themes.

Do not make it look AI-generated

Actively avoid:

- purple and blue-purple gradients
- glowing cyan borders
- glassmorphism
- translucent cards
- giant rounded cards
- giant KPI tiles
- huge whitespace
- gradient buttons
- floating card mosaics
- pills around every value
- sparkle/wand/brain icons
- abstract 3D blobs
- generic “Welcome back” dashboards
- meaningless mini charts
- excessive animation
- giant hero headings inside the app
- fake terminal window decoration

The product should look like a mature FOSS engineering tool intentionally designed by people who use CI, registries, containers, runners, and production infrastructure.

Look for the maturity and density of projects such as GitLab, Grafana, Argo CD, Harbor, Gitea, PatternFly, and Portainer without copying any one product.

Visual system

Typography:

IBM Plex Sans

IBM Plex Mono

Use IBM Plex Mono for logs, commands, hashes, OCI digests, artifact checksums, paths, YAML, package coordinates, and other machine-readable identifiers.

Use Tabler React icons for general product icons and Simple Icons for third-party vendor marks.

Use restrained geometry:

controls: 6px radius

panels: 8px radius

large dialogs: 10-12px maximum

Use a strict 4px spacing foundation.

Use borders and surface contrast before shadows. Shadows belong mainly on overlays such as menus, popovers, and dialogs.

Use medium-high desktop information density. Tables, lists, split panes, inspectors, timelines, and structured metadata are preferred over giant card grids.

Colors

Dark direction:

canvas: #0B0D10

surface: #111419

subtle surface: #15191F

border: #2A3038

primary text: #E7EAF0

muted text: #9AA4B2

Shipyards blue: #6E97FF

dock orange: #E2A45B

Light direction should use the light-theme tokens from the design contract, with Shipyards blue around #315EFB and restrained dock orange around #B86519.

Orange is rare. Status colors are semantic and must always be accompanied by an icon and/or text label.

Application shell

Create a consistent shell with a compact top bar and stable left navigation.

Navigation should include:

Dashboard

Projects

Pipelines

Artifacts

Registry

Packages

Releases

Deployments

Runners

Cluster

Settings

The top bar may contain organization context, global search, Cmd/Ctrl+K command palette access, notifications, help, and account controls.

Required concept screens

Create at least eight realistic screens that clearly belong to one design system.

1. Global operational dashboard

Answer immediately:

- Is Shipyard healthy?
- What is running?
- What recently failed?
- Are runners available?
- Are there cluster problems?
- What was recently released/deployed?

Use compact health information, running-pipeline tables, recent failures, releases/deployments, runner capacity, and cluster warnings. Do not make the page four enormous metric cards.

2. Pipeline run

Show a realistic pipeline run such as:

```
Universe
Pipeline #1842
production
commit a82c10f
Build and Deploy
```

Provide Graph and Jobs views.

Example DAG:

```
Checkout
  |
Tests
  |
Build
  / \
Package Docker Image
  \ /
Release
  |
Production
```

Nodes should remain compact and technical, showing job name, state, duration, and runner where useful.

3. Job inspection and live logs

Use a split layout with job details and a purpose-built live log viewer.

Log controls should include search, follow-tail, wrap, timestamps, copy/download, and step markers. Use IBM Plex Mono. Do not use fake macOS terminal dots.

4. Artifact browser

Create an engineering-focused artifact repository browser with path navigation and a table showing name, type, size, modified time, checksum, and producing pipeline.

Selecting an artifact should open a detail inspector with provenance, commit, runner, release/deployment relationships, download, URL copy, and SHA-256 copy actions.

5. OCI registry

Show repository tags, digests, platforms, sizes, publish times, source pipeline, manifest/layer information, provenance, and a prominent copyable `docker pull` command.

6. Runner fleet

Design for 20, 200, or 2,000 runners. Use a table, not a card wall.

Useful columns include runner, status, current job, labels, OS, architecture, executor, concurrency, CPU, memory, heartbeat, and version.

Support filters and actions such as drain, resume, disable, and revoke.

7. Cluster / HA

Prioritize health before topology.

Show control-plane members, leader/follower state, PostgreSQL, object storage, scheduler/lease status, connected runners, version skew, and other dependencies. A small topology view may exist but must not be the only truth.

8. Release and deployment traceability

Show the lineage:

```
commit -> pipeline -> artifacts/images -> release -> staging/production
```

Expose artifacts, images, SBOM, provenance, deployment history, health, initiator, and timestamps.

Themes and responsive behavior

Show at least one complete dark example and one complete light example. Do not simply invert the dark theme.

Also show at least one narrow/mobile adaptation for a pipeline or deployment page. Prioritize status, identity, failure reason, and important actions; collapse secondary metadata intelligently.

Components and states

Demonstrate realistic states for buttons, inputs, statuses, tables, filters, loading, empty state, failure state, permissions, and disabled actions.

Tables should demonstrate sorting/filtering, column control, row selection when meaningful, and compact/comfortable density.

Use content-shaped skeletons. Errors should explain what failed and what the user can do next.

Example good error:

```
No eligible runner
```

```
This job requires linux + arm64 + docker.  
No online runner currently matches all requirements.
```

Command palette

Design a Cmd/Ctrl+K command palette with actions such as:

```
Go to project...  
Run pipeline...  
Open runner...  
Search artifact...  
Open release...  
Go to cluster...  
Create project...
```

It should look like an engineering productivity feature, not an AI chatbot.

Final quality test

Review every screen and ask:

1. Could this be mistaken for a randomly generated SaaS admin dashboard?
2. Does it clearly look purpose-built for CI pipelines, runners, OCI images, artifacts, releases, deployments, and production infrastructure?
3. Is the information density useful without becoming chaotic?
4. Does every screen visibly belong to the same Shipyard product?

If the first answer is yes, redesign it.

Historical / Superseded Design Material

Reference only - do not implement from this section when it conflicts with the current documents.

The following material predates the research-based redesign.

Current authority remains:

1. DESIGN.md
2. Shipyard Developer Architecture Specification
3. Shipyard UI/UX Design Specification
4. Research-Based Design System Specification / Workbook

The older material is preserved only because this PDF is an archival handoff of the Shipyard documents produced during planning.

SHIPYARD

Design System & UI Specification

Normative product-design contract for the OSS CI/CD + artifact platform



Status	Normative draft
Version	0.1
Date	August 2026
Companion files	DESIGN.mnd + Shipyard_Design_Tokens.xlsx

Contents

1. Product design direction
2. Design principles
3. Brand language
4. Color system
5. Typography
6. Iconography
7. Layout, grid & density
8. Spacing, radii, borders & shadows
9. Motion & interaction
10. Application shell
11. Core component specifications
12. Shipyard-specific screens
13. Pipeline DAG
14. Logs & terminal UX
15. Artifacts & registry UX
16. Runner / cluster UX
17. Data visualization
18. Responsive behavior
19. Accessibility
20. Frontend implementation contract
21. QA / definition of done
22. Source references

1. Product design direction

Shipyards must look and behave like a modern developer platform, not a reskinned Jenkins administration panel. The interface should communicate operational confidence: dense enough for engineers, calm enough for long sessions, and explicit about state, risk, and provenance.

Visual thesis

Industrial precision with subtle harbor identity. Maritime references may influence naming, spatial language, and the signal palette, but the UI must not rely on literal anchors, ropes, ships, wood textures, or other themed decoration. The product is infrastructure software first.

Attribute	Requirement
Default appearance	Dark-first, with fully supported light and system themes.
Density	Compact professional UI with a comfortable reading rhythm; no oversized consumer-dashboard cards.
Visual hierarchy	Typography, whitespace, borders, and status indicators before shadows or gradients.
Brand accents	Signal blue as primary; cyan/teal as operational accents. Accent color is used intentionally, not as decoration.
Interaction	Fast, keyboard-friendly, predictable. Every live state transition must be understandable without a refresh.
Domain fidelity	Hashes, digests, tags, branches, environments, nodes, and artifacts are first-class visual objects.

2. Design principles

Principle	Interpretation
Operational clarity over decoration	A failed build, degraded node, expired token, or production deployment should be immediately legible.
One source of truth	The same status vocabulary and token set is shared across dashboard, pipeline, registry, runners, and cluster views.
Progressive disclosure	Default screens show the information required for the next decision; advanced metadata remains one click away.
Copy-first developer workflows	Commands, digests, coordinates, URLs, and IDs expose one-click copy actions with clear confirmation.
Keyboard is a first-class input	Global navigation, search, command palette, table actions, dialogs, and editors must be usable without a mouse.
No color-only meaning	Icons, labels, text, or shape differences must accompany status colors.
Scale without visual redesign	A one-node install and a 100-node cluster use the same information architecture; detail increases rather than changing concepts.

3. Brand language

The Shipyards brand should feel technical, resilient, and purposeful. Use plain operational language: Build, Run, Publish, Promote, Deploy, Drain, Retry, Roll Back. Avoid cute microcopy during failures or destructive operations.

Area	Do	Avoid
Naming	Harbor / dock / fleet references only where they make functional sense.	Renaming standard DevOps concepts solely to fit the nautical theme.

Area	Do	Avoid
Illustration	Simple abstract geometry for empty states or docs.	Cartoon ships or nautical skeuomorphism in the product UI.
Tone	Concise, factual, action-oriented.	Jokes, blame language, or vague errors.
Logo placement	Product mark in app shell and auth/marketing surfaces.	Repeating the logo inside cards or content areas.

4. Color system

Colors are semantic design tokens. Components must consume tokens, never hard-coded hex values. Dark and light themes map the same semantic token names to different values.

Dark theme

Token	Swatch	Value	Usage
bg.canvas		#090C12	Primary app background
bg.surface		#0F141D	Cards, panes, sidebar
bg.surfaceRaised		#151B26	Menus, dialogs, elevated cards
bg.surfaceHover		#1A2230	Hover state on dark surfaces
border.subtle		#202938	Default separators and card borders
border.strong		#344054	Focused structural borders
text.primary		#F7F9FC	Primary text
text.secondary		#B6C0CE	Secondary text
text.muted		#7F8A99	Muted text; still AA on canvas
brand.primary		#4DA3FF	Primary Shipyard signal blue
brand.primaryHover		#76B8FF	Primary hover/active highlight
brand.cyan		#22D3EE	Secondary signal / live data accent
brand.teal		#3DD6B4	Success-adjacent operational accent
focus.ring		#79B8FF	Keyboard focus ring

Light theme

Token	Swatch	Value	Usage
bg.canvas		#F6F8FB	Primary app background
bg.surface		#FFFFFF	Cards, panes, sidebar
bg.surfaceRaised		#FFFFFF	Menus, dialogs, elevated cards
bg.surfaceHover		#EEF3F8	Hover state
border.subtle		#D8E0EA	Default separators and card borders
border.strong		#AAB7C6	Focused structural borders
text.primary		#111827	Primary text
text.secondary		#475569	Secondary text
text.muted		#64748B	Muted text
brand.primary		#2563EB	Primary action blue
brand.primaryHover		#1D4ED8	Primary hover/pressed

Token	Swatch	Value	Usage
brand.cyan		#0891B2	Secondary signal accent
brand.teal		#0F8A72	Operational accent
focus.ring		#2563EB	Keyboard focus ring

Semantic status colors

Token	Swatch	Value	Usage
status.success		#22C55E	Succeeded, healthy, online
status.running		#3B82F6	Running, deploying, syncing
status.queued		#A78BFA	Queued or waiting
status.warning		#F59E0B	Warning, degraded, needs attention
status.error		#DC2626	Failed, offline, destructive
status.canceled		#64748B	Canceled, disabled, inactive
status.info		#06B6D4	Informational state

Status usage rules

State	Color	Icon	Example
Success	#22C55E	CircleCheckBig	Pipeline succeeded / runner healthy
Running	#3B82F6	LoaderCircle	Job running / deployment progressing
Queued	#A78BFA	Clock3	Waiting for runner / approval
Warning	#F59E0B	TriangleAlert	Degraded cluster / nearing quota
Error	#DC2626	CircleX	Failed job / offline node
Canceled	#64748B	Ban	Canceled pipeline / disabled runner

5. Typography

Primary UI font: Inter Variable. Monospace font: JetBrains Mono. Do not introduce a third type family in the application. Marketing pages may reuse Inter Display optical sizing rather than adding another face.

Style	Family	Weight	Size / line-height	Tracking	Use
Display	Inter Variable	700	36px / 44px	-0.025em	Landing / empty-state hero only
H1	Inter Variable	700	28px / 36px	-0.020em	Page title
H2	Inter Variable	650	22px / 30px	-0.015em	Section title
H3	Inter Variable	650	18px / 26px	-0.010em	Card / panel title
Body LG	Inter Variable	400	16px / 24px	0	Long-form descriptions
Body	Inter Variable	400	14px / 20px	0	Default application text
Body SM	Inter Variable	400	13px / 18px	0	Dense tables / metadata
Label	Inter Variable	600	12px / 16px	0.01em	Form labels / uppercase avoided
Micro	Inter Variable	500	11px / 16px	0.01em	Timestamps / tiny metadata

Style	Family	Weight	Size / line-height	Tracking	Use
Code	JetBrains Mono	400	13px / 20px	0	Logs, commands, hashes, YAML, code
Code SM	JetBrains Mono	400	12px / 18px	0	Compact hashes / package coordinates

Typography implementation rules

- Use tabular numerals for durations, storage values, counts, timestamps, and metric dashboards.
- Use JetBrains Mono for commands, logs, YAML, digests, package coordinates, paths, commit hashes, IP addresses, and machine identifiers.
- Do not use monospace for normal labels merely because the product is technical.
- Uppercase text is reserved for tiny protocol/acronym labels; normal UI labels use sentence case.
- Long hashes display a compact middle-ellipsis representation but copy the full value.
- Font loading must use font-display: swap and local/system fallbacks. Font files are dependencies, not embedded into this specification.

6. Iconography

Lucide React is the only general-purpose product icon family. Use direct named imports from lucide-react for tree-shaking. Use react-icons/si only for vendor or technology logos where a brand mark is genuinely useful. Do not mix Font Awesome, Material, Heroicons, and Lucide for ordinary UI controls.

Category	Lucide icon	Meaning
App / Navigation	LayoutDashboard	Dashboard / overview
App / Navigation	FolderGit2	Projects
App / Navigation	GitBranch	Pipelines / branches
App / Navigation	Package	Artifacts / packages
App / Navigation	Container	OCI / container registry
App / Navigation	Rocket	Releases / deployments
App / Navigation	Server	Runners / nodes
App / Navigation	Network	Cluster / topology
App / Navigation	Settings	Settings
App / Navigation	ShieldCheck	Security / policies
App / Navigation	Users	Members / teams
App / Navigation	KeyRound	Secrets / credentials
Global	Search	Global search
Global	Bell	Notifications
Global	CircleHelp	Help / docs
Global	Command	Command palette / shortcuts
Actions	Play	Run / retry pipeline
Actions	Square	Stop job
Actions	RotateCcw	Retry / rerun
Actions	RefreshCcw	Refresh / resync
Actions	Copy	Copy digest / command
Actions	Download	Download artifact
Actions	Upload	Upload artifact
Actions	ExternalLink	Open external resource
Actions	MoreHorizontal	Overflow menu
Actions	Trash2	Delete / destructive action
Status	CircleCheckBig	Succeeded / healthy
Status	CircleX	Failed / offline
Status	LoaderCircle	Running / loading (animated)

Category	Lucide icon	Meaning
Status	Clock3	Queued / scheduled
Status	TriangleAlert	Warning / degraded
Status	CircleDashed	Pending / unknown
Status	Ban	Canceled / blocked
DevOps	SquareTerminal	Logs / shell
DevOps	FileCode2	Pipeline configuration / source
DevOps	GitCommitHorizontal	Commit
DevOps	GitPullRequest	Pull / merge request
DevOps	Tag	Tag / release tag
DevOps	Hash	Digest / identifier
DevOps	Database	Database / metadata service
DevOps	HardDrive	Storage
DevOps	Cloud	Cloud storage / remote service
DevOps	LockKeyhole	Protected / secret
DevOps	Fingerprint	Signature / provenance
Tables	ListFilter	Filter
Tables	Columns3	Column visibility
View	Maximize2	Fullscreen
View	PanelLeftClose	Collapse sidebar
View	Moon	Dark theme
View	Sun	Light theme
View	Monitor	System theme

Brand icon exceptions

Brand	Package	Import	Use
GitHub	react-icons/si	SiGithub	Git provider / repository
GitLab	react-icons/si	SiGitlab	Git provider / repository
Docker	react-icons/si	SiDocker	OCI / Docker compatibility
Kubernetes	react-icons/si	SiKubernetes	Kubernetes executor / deployment
PostgreSQL	react-icons/si	SiPostgresql	Database integration / system status
Cloudflare	react-icons/si	SiCloudflare	R2 / edge integration
Google Cloud	react-icons/si	SiGooglecloud	Cloud integration
Microsoft Azure	react-icons/si	SiMicrosoftazure	Cloud integration

Icon sizing

Context	Size	Stroke	Rule
Inline / buttons	16px	2px	Default UI icon size
Sidebar	18px	2px	Align to 20px optical box
Status	16px	2px	Always pair with visible label in important contexts
Empty-state illustration icon	24-32px	1.75-2px	Use sparingly
Icon-only button	16px	2px	Button hit area must remain ≥ 32 px; accessible name required

```
import { Play, CircleCheckBig, TriangleAlert } from "lucide-react";
import { SiDocker, SiGithub, SiKubernetes } from "react-icons/si";

// Do: direct imports with semantic wrappers.
// Avoid: a dynamic icon loader for normal static UI icons.
```

7. Layout, grid & density

Token / element	Specification
Desktop baseline	1440px reference viewport; fluid to ultrawide.
Primary sidebar	248px expanded / 64px collapsed.
Top bar	56px.
Content gutters	24px desktop, 16px tablet, 12-16px mobile.
Grid	12-column for dashboards; 24px gutter.
Panel gap	16px default; 24px between major sections.
Readable prose width	72ch maximum for documentation/descriptive settings text.
Data surfaces	May span full available width; do not constrain logs/tables to prose width.
Control density	36px default control height; 32px compact tables/toolbars; 40px prominent primary actions.

Responsive breakpoints should follow Tailwind defaults unless a real product layout requires a local container-query breakpoint. Prefer container queries for reusable panels.

8. Spacing, radii, borders & shadows

Spacing token	Value
space.0	0px
space.1	4px
space.2	8px
space.3	12px
space.4	16px
space.5	20px
space.6	24px
space.8	32px
space.10	40px
space.12	48px
space.16	64px

Radius token	Value
radius.sm	6px
radius.md	8px
radius.lg	12px
radius.xl	16px
radius.full	999px

Shadow token	Value
shadow.popover	0 12px 36px rgba(0,0,0,.28)
shadow.dialog	0 24px 64px rgba(0,0,0,.36)
shadow.focus	0 0 3px color-mix(in srgb, var(--focus-ring) 32%, transparent)

Surfaces use borders before shadows. Default cards have a 1px subtle border and no shadow. Popovers and dialogs may use elevation shadows. Avoid glassmorphism and heavy blur in operational surfaces.

9. Motion & interaction

Interaction	Duration	Easing / behavior
Hover / press	100-140ms	ease-out

Interaction	Duration	Easing / behavior
Menu / popover	120-160ms	ease-out with minimal translate/scale
Dialog / sheet	160-220ms	ease-out; do not overshoot
Sidebar collapse	180-220ms	ease-in-out
Status spinner	Continuous	Only for genuinely active states
Live DAG changes	150-250ms	Opacity/color transition; avoid moving nodes unless topology changed

Respect prefers-reduced-motion. Never use flashing or pulsing animation to attract attention to failures. Critical status must remain obvious when animation is disabled.

10. Application shell

Region	Contents	Behavior
Sidebar	Shipyard mark, org switcher, Dashboard, Projects, Pipelines, Artifacts, Registry, Releases, Deployments, Runners, Cluster	Persistent; collapsible; selected item uses filled surface + subtle blue edge, not a giant saturated block.
Top bar	Breadcrumbs/project context, global search, command palette hint, notifications, help, profile	Sticky; page-specific primary action may live at right.
Page header	Title, short description, context badges, primary action	Avoid hero-sized headers inside the app.
Main	Cards, data tables, diagrams, logs, editors	Use route-level loading/error states that preserve shell.

11. Core component specifications

Component	Size	Variants	Rules
Button	32 / 36 / 40px heights	Primary, secondary, ghost, destructive, icon-only	No gradients; icon 16px; visible focus ring
Input	36px default	Text, password, search, token	Label above input; errors below; no placeholder-only labels
Select / Combobox	36px default	Single, multi, searchable	Use Radix/shadcn behavior; keyboard navigable
Tabs	32px compact	Underline or segmented depending context	Do not use tabs for primary app navigation
Card	12px radius	Default, interactive, metric, warning	1px subtle border; shadows only when elevation is meaningful
Table	36px row default	Dense data, virtualized when large	Sticky header, selectable rows, filters, column controls
Badge	20-24px	Status, environment, tag, role	Always pair status color with text/icon
Dialog	max 560px typical	Create/edit/confirm	Destructive confirms explicitly name the target
Sheet	420-520px wide	Details, filters, quick edit	Use when context should remain visible
Toast	Auto	Success/info/error	Never the only place critical errors are shown
Tooltip	Short only	Icon clarification	Not for essential instructions
Command palette	Centered	Navigation + actions	Cmd/Ctrl+K; search projects, pipelines, artifacts, settings

Component	Size	Variants	Rules
Skeleton	Match content shape	Loading	Prefer over layout-jumping spinners
Empty state	Purpose-built	No data / first use	Explain next action; one primary CTA

12. Shipyard-specific screens

Screen	Primary content	Design requirement
Global dashboard	Health, recent pipelines, releases, storage, runner/cluster status	One-screen operational pulse; configurable widgets later
Project overview	Repo, latest commit, pipeline health, artifacts, environments	Project context should persist in header/breadcrumbs
Pipeline list	Status, branch, commit, actor, duration, created time	Fast scanning; filters persisted in URL
Pipeline run	DAG + job list + details + live logs	DAG is informative, not decorative; status visible without relying on color
Job logs	Virtualized xterm-like viewer, search, timestamps, wrap toggle	Monospace; ANSI support; copy/download; pause auto-scroll
Artifacts	Repository browser, metadata, provenance, checksums, downloads	Breadcrumb path + metadata side panel
OCI image	Tags, manifests, platforms, layers, digest, pull command	Copy-first workflow; vulnerability/provenance slots reserved
Package repository	Package versions, coordinates, install snippet, metadata	Package-manager-aware install snippets
Releases	Version, artifacts, source commit, notes, environments	Tie source -> build -> artifact -> release -> deployment
Deployments	Environment timeline, status, approvals, rollback	Strong production warnings; environment badge always visible
Runners	Fleet table + capacity + labels + heartbeat + drain state	Bulk operations and filters for large fleets
Cluster	Control plane topology, DB/object store dependencies, leader/lease state	Health first; topology secondary; avoid fake network-map animation
Settings	Org/project tabs, RBAC, tokens, secrets, retention	Separate dangerous actions into explicit danger zone

13. Pipeline DAG

Use React Flow for the pipeline graph. The graph is a visualization of the canonical pipeline/job model, not the source of truth itself. A corresponding list/table view must remain available for accessibility and very large pipelines.

Node state	Visual treatment
Pending	Neutral border + CircleDashed
Queued	Violet status strip + Clock3
Running	Blue status strip + LoaderCircle; subtle animated spinner only
Succeeded	Green status strip + CircleCheckBig
Failed	Red status strip + CircleX
Canceled	Slate status strip + Ban
Skipped	Muted/dashed border + explicit Skipped label

Node content order: job name; status + duration; runner/executor; optional matrix key. Clicking opens a details sheet. Hover is supplementary only. Fit-to-view is allowed on first load; do not constantly recenter while jobs update.

14. Logs & terminal UX

Use xterm.js for ANSI-aware live logs where terminal fidelity matters. The surrounding Shipyard log viewer owns search, timestamps, controls, download, and job metadata.

Capability	Requirement
Streaming	Append without re-rendering the entire buffer; indicate reconnecting state.
Auto-scroll	Enabled until the user scrolls away; show “Jump to latest” when paused.
Search	Find text with next/previous and match count.
Timestamps	Toggleable; preserve original ordering.
Wrap	User-toggleable; default on for narrow layouts.
ANSI	Supported but normalized to maintain readability across light/dark themes.
Secrets	Redacted before logs are transmitted to the browser; UI masking is not the security boundary.
Large logs	Virtualized/buffered; downloading full raw logs remains available.

15. Artifacts & registry UX

The registry UI should be optimized for “identify, verify, copy, use.” Artifact identity and provenance are more important than decorative previews.

Object	Must show
Generic artifact	Path/name, size, SHA-256, content type, created time, producing pipeline/job, source commit, download/copy actions
OCI repository	Repository name, tag count, latest push, retention/policy status
OCI image/tag	Digest, manifest media type, platforms, layers, size, labels/annotations, push source, copy pull command
Package	Coordinates, versions, install command, checksum, source build, metadata, repository type
Release	Version/tag, source commit, artifact set, notes, promotion/deployment history

16. Runner / cluster UX

Standalone, distributed, and HA cluster deployments must share a common visual vocabulary. The difference is the amount of topology and redundancy shown, not a different product experience.

View	Requirement
Runner fleet	Name, online state, busy/idle, executor type, labels, architecture/OS, running jobs, capacity, last heartbeat, version, drain toggle

View	Requirement
Runner detail	Current/recent jobs, capabilities, registration identity, runtime diagnostics, logs, upgrade state
Control-plane nodes	Healthy/degraded/offline, role, version, active leases, request/error metrics
Cluster dependencies	Postgres, object storage, optional broker/cache, with dependency health distinct from Shipyard node health
Topology	Static meaningful graph showing real dependencies; no animated “network map” purely for aesthetics

17. Data visualization

Visualization	Rule
Metric cards	One primary number, trend/context secondary; no unnecessary sparkline if it adds no decision value.
Time-series charts	Consistent time axis, tooltips, explicit units, user timezone indication.
Status charts	Prefer bars/stacked bars to donut charts when precise comparison matters.
Storage	Show used / available with numeric labels; color only reinforces thresholds.
Durations	Use seconds/minutes formatting consistently; p50/p95 clearly labeled when shown.
Color	Chart palette must remain distinguishable in both themes and must not rely on red/green alone.

18. Responsive behavior

Width	Behavior
$\geq 1280\text{px}$	Full sidebar, multi-column dashboards, side-by-side DAG/details when useful.
768-1279px	Collapsible sidebar; dashboards reduce columns; details become sheets.
$< 768\text{px}$	Monitoring-first mobile UI: stacked cards, horizontal table overflow or purpose-built list views, simplified DAG read-only view.

Monaco Editor is not treated as a primary mobile editing surface. On small screens, allow read-only config inspection and defer advanced editing to desktop-sized layouts.

19. Accessibility

Shipyard targets WCAG 2.2 AA. Accessible behavior is part of component correctness, not a later QA pass.

Requirement	Rule
Contrast	Normal text $\geq 4.5:1$; large text $\geq 3:1$; meaningful UI boundaries/icons satisfy non-text contrast.
Keyboard	All interactive controls reachable and operable by keyboard; no keyboard traps.
Focus	Visible focus ring; focus is not obscured by sticky headers or overlays.

Requirement	Rule
Target size	Prefer $\geq 32 \times 32$ px control hit areas; critical mobile actions target ≥ 44 px where layout permits.
Status	Never communicated by color alone.
Labels	Icon-only actions require accessible names; forms use persistent visible labels.
Live status	Use polite ARIA live regions for important state changes; do not announce every streamed log line.
Motion	Respect reduced motion and avoid flashing.
DAG	Provide textual/list equivalent and keyboard-selectable nodes.
Errors	Describe what failed and recovery action where known; connect field errors to fields programmatically.

20. Frontend implementation contract

Layer	Choice / rule
Framework	React + TypeScript. Keep UI components framework-local; business API client isolated.
Styling	Tailwind CSS theme variables + CSS custom properties as the design-token API.
Component foundation	shadcn/ui source-owned components, backed by accessible primitives where appropriate.
Data fetching	TanStack Query for server-state caching, retries, invalidation, and background refresh.
Icons	lucide-react for product UI; react-icons/si for brand marks only.
Pipeline graph	@xyflow/react (React Flow).
Logs	@xterm/xterm plus required addons only.
Editor	monaco-editor for shipyard.yml editing/diff/validation on desktop.
State	Prefer URL state + server state; use local/global client stores only for true client-only state.
Testing	Unit/component tests plus Playwright-style end-to-end and visual regression coverage for critical routes.

Canonical token CSS

```

:root {
  --bg-canvas: #F6F8FB;
  --bg-surface: #FFFFFF;
  --text-primary: #111827;
  --text-secondary: #475569;
  --border-subtle: #D8E0EA;
  --brand-primary: #2563EB;
  --focus-ring: #2563EB;
}
.dark {
  --bg-canvas: #090C12;
  --bg-surface: #0F141D;
  --text-primary: #F7F9FC;
  --text-secondary: #B6C0CE;
  --border-subtle: #202938;
  --brand-primary: #4DA3FF;
  --focus-ring: #79B8FF;
}

```

Tailwind mapping example

```
@import "tailwindcss";

@theme {
  --font-sans: InterVariable, Inter, ui-sans-serif, system-ui, sans-serif;
  --font-mono: "JetBrains Mono", ui-monospace, SFMono-Regular, monospace;
  --color-shipyard-primary: var(--brand-primary);
  --color-canvas: var(--bg-canvas);
  --color-surface: var(--bg-surface);
}
```

Repository convention

```
apps/web/src/
  components/ui/           # owned shadcn-derived primitives
  components/shipyard/     # domain components
  features/pipelines/
  features/artifacts/
  features/registry/
  features/runners/
  features/cluster/
  styles/tokens.css
  styles/theme.css
  lib/icons.ts
  lib/query.ts
  lib/api/
DESIGN.mnd                 # normative design contract
```

21. QA / definition of done

- ☐ Uses only approved semantic tokens; no unexplained hard-coded color values.
- ☐ Works in dark and light themes, including hover, focus, disabled, error, and loading states.
- ☐ Keyboard path tested; visible focus order is logical.
- ☐ No color-only status communication.
- ☐ Loading, empty, partial-error, full-error, permission-denied, and offline/reconnecting states are implemented where relevant.
- ☐ Responsive behavior verified at mobile, tablet, standard desktop, and ultrawide widths.
- ☐ Dense data does not overflow invisibly; copyable machine identifiers remain available in full.
- ☐ Icons come from Lucide unless documented as a brand exception.
- ☐ Inter and JetBrains Mono roles are respected; arbitrary fonts are not added.
- ☐ Pipeline/log/registry live updates do not cause page jumps or destroy user scroll/selection state.
- ☐ Visual regression snapshots cover core states of shared components and critical pages.
- ☐ Accessibility checks include automated audits plus manual keyboard and screen-reader smoke tests for key workflows.

22. Source references

These references are implementation inputs, not permission to copy another product's visual identity. Shipyard should use the cited projects for documented behavior and capabilities while maintaining its own design system.

shadcn/ui — <https://ui.shadcn.com/docs> — Open-code component foundation and composition model

Tailwind CSS theme variables — <https://tailwindcss.com/docs/theme> — Design tokens as theme/CSS variables

Lucide React — <https://lucide.dev/guide/react/> — Primary React icon system; typed SVG components and tree-shaking

React Icons — <https://react-icons.github.io/react-icons/> — Brand icon integration through ES module imports

Simple Icons — <https://simpleicons.org/> — Canonical popular-brand SVG icon source used via react-icons/si where applicable

React Flow — <https://reactflow.dev/> — Pipeline DAG / topology visualization

xterm.js — <https://xtermjs.org/docs/> — Browser terminal/log rendering

Monaco Editor — <https://microsoft.github.io/monaco-editor/> — shipyard.yml editor and diff/validation UI

TanStack Query — <https://tanstack.com/query/latest/docs/framework/react> — Server-state fetching/caching lifecycle

Radix Primitives accessibility — <https://www.radix-ui.com/primitives/docs/overview/accessibility> — Keyboard, focus, ARIA behavior underlying accessible components

WCAG 2.2 — <https://www.w3.org/TR/WCAG22/> — Accessibility conformance target

Inter — <https://rsms.me/inter/> — Primary UI typeface; open-source UI-oriented family

JetBrains Mono — <https://www.jetbrains.com/lp/mono/> — Monospace typeface for code/logs/digests

Shipyard Design System

Design-token reference sheet for the Shipyard OSS CI/CD + artifact platform. Canonical prose rules live in Sh DESIGN.mnd.

Area	Decision	Implementation
Direction	Industrial precision; subtle harbor identity	Dark-first + complete light/system themes
UI font	Inter Variable	400-700 weights
Mono font	JetBrains Mono	400-600 weights
Product icons	Lucide React	lucide-react direct imports
Brand icons	Simple Icons via React Icons	react-icons/si
Components	shadcn/ui source-owned components	Tailwind + accessible primitives
Pipeline DAG	React Flow	@xyflow/react
Logs	xterm.js	@xterm/xterm
Editor	Monaco Editor	monaco-editor
Accessibility	WCAG 2.2 AA	Keyboard + contrast + focus + non-color status

ipyard_Design_System_Spec.docx and

Rule

No decorative nautical skeuomorphism

Only application sans-serif

Logs, code, YAML, paths, digests, hashes

Only general-purpose icon family

Vendor marks only; respect brand rules

Own and theme component code

Always provide a list/table equivalent

Shipyards own controls around terminal surface

Desktop-focused shipyard.yml editor

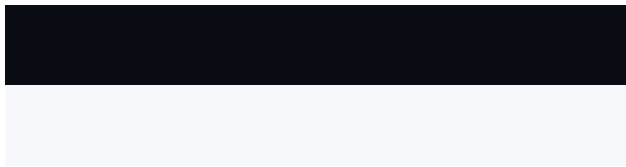
Part of Definition of Done

Color Tokens

All product components consume semantic tokens, not raw hex literals.

Token	Dark	Light
bg.canvas	#090C12	#F6F8FB
bg.surface	#0F141D	#FFFFFF
bg.surfaceRaised	#151B26	#FFFFFF
bg.surfaceHover	#1A2230	#EEF3F8
border.subtle	#202938	#D8E0EA
border.strong	#344054	#AAB7C6
text.primary	#F7F9FC	
text.secondary	#B6C0CE	#475569
text.muted	#7F8A99	#64748B
brand.primary	#4DA3FF	#2563EB
brand.primaryHover	#76B8FF	#1D4ED8
brand.cyan	#22D3EE	#0891B2
brand.teal	#3DD6B4	#0F8A72
focus.ring	#79B8FF	#2563EB

Status Token	Value	Icon
status.success	#22C55E	CircleCheckBig
status.running	#3B82F6	LoaderCircle
status.queued	#A78BFA	Clock3
status.warning	#F59E0B	TriangleAlert
status.error	#DC2626	CircleX
status.canceled	#64748B	Ban
status.info	#06B6D4	Info



Usage

Primary application background
Cards, panes, sidebar
Menus, dialogs, elevated cards
Hover state
Default border/separator
Strong structural/focus boundary
Primary text
Secondary text
Muted metadata
Primary actions and selected signal
Hover/pressed primary
Live/secondary operational signal
Operational accent
Keyboard focus ring

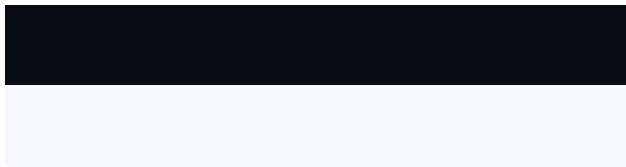
Meaning

Succeeded / healthy / online
Running / deploying / syncing
Queued / waiting
Warning / degraded
Failed / offline / destructive
Canceled / disabled
Informational

Typography

Inter Variable for UI; JetBrains Mono for machine-readable/code surfaces.

Style	Family	Weight	Size / Line	Tracking
Display	Inter Variable	700	36 / 44	-0.025em
H1	Inter Variable	700	28 / 36	-0.020em
H2	Inter Variable	650	22 / 30	-0.015em
H3	Inter Variable	650	18 / 26	-0.010em
Body LG	Inter Variable	400	16 / 24	0
Body	Inter Variable	400	14 / 20	0
Body SM	Inter Variable	400	13 / 18	0
Label	Inter Variable	600	12 / 16	0.01em
Micro	Inter Variable	500	11 / 16	0.01em
Code	JetBrains Mono	400	13 / 20	0
Code SM	JetBrains Mono	400	12 / 18	0



Use

Landing/empty-state hero only

Page title

Section title

Card/panel title

Long descriptions

Default UI text

Dense tables/metadata

Form labels

Tiny metadata/timestamps

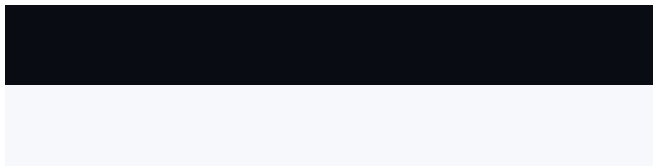
Logs, commands, YAML, hashes

Compact digests/coordinates

Spacing & Shape

4px base grid. Borders before shadows.

Token	Value	Category
space.0	0px	Spacing
space.1	4px	Spacing
space.2	8px	Spacing
space.3	12px	Spacing
space.4	16px	Spacing
space.5	20px	Spacing
space.6	24px	Spacing
space.8	32px	Spacing
space.10	40px	Spacing
space.12	48px	Spacing
space.16	64px	Spacing
radius.sm	6px	Radius
radius.md	8px	Radius
radius.lg	12px	Radius
radius.xl	16px	Radius
radius.full	999px	Radius
control.compact	32px	Sizing
control.default	36px	Sizing
control.prominent	40px	Sizing
sidebar.expanded	248px	Layout
sidebar.collapsed	64px	Layout
topbar.height	56px	Layout



Rule

Zero

Tight inline gap

Small gap

Compact component padding

Default panel/component gap

Expanded internal gap

Major section / desktop gutter

Large separation

Large vertical rhythm

Page-section separation

Rare large whitespace

Small controls

Inputs/buttons/popovers

Cards/panels

Large modal/special surface

Pills/avatars only

Dense table/toolbars

Default control

Primary prominent action

Desktop expanded navigation

Collapsed navigation

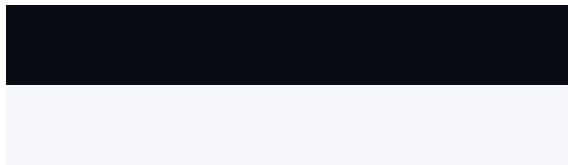
Sticky top bar

Icons

Lucide for product UI; react-icons/si only for third-party brands.

Group	Package	Icon / Import
Navigation	lucide-react	LayoutDashboard
Navigation	lucide-react	FolderGit2
Navigation	lucide-react	GitBranch
Navigation	lucide-react	Package
Navigation	lucide-react	Container
Navigation	lucide-react	Rocket
Navigation	lucide-react	Server
Navigation	lucide-react	Network
Navigation	lucide-react	Settings
Security	lucide-react	ShieldCheck
Security	lucide-react	KeyRound
Global	lucide-react	Search
Global	lucide-react	Bell
Global	lucide-react	CircleHelp
Global	lucide-react	Command
Action	lucide-react	Play
Action	lucide-react	Square
Action	lucide-react	RotateCcw
Action	lucide-react	Copy
Action	lucide-react	Download
Action	lucide-react	Upload
Action	lucide-react	Trash2
Status	lucide-react	CircleCheckBig
Status	lucide-react	CircleX
Status	lucide-react	LoaderCircle
Status	lucide-react	Clock3
Status	lucide-react	TriangleAlert
Status	lucide-react	Ban
DevOps	lucide-react	SquareTerminal
DevOps	lucide-react	FileCode2
DevOps	lucide-react	GitCommitHorizontal

DevOps	lucide-react	GitPullRequest
DevOps	lucide-react	Tag
DevOps	lucide-react	Hash
Brand	react-icons/si	SiGithub
Brand	react-icons/si	SiGitlab
Brand	react-icons/si	SiDocker
Brand	react-icons/si	SiKubernetes
Brand	react-icons/si	SiPostgresql
Brand	react-icons/si	SiCloudflare
Brand	react-icons/si	SiGooglecloud
Brand	react-icons/si	SiMicrosoftazure



Meaning

Dashboard
Projects
Pipelines
Artifacts/packages
OCI registry
Releases/deployments
Runners/nodes
Cluster/topology
Settings
Policies/security
Secrets/credentials
Global search
Notifications
Help/docs
Command palette
Run/retry
Stop job
Rerun/retry
Copy
Download artifact
Upload artifact
Delete/destructive
Succeeded/healthy
Failed/offline
Running/loading
Queued/scheduled
Warning/degraded
Canceled/blocked
Logs/shell
Pipeline/source config
Commit

PR/MR

Tag/release tag

Digest/identifier

GitHub

GitLab

Docker

Kubernetes

PostgreSQL

Cloudflare

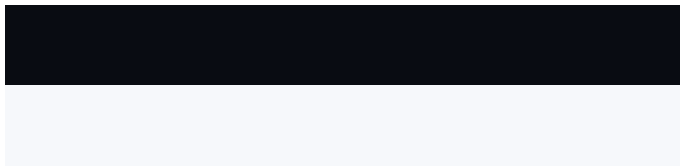
Google Cloud

Microsoft Azure

Component Rules

Shared component dimensions and behavior.

Component	Size	Variants
Button	32 / 36 / 40px	Primary, secondary, ghost, destructive, icon-only
Input	36px	Text, password, search, token
Select / Combobox	36px	Single, multi, searchable
Tabs	32px	Underline / segmented
Card	12px radius	Default, interactive, metric, warning
Table	36px row	Dense, selectable, virtualized
Badge	20-24px	Status, env, tag, role
Dialog	<=560px typical	Create/edit/confirm
Sheet	420-520px	Details, filters, quick edit
Toast	Auto	Success/info/error
Tooltip	Short	Clarification
Command palette	Centered	Navigation/actions
Skeleton	Content-shaped	Loading
Empty state	Purpose-built	First use/no data



Rule

No gradients; 16px icon; visible focus ring

Persistent label; errors below

Keyboard navigable

Never primary app navigation

1px subtle border; shadow only when elevated

Sticky header, filters, columns

Status uses text/icon + color

Destructive confirms name target

Use when context should remain visible

Never sole location of critical error

Never essential instruction

Cmd/Ctrl+K

Prefer to layout-shifting spinner

Explain next action; one CTA

Shipyard Screens

Domain-specific UX requirements.

Screen	Primary Content
Dashboard	Health, recent pipelines/releases, storage, runners/cluster
Project overview	Repo, latest commit, pipelines, artifacts, environments
Pipeline run	DAG, jobs, details, live logs
Job logs	Live terminal/log stream + search/timestamps/wrap
Artifacts	Browser, provenance, checksums, download
OCI image	Tags, manifests, platforms, layers, digest, pull command
Packages	Versions, coordinates, install snippet, metadata
Releases	Version, source commit, artifacts, environments
Deployments	Timeline, approvals, rollback, status
Runners	Fleet, capacity, labels, heartbeat, drain
Cluster	Control plane/dependencies/leader/lease health
Settings	RBAC, tokens, secrets, retention

Requirement

One-screen operational pulse
Persist project context in header
DAG informative; list equivalent
Preserve scroll and selection during live updates
Identify -> verify -> copy/use
Copy-first and provenance-first
Package-manager-aware commands
Source -> build -> artifact -> release -> deploy
Production context always explicit
Bulk operations for large fleets
Health first; topology secondary
Danger zone separated

Accessibility

Target: WCAG 2.2 AA. Accessibility is part of component correctness.

Area	Requirement
Contrast	Normal text $\geq 4.5:1$; large text $\geq 3:1$
Keyboard	All interactive controls operable
Focus	Visible and unobscured
Status	Never color-only
Labels	Persistent form labels
Motion	Reduced-motion support
DAG	Text/list equivalent
Live updates	Meaningful state announcements
Errors	Describe failure + recovery when known
Responsive	No inaccessible hidden overflow



Definition of Done

Verified in both themes

No traps; logical focus order

Sticky headers/overlays must not hide focus

Use icon + label + color

Icon-only buttons have accessible names

No flashing/pulsing failures

Keyboard-selectable nodes

Do not ARIA-announce every log line

Field errors programmatically associated

Mobile monitoring remains usable

Official Sources

Official/reference documentation used to ground the design-system choices.

Source	URL
shadcn/ui	https://ui.shadcn.com/docs
Tailwind CSS	https://tailwindcss.com/docs/theme
Lucide React	https://lucide.dev/guide/react/
React Icons	https://react-icons.github.io/react-icons/
Simple Icons	https://simpleicons.org/
React Flow	https://reactflow.dev/
xterm.js	https://xtermjs.org/docs/
Monaco Editor	https://microsoft.github.io/monaco-editor/
TanStack Query	https://tanstack.com/query/latest/docs/framework/react
Radix Accessibility	https://www.radix-ui.com/primitives/docs/overview/accessibility
WCAG 2.2	https://www.w3.org/TR/WCAG22/
Inter	https://rsms.me/inter/
JetBrains Mono	https://www.jetbrains.com/lp/mono/

Purpose

Open-code component foundation

Theme variables/design tokens

Primary icon system

Brand icon React integration

Popular brand SVG source

Pipeline DAG/topology

Browser terminal/log rendering

YAML/code editor

Server-state lifecycle

Accessible primitive behavior

Accessibility target

Primary UI font

Code/log font

Shipyard Design System

Status: **Normative draft**

Version: **0.1**

Product: **Shipyard**

Scope: Shipyard Web UI, embedded admin UI, OSS distribution, standalone/distributed/cluster modes.

This file is the repository-level design contract. If implementation and an informal mockup disagree, follow this file unless a design-system change is explicitly approved.

1. Direction

Industrial precision, subtle harbor identity. Shipyard is infrastructure software first. Use maritime language only when it improves meaning. Do not add decorative anchors, ships, ropes, wood textures, fake radar, or animated network maps.

Goals: - dark-first, fully supported light/system themes; - compact professional density; - excellent pipeline/log/artifact workflows; - status and failure clarity; - keyboard-first operation; - identical mental model from one-node installations to HA clusters.

2. Frontend stack

Concern	Standard
UI	React + TypeScript
Styling/tokens	Tailwind CSS theme variables + CSS custom properties
Components	shadcn/ui source-owned components; accessible primitives as needed
Product icons	lucide-react
Brand icons	react-icons/si only
Server state	TanStack Query
Pipeline DAG	@xyflow/react
Logs	@xterm/xterm
YAML/code editor	monaco-editor

3. Typography

- **UI / product:** Inter Variable
- **Code / logs / hashes / commands:** JetBrains Mono
- Do not introduce a third application font without a design-system change.
- Default app body: 14px/20px Inter 400.
- Default code/log: 13px/20px JetBrains Mono 400.

- Use tabular numerals for metrics, times, counts, and storage values.

Type scale

Token	Font	Weight	Size / line-height
Display	Inter Variable	700	36px / 44px
H1	Inter Variable	700	28px / 36px
H2	Inter Variable	650	22px / 30px
H3	Inter Variable	650	18px / 26px
Body LG	Inter Variable	400	16px / 24px
Body	Inter Variable	400	14px / 20px
Body SM	Inter Variable	400	13px / 18px
Label	Inter Variable	600	12px / 16px
Micro	Inter Variable	500	11px / 16px
Code	JetBrains Mono	400	13px / 20px
Code SM	JetBrains Mono	400	12px / 18px

4. Color tokens

Never place raw hex values in product components. Use semantic variables.

Dark

Token	Value	Purpose
--bg-canvas	#090C12	Primary app background
--bg-surface	#0F141D	Cards, panes, sidebar
--bg-surfaceRaised	#151B26	Menus, dialogs, elevated cards
--bg-surfaceHover	#1A2230	Hover state on dark surfaces
--border-subtle	#202938	Default separators and card borders
--border-strong	#344054	Focused structural borders
--text-primary	#F7F9FC	Primary text
--text-secondary	#B6C0CE	Secondary text
--text-muted	#7F8A99	Muted text; still AA on canvas
--brand-primary	#4DA3FF	Primary Shipyard signal blue
--brand-primaryHover	#76B8FF	Primary hover/active highlight
--brand-cyan	#22D3EE	Secondary signal / live data accent
--brand-teal	#3DD6B4	Success-adjacent operational accent
--focus-ring	#79B8FF	Keyboard focus ring

Light

Token	Value	Purpose
--bg-canvas	#F6F8FB	Primary app background
--bg-surface	#FFFFFF	Cards, panes, sidebar
--bg-surfaceRaised	#FFFFFF	Menus, dialogs, elevated cards
--bg-surfaceHover	#EEF3F8	Hover state
--border-subtle	#D8E0EA	Default separators and card borders
--border-strong	#AAB7C6	Focused structural borders
--text-primary	#111827	Primary text
--text-secondary	#475569	Secondary text
--text-muted	#64748B	Muted text
--brand-primary	#2563EB	Primary action blue
--brand-primaryHover	#1D4ED8	Primary hover/pressed
--brand-cyan	#0891B2	Secondary signal accent
--brand-teal	#0F8A72	Operational accent
--focus-ring	#2563EB	Keyboard focus ring

Status

Token	Value	Meaning
--status-success	#22C55E	Succeeded, healthy, online
--status-running	#3B82F6	Running, deploying, syncing
--status-queued	#A78BFA	Queued or waiting
--status-warning	#F59E0B	Warning, degraded, needs attention
--status-error	#DC2626	Failed, offline, destructive
--status-canceled	#64748B	Canceled, disabled, inactive
--status-info	#06B6D4	Informational state

Status must use **color + icon + visible text** in important contexts.

5. Spacing / shape

Base spacing unit: **4px**.

- `space.0` = 0px
- `space.1` = 4px
- `space.2` = 8px
- `space.3` = 12px
- `space.4` = 16px
- `space.5` = 20px
- `space.6` = 24px
- `space.8` = 32px
- `space.10` = 40px
- `space.12` = 48px

- `space.16 = 64px`

Radii: - `radius.sm = 6px` - `radius.md = 8px` - `radius.lg = 12px` - `radius.xl = 16px` - `radius.full = 999px`

Default card: 1px subtle border, 12px radius, no shadow. Popovers/dialogs may use elevation shadows. No glassmorphism in operational surfaces.

6. Icons

Product icons

Use `lucide-react` direct named imports. Default size 16px, stroke width 2. Sidebar 18px.

Group	Icon	Meaning
App / Navigation	<code>LayoutDashboard</code>	Dashboard / overview
App / Navigation	<code>FolderGit2</code>	Projects
App / Navigation	<code>GitBranch</code>	Pipelines / branches
App / Navigation	<code>Package</code>	Artifacts / packages
App / Navigation	<code>Container</code>	OCI / container registry
App / Navigation	<code>Rocket</code>	Releases / deployments
App / Navigation	<code>Server</code>	Runners / nodes
App / Navigation	<code>Network</code>	Cluster / topology
App / Navigation	<code>Settings</code>	Settings
App / Navigation	<code>ShieldCheck</code>	Security / policies
App / Navigation	<code>Users</code>	Members / teams
App / Navigation	<code>KeyRound</code>	Secrets / credentials
Global	<code>Search</code>	Global search
Global	<code>Bell</code>	Notifications
Global	<code>CircleHelp</code>	Help / docs
Global	<code>Command</code>	Command palette / shortcuts
Actions	<code>Play</code>	Run / retry pipeline
Actions	<code>Square</code>	Stop job
Actions	<code>RotateCcw</code>	Retry / rerun
Actions	<code>RefreshCcw</code>	Refresh / resync
Actions	<code>Copy</code>	Copy digest / command
Actions	<code>Download</code>	Download artifact
Actions	<code>Upload</code>	Upload artifact
Actions	<code>ExternalLink</code>	Open external resource
Actions	<code>MoreHorizontal</code>	Overflow menu
Actions	<code>Trash2</code>	Delete / destructive action
Status	<code>CircleCheckBig</code>	Succeeded / healthy
Status	<code>CircleX</code>	Failed / offline
Status	<code>LoaderCircle</code>	Running / loading (animated)
Status	<code>Clock3</code>	Queued / scheduled
Status	<code>TriangleAlert</code>	Warning / degraded

Group	Icon	Meaning
Status	CircleDashed	Pending / unknown
Status	Ban	Canceled / blocked
DevOps	SquareTerminal	Logs / shell
DevOps	FileCode2	Pipeline configuration / source
DevOps	GitCommitHorizontal	Commit
DevOps	GitPullRequest	Pull / merge request
DevOps	Tag	Tag / release tag
DevOps	Hash	Digest / identifier
DevOps	Database	Database / metadata service
DevOps	HardDrive	Storage
DevOps	Cloud	Cloud storage / remote service
DevOps	LockKeyhole	Protected / secret
DevOps	Fingerprint	Signature / provenance
Tables	ListFilter	Filter
Tables	Columns3	Column visibility
View	Maximize2	Fullscreen
View	PanelLeftClose	Collapse sidebar
View	Moon	Dark theme
View	Sun	Light theme
View	Monitor	System theme

Brand icons

Use `react-icons/si` for vendor marks only. Respect the vendor trademark/brand rules and the underlying icon license.

Brand	Import	Use
GitHub	SiGithub	Git provider / repository
GitLab	SiGitlab	Git provider / repository
Docker	SiDocker	OCI / Docker compatibility
Kubernetes	SiKubernetes	Kubernetes executor / deployment
PostgreSQL	SiPostgresql	Database integration / system status
Cloudflare	SiCloudflare	R2 / edge integration
Google Cloud	SiGooglecloud	Cloud integration
Microsoft Azure	SiMicrosoftazure	Cloud integration

Do **not** use multiple general-purpose icon families. No Font Awesome/Material/Heroicons for ordinary product controls.

7. App shell

- Sidebar: 248px expanded / 64px collapsed.

- Top bar: 56px.
- Desktop content gutter: 24px.
- Tablet: 16px.
- Mobile: 12-16px.
- Default control height: 36px.
- Dense table/toolbars: 32px.
- Prominent primary action: 40px.

Primary navigation: 1. Dashboard 2. Projects 3. Pipelines 4. Artifacts 5. Registry 6. Releases 7. Deployments 8. Runners 9. Cluster 10. Settings

8. Components

- **Button:** 32 / 36 / 40px heights. Variants: Primary, secondary, ghost, destructive, icon-only. No gradients; icon 16px; visible focus ring
- **Input:** 36px default. Variants: Text, password, search, token. Label above input; errors below; no placeholder-only labels
- **Select / Combobox:** 36px default. Variants: Single, multi, searchable. Use Radix/shadcn behavior; keyboard navigable
- **Tabs:** 32px compact. Variants: Underline or segmented depending context. Do not use tabs for primary app navigation
- **Card:** 12px radius. Variants: Default, interactive, metric, warning. 1px subtle border; shadows only when elevation is meaningful
- **Table:** 36px row default. Variants: Dense data, virtualized when large. Sticky header, selectable rows, filters, column controls
- **Badge:** 20-24px. Variants: Status, environment, tag, role. Always pair status color with text/icon
- **Dialog:** max 560px typical. Variants: Create/edit/confirm. Destructive confirms explicitly name the target
- **Sheet:** 420-520px wide. Variants: Details, filters, quick edit. Use when context should remain visible
- **Toast:** Auto. Variants: Success/info/error. Never the only place critical errors are shown
- **Tooltip:** Short only. Variants: Icon clarification. Not for essential instructions
- **Command palette:** Centered. Variants: Navigation + actions. Cmd/Ctrl+K; search projects, pipelines, artifacts, settings
- **Skeleton:** Match content shape. Variants: Loading. Prefer over layout-jumping spinners
- **Empty state:** Purpose-built. Variants: No data / first use. Explain next action; one primary CTA

9. Pipeline UI

- React Flow visualizes the canonical pipeline/job graph.
- Always provide a list/table equivalent.

- Node contents: job name -> status/duration -> runner/executor -> optional matrix key.
- Do not auto-recenter after every live update.
- Clicking a node opens details without losing graph context.
- Running uses blue + **LoaderCircle**; succeeded green + **CircleCheckBig**; failed red + **CircleX**; queued violet + **Clock3**; canceled slate + **Ban**.

10. Logs

- xterm.js for ANSI-aware rendering.
- Search, copy, download, wrap, timestamps, and auto-scroll pause are Shipyard controls around the terminal surface.
- Do not announce streamed log lines through ARIA live regions.
- Secrets must be redacted server-side before browser delivery.
- Very large logs must not freeze the route.

11. Artifacts and registry

Every artifact detail page prioritizes: 1. identity; 2. checksum/digest; 3. provenance; 4. producing pipeline/job; 5. source commit; 6. size/media type; 7. copy/download/pull/install action; 8. retention/policy/security metadata.

OCI image pages must expose digest, tag, manifest type, platform matrix, layers, annotations, size, producing build, and a one-click copyable pull command.

12. Runners and clusters

A one-node installation and a cluster use the same vocabulary. Cluster mode adds topology/redundancy detail; it does not use a separate UI paradigm.

Runner state must show online/offline, busy/idle, drain state, executor, labels, OS/arch, capacity, running jobs, heartbeat, and version.

Cluster views prioritize real health and dependencies over decorative topology. PostgreSQL/object storage dependency failures must be visually distinct from Shipyard node failures.

13. Motion

- Hover/press: 100-140ms.
- Menus/popovers: 120-160ms.
- Dialogs/sheets: 160-220ms.
- Sidebar: 180-220ms.
- Respect **prefers-reduced-motion**.
- Never flash/pulse failures for attention.

14. Accessibility

Target **WCAG 2.2 AA**.

Required: - normal text contrast $\geq 4.5:1$; - large text $\geq 3:1$; - keyboard operation for all controls; - visible, unobscured focus; - persistent labels for forms; - accessible names on icon-only controls; - no color-only status; - reduced-motion support; - text/list equivalent for DAGs; - errors associated with their fields; - critical state changes announced without noisy log announcements.

15. Token CSS

```
:root {{
  --bg-canvas: #F6F8FB;
  --bg-surface: #FFFFFF;
  --text-primary: #111827;
  --text-secondary: #475569;
  --border-subtle: #D8E0EA;
  --brand-primary: #2563EB;
  --focus-ring: #2563EB;
}}

.dark {{
  --bg-canvas: #090C12;
  --bg-surface: #0F141D;
  --text-primary: #F7F9FC;
  --text-secondary: #B6C0CE;
  --border-subtle: #202938;
  --brand-primary: #4DA3FF;
  --focus-ring: #79B8FF;
}}
```

16. Icon imports

```
import {{
  LayoutDashboard,
  GitBranch,
  Package,
  Container,
  Rocket,
  Server,
  Network,
  CircleCheckBig,
  CircleX,
  LoaderCircle,
  TriangleAlert,
}} from "lucide-react";
```

```
import {{ SiDocker, SiGithub, SiGitlab, SiKubernetes }} from "react-icons/si";
```

17. Repository structure

```
apps/web/src/  
  components/ui/  
  components/shipyard/  
  features/pipelines/  
  features/artifacts/  
  features/registry/  
  features/runners/  
  features/cluster/  
  styles/tokens.css  
  styles/theme.css  
  lib/icons.ts  
  lib/query.ts  
  lib/api/  
DESIGN.mmd
```

18. Definition of done for UI PRs

- ☐ Uses approved semantic design tokens.
- ☐ Dark and light themes are complete.
- ☐ Hover/focus/pressed/disabled/error/loading states are implemented.
- ☐ Keyboard workflow is verified.
- ☐ Status is not color-only.
- ☐ Empty/loading/error/permission/reconnect states exist where relevant.
- ☐ Responsive behavior checked on mobile/tablet/desktop/ultrawide.
- ☐ Machine identifiers are copyable in full.
- ☐ General UI icons are Lucide.
- ☐ Typography follows Inter / JetBrains Mono roles.
- ☐ Live updates preserve scroll, selection, and user context.
- ☐ Critical routes have E2E + visual regression coverage.
- ☐ Accessibility has automated + manual keyboard smoke testing.

19. Sources

- **shadcn/ui:** <https://ui.shadcn.com/docs> — Open-code component foundation and composition model
- **Tailwind CSS theme variables:** <https://tailwindcss.com/docs/theme> — Design tokens as theme/CSS variables
- **Lucide React:** <https://lucide.dev/guide/react/> — Primary React icon system; typed SVG components and tree-shaking
- **React Icons:** <https://react-icons.github.io/react-icons/> — Brand icon integration through ES module imports

- **Simple Icons:** <https://simpleicons.org/> — Canonical popular-brand SVG icon source used via react-icons/si where applicable
- **React Flow:** <https://reactflow.dev/> — Pipeline DAG / topology visualization
- **xterm.js:** <https://xtermjs.org/docs/> — Browser terminal/log rendering
- **Monaco Editor:** <https://microsoft.github.io/monaco-editor/> — shipyard.yml editor and diff/validation UI
- **TanStack Query:** <https://tanstack.com/query/latest/docs/framework/react> — Server-state fetching/caching lifecycle
- **Radix Primitives accessibility:** <https://www.radix-ui.com/primitives/docs/overview/accessibility> — Keyboard, focus, ARIA behavior underlying accessible components
- **WCAG 2.2:** <https://www.w3.org/TR/WCAG22/> — Accessibility conformance target
- **Inter:** <https://rsms.me/inter/> — Primary UI typeface; open-source UI-oriented family
- **JetBrains Mono:** <https://www.jetbrains.com/lp/mono/> — Monospace typeface for code/logs/digests